



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

SECP2523: DATABASE (WBL)

SEM I 2025/2026

PROJECT 3 - LOGICAL DATABASE DESIGN

SYSTEM: HASTA CAR RENTAL SYSTEM

Group Name: SpinTheWheel

Members:

- 1) Chun Yao Ting (A24CS0239) [Team Leader]**
- 2) Evelyn Ang (A24CS0068)**
- 3) Mirza As-Siddiq Bin Tohari (A24CS0112)**
- 4) Muhammad Aliff Bin Helmi (A24CS0272)**

Stakeholder: Hasta Travel & Tours Sdn. Bhd.

Representative Name	Date Interviewed
Encik Alif	20/11/2025

Printing Date: 03/02/2026

Table of Contents

1.1	Introduction	4
1.1.1	Overview of Company	5
1.1.2	Existing System	6
1.1.3	Proposed System	14
1.2	Overview of Project	35
1.3	Global Conceptual Data Model (ERD)	36
1.3.1	Updated Business Rules	36
1.3.2	Global Conceptual Database Design (ERD)	39
1.3.3	Updated Data Dictionary of Global Conceptual ERD	40
1.4	Logical Data Model (Logical ERD)	57
1.4.1	Logical Database Design (Logical ERD)	57
1.4.2	Updated Data Dictionary for the Logical ERD	58
1.4.3	Normalization	77
1.5	Relational Database Schema(s)	91
1.6	Data Definition Language (DDL)	93
1.7	Interface Design and SQL Implementation	101
1.7.1	Register & Login	101
1.7.2	Customer Homepage	103
1.7.3	Customer Activation	105
1.7.4	Manage Activation Request	108
1.7.5	Profile Page	112
1.7.6	Booking Process	114
1.7.7	Manage Booking	123
1.7.8	View Booking List	126
1.7.9	Pickup Process	127
1.7.10	Return Process	131
1.7.11	Staff Manage Return	138

1.7.12	Loyalty Stamps and Vouchers	142
1.7.13	Staff Dashboard	144
1.7.14	Customer List	147
1.7.15	Car List	151
1.7.16	View Car Availability	158
1.7.17	Rental List	163
1.7.18	Payment List	168
1.7.19	Delivery Location List	171
1.7.20	Voucher List	173
1.7.21	Staff List	179
1.7.22	Admin Dashboard	182
1.7.23	Late Return Scenario	188
1.8	Summary	193

1.1 Introduction

The Hasta Car Rental System is a web-based system intended to handle the car rental services between three primary actors: customers, staff, and admins. Customers can register their accounts, activate their accounts by submitting activation documents for verification, view available cars, make bookings by selecting dates and locations, and make payments by uploading receipts for staff verification. Finally, customers can complete the car rental process by submitting pickup and return documents, reporting any issues, providing feedback, and getting refunds of their deposits.

Staff are the operational actors who facilitate the processing of car rental services by verifying customer activation requests, approving or rejecting bookings, verifying payment receipts, managing pickup and return records, issuing refunds, and processing late return penalties. They also can update car information regarding fuel levels, mileage, and maintenance status based on customer return documents. On the other hand, admins are responsible for staff account management and reporting for monitoring car rental services, revenues, customer statistics, and booking trends.

Therefore, to support these workflows, the system will need a database with structured data that stores user and profile data, activation records, car fleet information, rental records, records of payments and refunds, loyalty stamps and vouchers, penalties and delivery locations. This logical database design will provide logical organization of the system data and reliability to support all the operation and reporting functions of the system.

1.1.1 Overview of Company

Hasta Travel is a company operating in Universiti Teknologi Malaysia (UTM), mainly providing their car rental service for UTM students and staff. Hasta Travel provides a highly customizable service for their customers such as delivery service for out-station pickup and return, booking outside working hours, short period booking within one day, cancel booking before one day of pickup and so on. Due to their small customer base, the company staff are still able to manage the operation of the company that highly relies on the manual processes and human judgment on eligibility verification, office-hour constraint and location charges. Although this manual approach allows flexibility and still can fulfil the current requirements, this operation mode will fall off once the booking volume increases as the company grows over the time. Additionally, Hasta also decided to transition from physical operation to mixed operation, where customers can do their booking offline or online. This means that the current operation mode already cannot address the future digitalized booking mode.

Overall, we can see there are many limitations in current business operation mode. Therefore, we need to implement a centralized relational database to store customer, vehicle, booking and transaction data in a systematic way, with clearly defined relationships between data and entities. This allows real-time updates in data, as well as reduces data redundancy by normalization, and also ensures data consistency and integrity for all operations. With this proposed solution on database, efficient querying of data and transaction handling can be fulfilled, which can reduce staff workload and overall service quality for customers.

1.1.2 Existing System

a) Start Booking

The current operation mode for booking in Hasta Travel & Tours Sdn. Bhd. is mostly manual, meaning that staff have a heavy workload. To book a car, customers had to contact Hasta Travel & Tours Sdn. Bhd. by WhatsApp message. Then, the staff will generate an immediate response to prompt the user to fill in the information such as are you a UTM student or staff, date, time, and location for pickup and return purpose, and finally the destination. It also notified the customers about the rule of minimum booking of 2 days for destinations outside Johor. This first message is for staff to check the availability of the car. Next, after the customer fills in the information, staff have to confirm the customer is a UTM student or staff, check whether the pickup location and return location is in UTM Student Mall, and check whether the pickup and return time are within office hours. (9am-5pm) If the customer is not a UTM student or staff, the booking cannot be continued. If the location is outside UTM or the time is not within working hours, it requires the staff on duty to determine how much charge will be given for the customer, such as delivery fee.

Then, after checking the availability of the car through the availability whiteboard or through Wahdah, the staff will inform the customer about the available cars and their respective price. Next, after confirming the car selected, staff will prompt the customer again to fill in the personal details such as IC or passport number, phone number, matric number, race, college, faculty, email. The message also gives deposit refund details and requires the customer to give a payment receipt for the deposit of RM50. If the pickup or return location is outside UTM, and pickup or return time is out of working hours, the PIC will be given as well, and staff will notify the customer of the extra charge as well. Next, staff will verify the total amount and whether the payment date and time in the receipt is aligned with the bank statement. After deposit verification, customers need to send the front and back picture of IC/Passport, driving license, and matric card with white background. After verification of documents, the customer side procedure is completed, and the staff will mark the date, pickup time, return time and customer name in the availability whiteboard. Next, the staff will manually key in all the information at Wahdah staff system, then finally the rental information is recorded. To record the new booking, they also store the rental information in daily sales reports in Excel format. This means that staff also need to manually key in all the required information.

b) Car Pickup

To handover the car key during pickup, first the staff will update the booking information if it is needed. (e.g. change the status from “Unpaid” to “Paid”) Then, they need to download an inspection form and agreement form, which will be automatically filled with all the information recorded in the Wahdah system. If the pickup is via online, the document will be sent to the customer for checking and signature before 30 minutes of booking time, with the pickup and return procedure message attached. Customers will sign the form and send it back to Hasta Travel through WhatsApp, with the picture of the cars and payment receipt attached. Afterwards, staff have to print the inspection form, agreement form, and verification documents to store as hardcopy. They also need to manually write down the mileage, fuel level, date, time and PIC, as this information is not automatically filled in the inspection form generated. If the pickup is physical, staff just need to print out the form and ask the customer to fill in on spot and have the inspection session physically. The sales report is also updated.

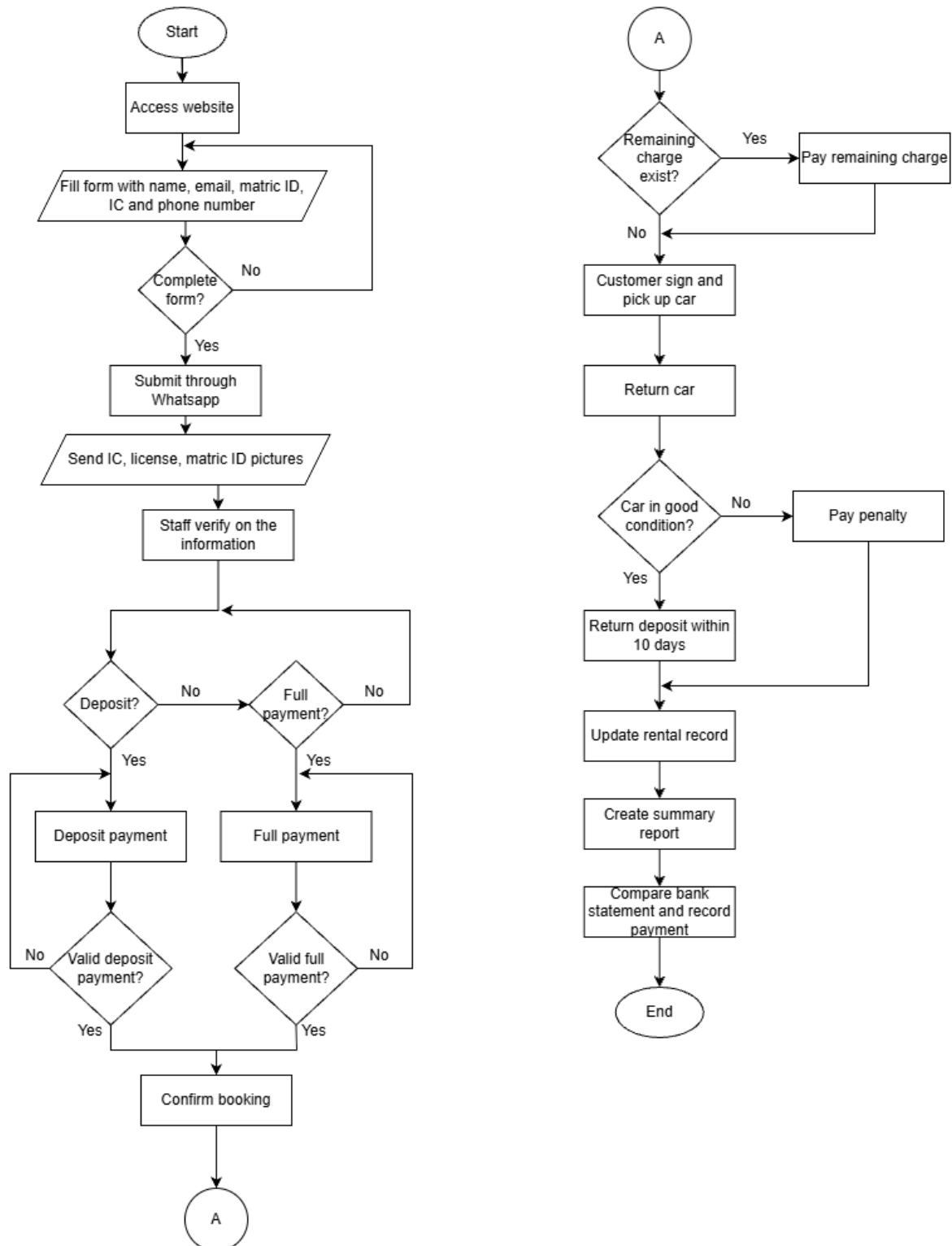
c) Return Car

To track the return status of the car, Hasta Travel uses Tracksolid Pro to track the location of the car, as they had their GPS installed on the rented car. Staff will select the car they wish to track, with the return date and time. Then, the location of the car will be shown. After the customer returns the car, staff will record the mileage and fuel level after booking. If the return is late, the return penalty will be counted starting from 30 minutes of return time, and staff will put remarks on the agreement hardcopy form. If there is damage to the car, staff will enquiry customers on the reason of damage, with the comparison of the car picture before booking. Some penalties could be imposed on customers. If there is no issue, staff will send the car picture after returning to the company, and the deposit will be refunded in 10 days. After returning the car, the information is also updated in the Excel sheet of the sales report.

d) Loyalty Card Feature

The loyalty record will be recorded by staff in the Excel sheet. To get the first stamp, the customer had to book at least 12 hours for any type of car. To get the stamp afterwards, the customer had to book at least 9 hours for any type of car. After giving the loyalty card to a customer, record the customer’s name according to the card’s serial number. Mark tick in the Excel sheet and write down the date and the name of the person in charge. If the customer rents again for the next 9 hours and reached the box that provides a discount, mark as “Claimed”.

1.1.2.1 Current System Workflow Flowchart



1.1.2.2 Comparison with Existing System

	Existing System	drive.my	GoCar
Verification Form	Customers are required to send a copy of their IC/Passport, driving license, and student/staff card 3 days before their booking session. The verification is done by the admin through WhatsApp manually.	There is no verification done on this website.	After signing up an account, the system requires customers to verify their identity by uploading their IC, driving license and a selfie of the customer.
Registration form	In the existing system, personal information is given through WhatsApp, and the admin will book the session. There will be no account created for the customer at this point.	A simple registration consists of username, full name, email address, password, and their agreement with drive.my terms and privacy policy.	A simple registration consists of full name, email address, password, and their agreement. It also has a direct sign-up option for google accounts.
Booking form	All bookings will be confirmed by the admin manually through WhatsApp. For pickup and return done within office hours (9am - 5pm), and pickup and return point at UTM Student Mall, there will be no additional charges. However, for bookings done outside office hours, or pickup and return point other than UTM Student Mall, a staff member will be needed to handle the customer. The booking cannot proceed until there is a staff member available to handle the customer. The admin will confirm with the staff on duty the amount to be charged and inform the customer about the applicable charges.	After filling up pickup, return location, and the date, the list of cars available can be chosen by the customer. After choosing the designated vehicle, there's a list of add-ons that can be made by the customer. The info of pickup and return will be shown as well as the total of payment of the booking alongside it. After confirming the price, the customer can proceed with the payment.	The customer needs to choose their booking dates and time. After filtering the dates, the available car with a price can be chosen by the customer. The chosen car will be shown where we need to pick it up, how long the duration of the booking along with its price.

Payment form	Customers need to inform the booking price of the car to the staff incharge. The customer needs to pay an additional RM 50 as deposit to the staff. The online payment by the customer needs to come with a receipt from the bank as proof of payment and it will be put on Wahdah by the staff manually. If the customer pays with cash, the staff needs to bank the cash inside his/her own account and immediately transfer it to the HASTA account. This transaction needs to be with a receipt as it will be attached inside DSR.	The customer needs to enter their billing information such as their full name, email address, address, and preferred payment method (Direct bank transfer or Payex). If the customer chooses direct bank transfer, the system will send an email, and the booking will be on-hold until the payment has been received. If the customer chooses Payex, they will have four different options to choose from (card, online banking, eWallet, and instalment).	After selecting the car, customers are required to choose one of Collision Damage Waiver(CDW) options and one choice of flex package. The CDW will affect the amount that needs to be paid when a collision occurs during your ride. The flex package includes the basic package and Premium Flex. Premium Flex package will provide the customer flexibility to return the car 30 minutes late, refund for cancellation via GoPocket points, and unlimited change of booking date and time. The total payment consists of subtotal, CDW, flex, and tax. There will be no deposit needed. The customer can choose their preferred payment method (GoPocket, credit/debit card, online banking, and eWallet).
---------------------	---	--	---

Deposit payment	After filling up the booking form, the customer needs to pay forward the deposit payment. The payment must include a receipt that has the same date as the day the payment was made, and the deposit amount must be correct. Deposit payment is a must as it ensures the booking can be made. The deposit will be returned in 10 working days if: • No damage to the car & the car is in good condition, same as it was. • Off returned same as given. • No summons during your rental • Car is in clean condition. • Return on time.	Customers are required to pay a refundable deposit depending on which car model. The deposit prices range from RM300 to RM 1000.	No deposit needed to be paid.
Filter features	No filter features.	These filters help customers to find suitable price ranges, vehicle categories and vehicle brands that suit the customer preference.	This app allows customers to filter the car type, number of seats, and model that they want. This will improve the usability of the application for the customer experience.

1.1.2.3 Problem Statement

Based on the case study, we can observe that Hasta Travel still uses a fully manual and fragmented workflow in the company operation, such as the process of booking, pickup, return and customer management. This had caused a heavy workload on the staff side, resulting in operational efficiency, and increasing risk of human error. Meanwhile, for the customer side, the booking, pickup and return process is also inefficient, which affects the customer experience and brings a negative effect on company image.

Currently, the booking process fully relies on WhatsApp communication, which requires staff to repeatedly prompt customers to send information, manually verify their eligibility to book, check booking constraints such as location, working hours and minimum rental period, as well as determine additional charges for different types of special cases. Meanwhile, customers also have to manually send the details, and their details are not stored, meaning that they still need to send the send information again during the next booking, which is unnecessary and complicated for the customer booking process.

Besides, staff even need to check the car availability to approve the booking by using a physical whiteboard and a separate staff system which is Wahdah. They also need to manually record down booking details across different platforms, including WhatsApp, Wahdah, physical whiteboards, notebook, and Excel daily sales report. This is a bad practice for data transactions, as the data redundancy, delays and errors are likely to occur.

Furthermore, staff also must manually generate, print, fill, verify and store inspection and agreement forms signed by customers during pickup. Important information such as mileage, fuel level, pickup time and staff in charge are required to be filled in by staff manually, as these details are also not automatically inserted. The document is also stored in an unorganized way, as some of them will be in digital documents, some of them will be manual, so it is hard to track the legal documents if needed. Both online and physical pickups require human intervention, like document handling and updates to sales records repeatedly.

Additionally, staff also depend on external GPS tracking systems as well as manual inspection to record mileage, fuel level, late return penalties, and damages. Any issues during rental such as delays or damages will require documentation, customer communication and penalty calculation, all by manual work from staff. Other than that, deposit refunds are also tracked manually, and return information also needs to be recorded into Excel. This provides no flexibility to customer and staff, and also contributes to repetitive workload for staff.

Overall, the absence of integrated and automated booking and management system causes excessive manual data entry across different platforms, high dependency on staff accuracy, judgment and availability, long processing time for booking and transactions, high risk for data inconsistency, lost records and human error, poor customer service due to long response time and repeated information request, and no scalability to address future growth in company size and booking volume. As Hasta also wishes to have a more flexible workflow by transiting to off-station booking, this requires a full revolution in the current system.

1.1.3 Proposed System

1.1.3.1 Mission Statement of TO-BE System

1. Digitalize and Automate Workflow for Each Booking.
2. Reduce Manual Workload and Repetitive Data Entry.
3. Centralize Data Management in Organized and Systematic Database.
4. Enhance Booking Accuracy and Customer Experience.
5. Improve Task Efficiency and Operational Control of Staff.
6. Support Future Scalability for Business Growth.

1.1.3.2 Overview Description

To improve the current system workflow, we came out with our proposed solution which digitalizes the car rental system and organizes the data systematically in the database. We proposed the system to have both the customer side and staff side. The customer side is where the customers do the booking, while the staff side is for the staff to access and manage all the bookings. Next, we will explain the overall workflow of our proposed system.

First, as currently customers need to fill the information again and staff need to record the information again during each booking, we propose a user management system where staff and customer information will be stored in their respective accounts. This will be covered in the authentication module which include register, login, activation, and access control. Staff and customer will use the same login page but will be redirected to staff side or customer side based on their respective role.

To register, the customer has to fill in their full name as per NRIC, email, and their password, and each email should be unique, meaning that email cannot be reused for different accounts. After registration, customers can login into their account by entering correct credentials, but they still cannot do booking because their account is not activated yet. To activate an account, customers have to fill in all the information required such as IC/passport, nationality, UTM ID (Staff/Student), contact, emergency contact, emergency contact relationship, license number, license expiry date, college, college block, faculty, and home address. Afterwards, they also have to submit a front & back picture of IC/passport, UTM matric card, and car license. The submitted information will be sent to the staff side for manual verification. If staff notice any issue, they can decide to reject it with a reason, else they can approve the activation and customers can start booking now. For activation requests that are rejected, customers may resubmit the activation request and wait for staff approval. Additionally, the system will also send email to notify successful or unsuccessful activation.

Now, we propose the digitalized booking process, which is much easier and more convenient than the current booking process. First of all, customers need to select the preferred date, time, and location for both pickup and return. The system will filter based on the time range of booking and display available cars during that period, with the car model, car plate, features and price. Then, customers can select the preferred car and proceed to the payment page. The payment page will have the payment breakdown section to display all the costs such as rental cost, deposit of RM50, and delivery fee (if the pickup location and return location is not in UTM). Customers also could use vouchers to gain discounts and the discounted price will be shown as well. As the stakeholder does not want to implement the online payment gateway, hence payment screenshot is still used for payment submission. Customers can either pay by QR or using Hasta bank details shown in the payment page. Last but not least, they need to submit the receipt or payment screenshot for verification purposes.

Currently, the booking is under pending status. So, staff will review the customer booking and check if the payment receipt is correct or not, and double check the availability of the car. If there is no issue, staff can approve the booking, else they can reject the booking by stating the reasonings and issue the rejection refund by submitting a refund receipt. Customers will also receive email for booking approval and rejection. If customers wish to cancel their booking, they can do so before 24 hours of pickup time. Then, staff need to review it and issue the cancellation refund by submitting a refund receipt.

After booking approval, customers can do pickup before 30 minutes of pickup time, and there will also be email reminders to notify customers to pick up their car. There will be a few steps to finish the pickup process. First, customers have to review the booking details. Secondly, customers need to download the inspection form and agreement form, fill it, and submit the filled form in the system. Thirdly, customers need to take 5 pictures, which are front, back, left, right and odometer of the car and submit it to the system. Finally, customers can finish the pickup by clicking submit, and the rental status will become picked up.

To return the car, customers can issue a return before 24 hours of return time, and customers will receive an email reminder as well. There will be some steps to complete the return as well. First, customers need to review the booking details. Secondly, customers have to submit 6 pictures, which are front, back, left, right, odometer and car key of the car. Thirdly, customers have to submit their feedback, if there is any issue during booking, customers can press the tickbox in the page as well. Fourthly, customers need to fill in their bank details which are cardholder name, bank name, and account number for deposit refund later on. They may use an existing bank detail if they stored the information in their profile. Finally, customers can submit the return form, and the rental status will change to returned.

After the car returns, there are few tasks that need to be done by the staff. First, staff need to check the feedback of customers for this booking. If there is any issue, the car will automatically become maintenance status. Only after the staff solve the issue (e.g. Aircond no function), staff can click “Resolve” and the car will be available. Secondly, staff need to update the mileage and fuel bar of the cars based on the odometer submitted by the customer. Finally, the booking is finally completed after staff issued the deposit refund by submitting the refund receipt.

For late return, the status of booking on the customer side will be changed to late return after 30 minutes of original return time. The penalty imposed will be based on the return time, for every hour after 30 minutes of original return time will be charged for RM50. Customers have to complete the return process first. Afterwards, their account will be directly blacklisted and there will be one pop out to ask the customer to pay the penalty first. Customers cannot do any new booking as their account is suspended. After they submit the payment receipt for penalty, they still need to wait for approval and their account remains suspended. So, the staff will check the payment receipt submitted by the customer and approve it if there is no issue. After that, customers will remove from the blacklist and can do booking as normal.

Now, we will explain our proposed solution to manage vouchers. There will be two methods of gaining vouchers, one generated automatically by system and another one by staff doing voucher assignments to customers. For automatically generated vouchers, it can be generated in two ways, the first method is new customers can receive one NEWUSER10 discount, and the second method is claimed by loyalty cards. For loyalty card features, the customer will receive one loyalty stamp after return of each booking if the booking exceeds 12 hours. After collecting three stamps, they may claim the LOYALTY10 vouchers, which they can enjoy the discount on their future booking. For manual vouchers assignment, this only happens if there is any issue during booking and rewards for loyal customers. So, staff can also assign the vouchers to their account. On the staff side, they are also able to view the voucher's usage rate and history.

Furthermore, there is also car fleet management on the staff side. Staff is able to view the car details in the car list tab, such as the car plate, car model, price rate, features, last booked time and its status. Staff can directly manipulate the status using a dropdown, and also be able to edit the car details such as car picture and price. Staff is able to add new cars as well if there are any new leases in future. They will need to fill in the car plate, car model, car picture, price rate for 1 hours, 3 hours, 5 hours, 7 hours, 9 hours, 12 hours, and 24 hours. Staff are also able to view the past rental records of the cars for tracing purposes.

Additionally, there is customer management on the staff side. Staff can see all the details of each customer, such as their college, faculty, contact number, email, and status, and also can search for certain customers by input substring. The status of the customer also can be changed by dropdown, such as new, active, blacklisted, inactive. The customer is managed using soft delete instead of hard delete by setting the customer status to inactive so that our stakeholder can trace old customer information.

Moreover, there is rental management on the staff side. Staff can view all the rental details such as status, customer, car plate and booking period. Staff also can filter searching by customer name, status, car plate, month, or year. Meanwhile, admins can generate a report of rental by the filter as well. There is also a list of locations offered for pickup and return, and staff can view the delivery price for each location and be able to edit the price as well. Staff can also add a new location by filling in the location name and its delivery price.

Other than that, there is also payment management on the staff side. Staff can view all the transactions done by the customer, with the amount, rental duration and the receipt submitted. Certain transactions also can be filtered based on the customer's name, rental ID, year, or month. Meanwhile, admins can also generate a transaction report by the filter as well.

Now we move to the admin side. Admins have the privilege to view all staff and add new staff by inserting their information such as full name, email, position and password. Admin also have the privilege to view analytics dashboard which show the graph of booking, revenue, customer gain, customer distribution by faculty and college and booking heatmap, filter by last 7 days, last 12 months and last 5 years.

1.1.3.3 Proposed System Flowchart

This section presents the proposed system workflows through flowcharts. An overall system flowchart gives a comprehensive view of the entire process flow, while six sub-flowcharts detail specific processes such as Activation, Booking, Pickup, Return, Staff Issue Return, and Late Return. These flowcharts are used to show the step-by-step procedures and decision-making processes within the system.

Figure below shows the main workflow of Hasta Car Rental System, starting from process user registration and login until the booking completion. It illustrates the main path for user roles including customer and staff and shows how different processes connect within the system.

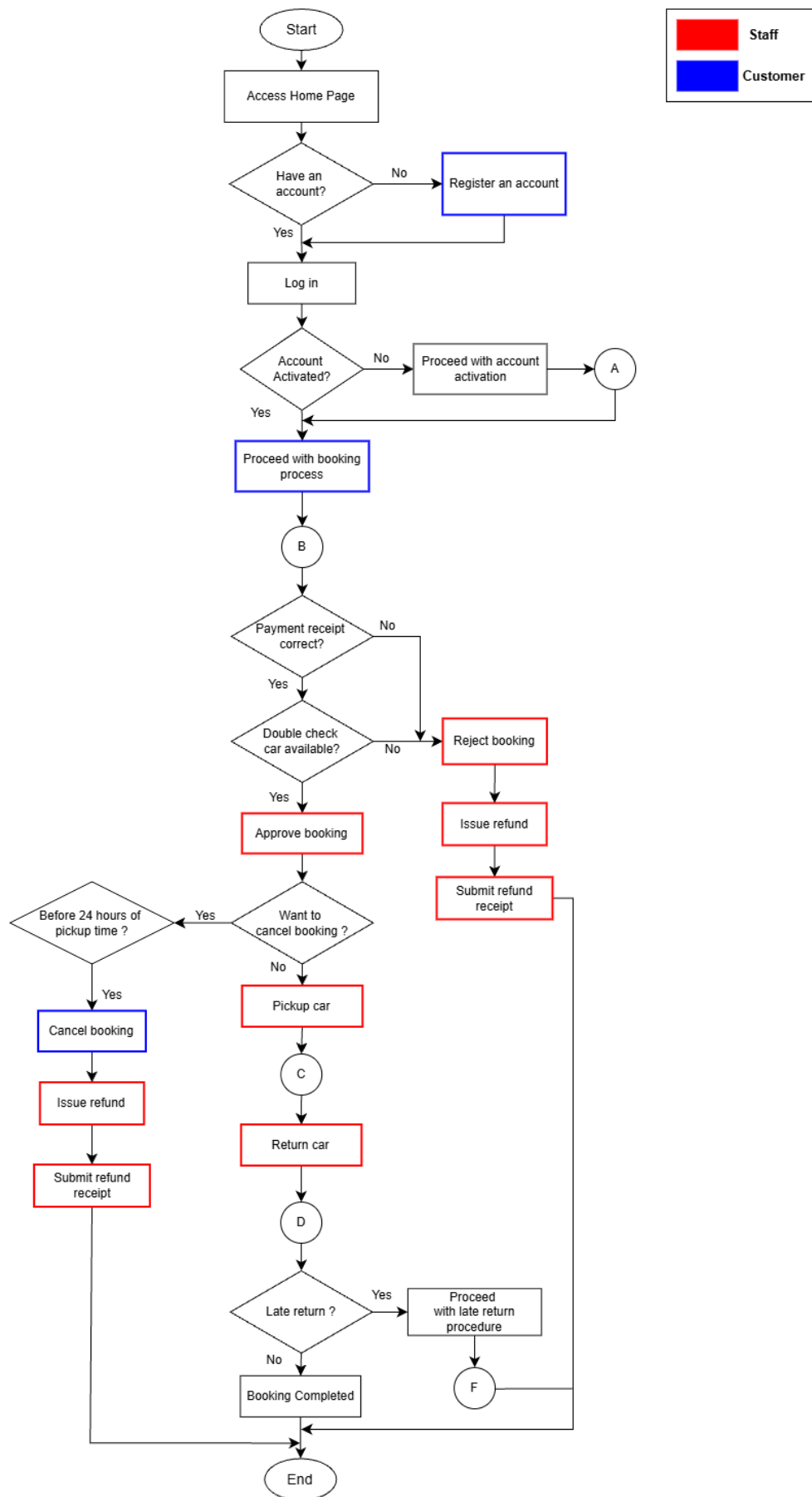


Figure below demonstrates the details of the account activation process. Customers need to provide all the personal information and submit required documents for staff to review them. Staff will approve or reject the activation after verifying the documents. Upon approval, customers can get full access to make booking. If the activation is rejected, customers can review the disapproval reason and submit a new activation request.

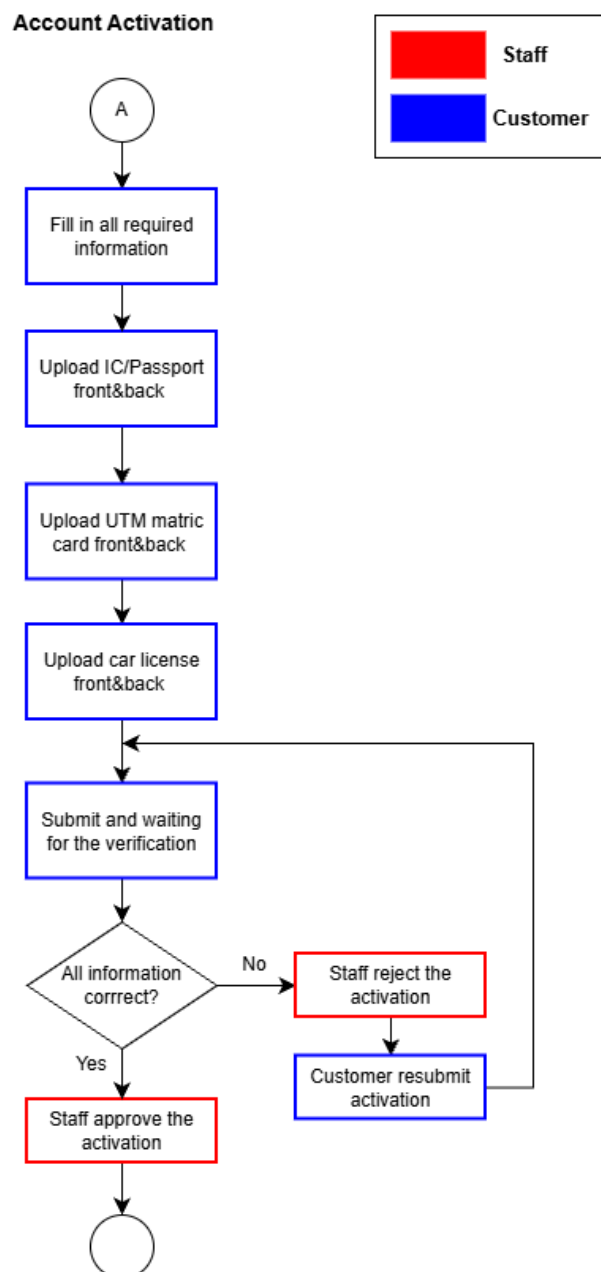


Figure below shows the booking process flows starting from select pickup and return details. After that, customers can select available cars that are filtered based on the booking details. A delivery fee will be required if the pickup or return or both are outside of UTM. Then customers can choose to use a voucher if available and complete the payment. The submitted booking needs to wait for staff to verify the payment and approve the booking.

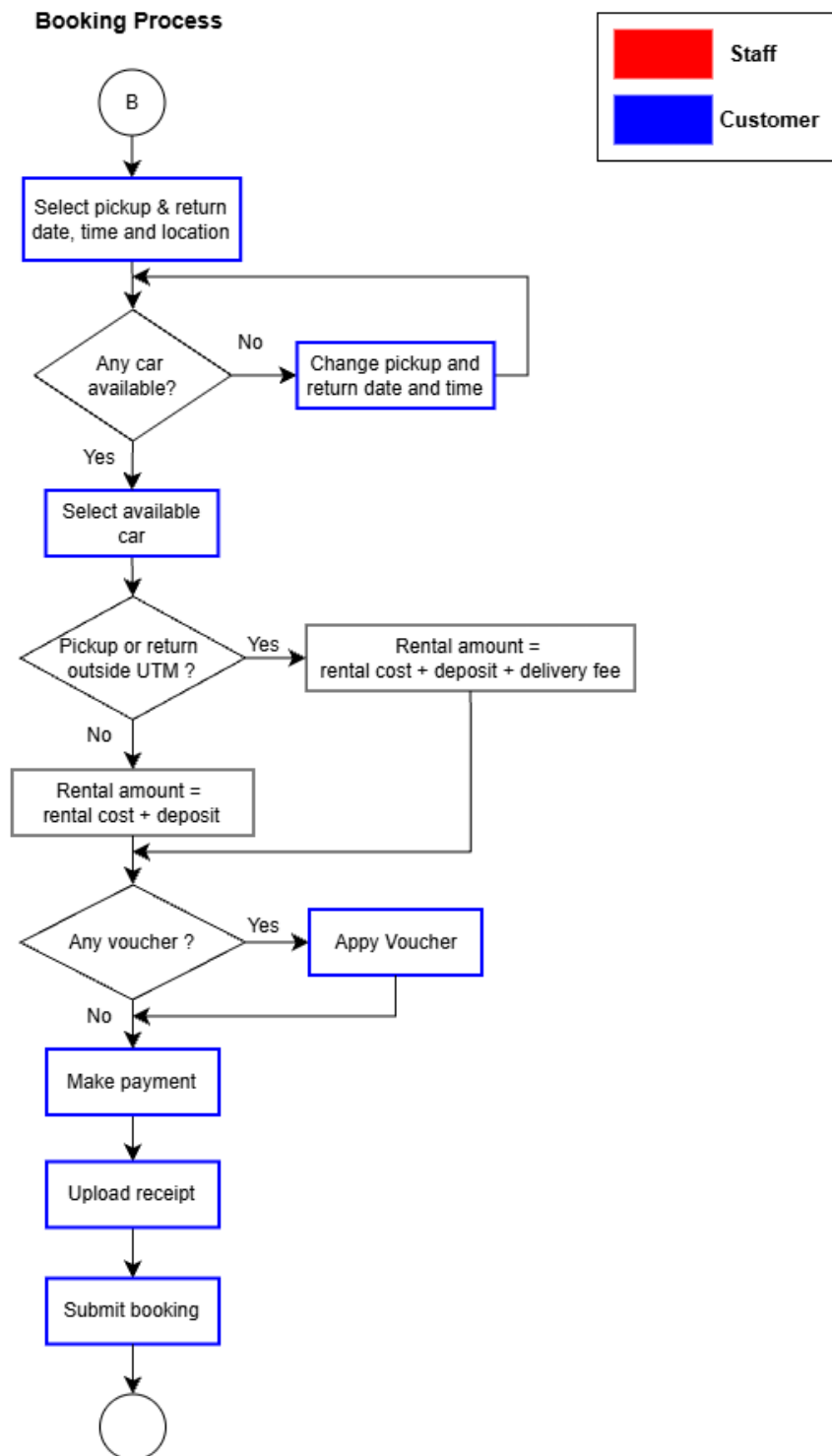


Figure below shows the car pickup process where customers can only pickup 30 minutes before the scheduled pickup time. Customers must download the agreement and inspection form, sign them, and then upload back to the system. Next, they need to upload five car photos including front, back, left, right view and the car odometer before submitting the form.

Pickup Process

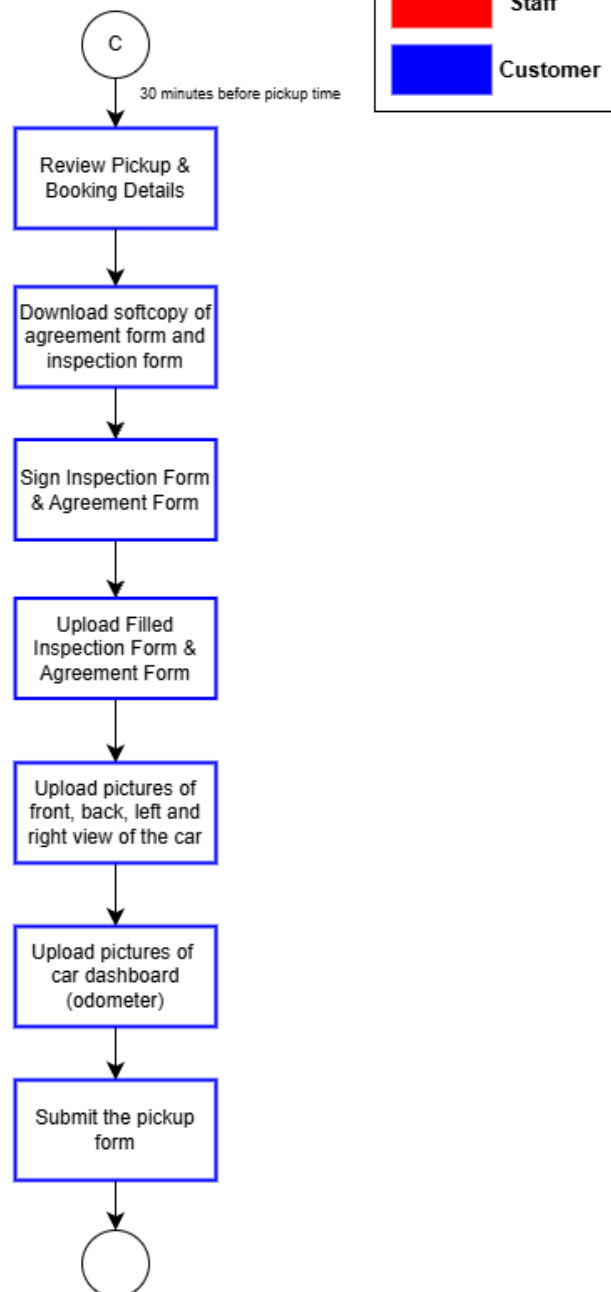


Figure below outlines the on-time car return process where customers need to upload six photos (including photo of car key), report an issue if faced any problem during rental, provide feedback and provide bank details in order to get the deposit refund. After the car return, there are some tasks needed to be completed by the staff including checking customer feedback, resolve issue if any, update car mileage and fuel bar, as well as issue deposit refunds.

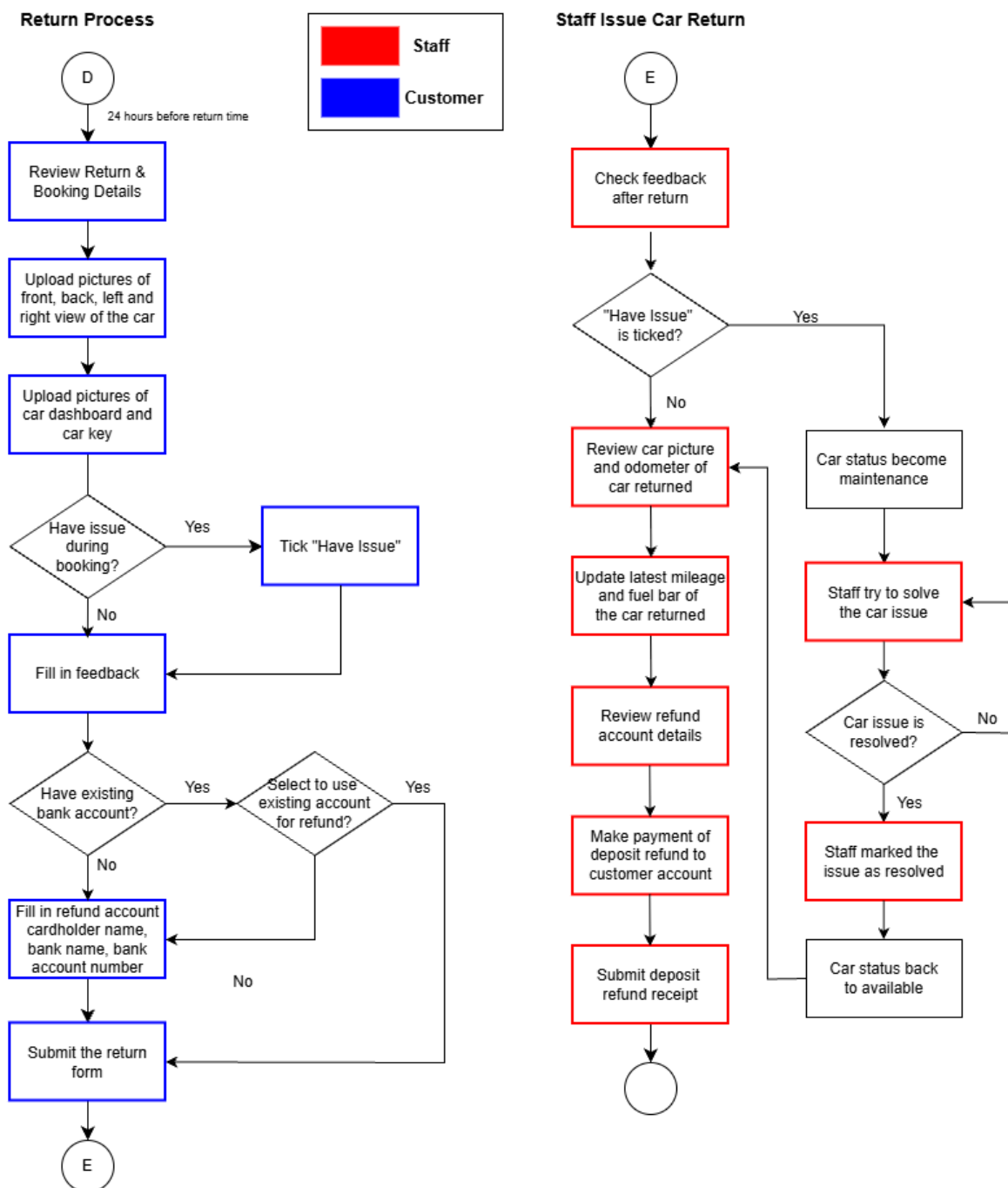
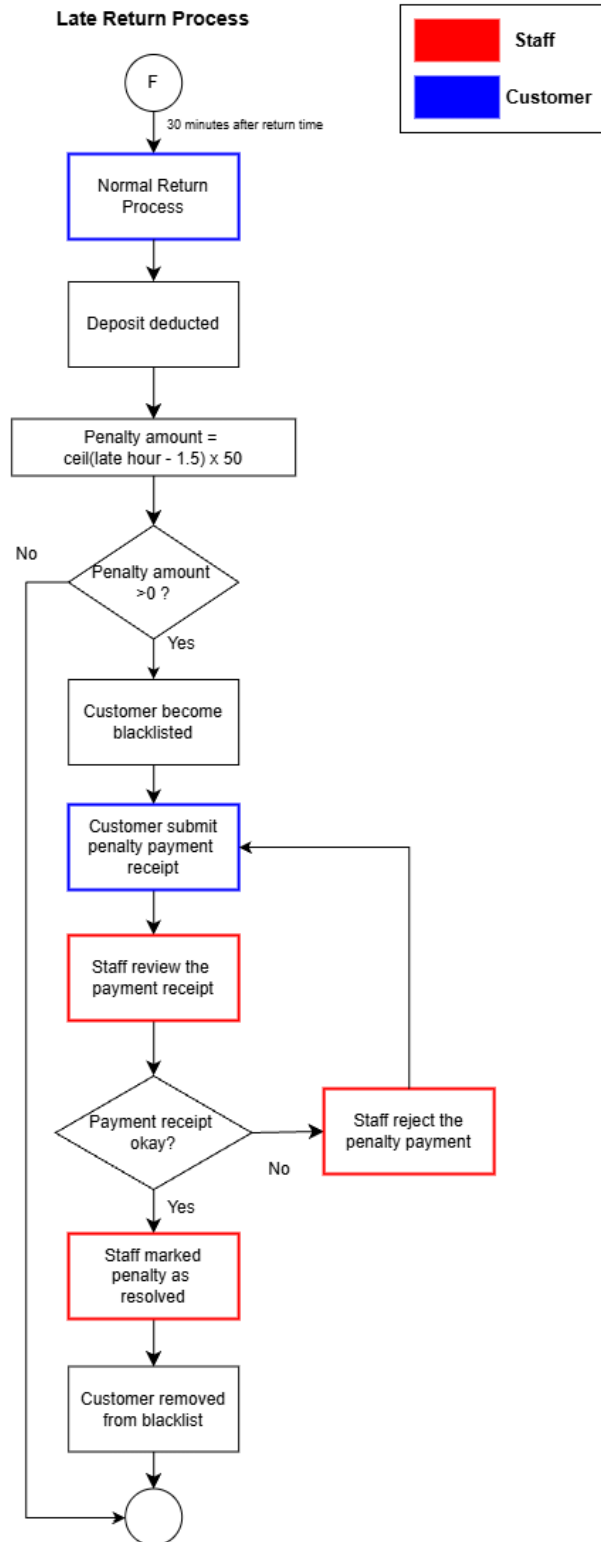


Figure below shows the late return process if the car is returned more than 30 minutes after the scheduled return time. Customers will be charged a penalty of RM50 per hour and their accounts will be automatically blacklisted. This causes them to not do any booking as their accounts are suspended until the penalty payment is approved by the staff.



1.1.3.4 Proposed System Sitemap

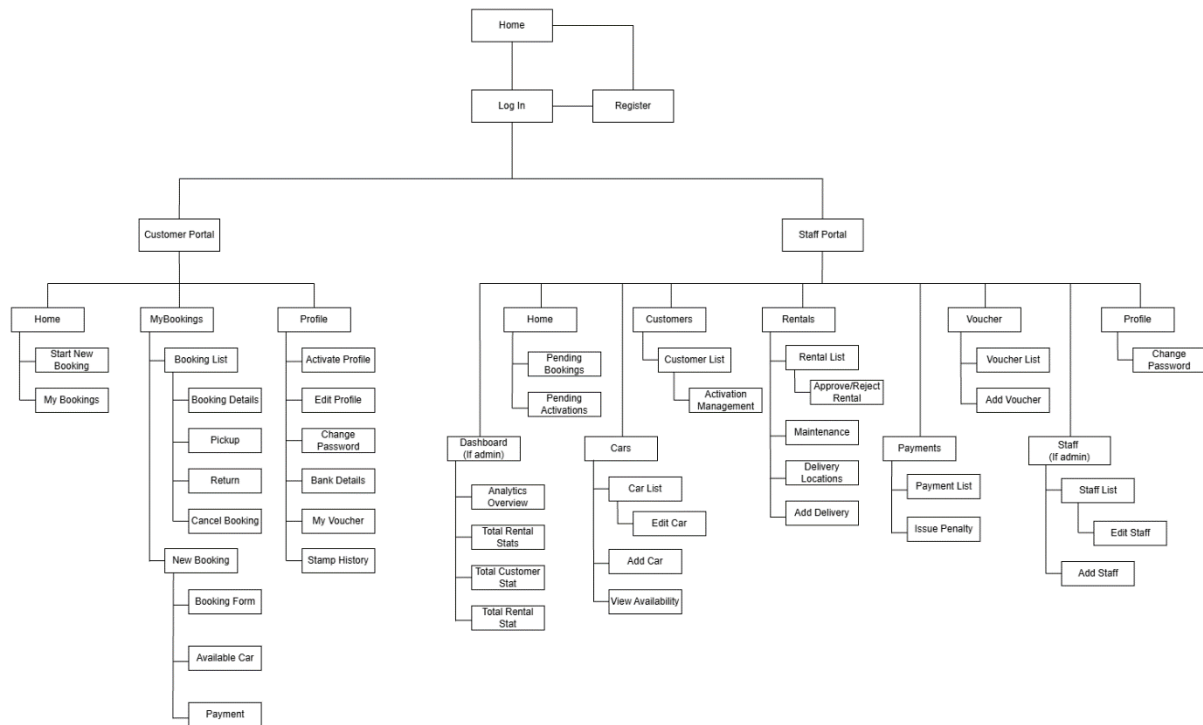


Figure above is the sitemap that shows the hierarchical structure and navigation flow of the Hasta Car Rental System. It shows all the accessible pages organized by user roles which are Customer, Staff and Admin.

1.1.3.5 Use Case Diagram

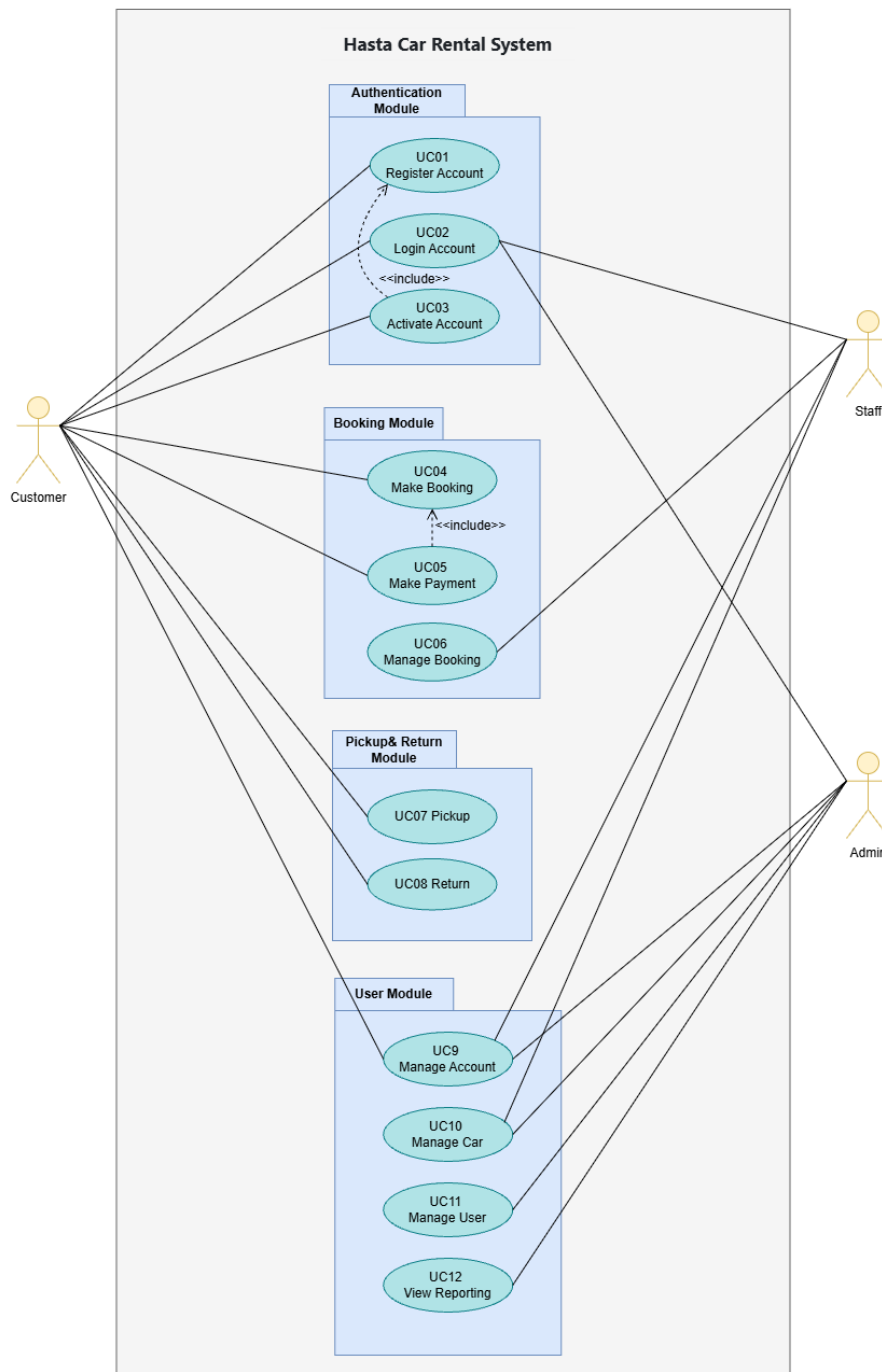


Figure above shows the use case diagram for the Hasta Car Rental System with three actors: Customer, Staff, and Admin. Customers can register, login, activate accounts, make bookings, submit payments, pickup and return vehicles, and manage their accounts. Staff handle account activation, booking management and car management. Admins manage users, cars, and view system reports.

Use Case	Function	Description
UC01	Register Account	This use case allows customers to sign up as a user for the system.
UC02	Login Account	This use case allows users to log in to the system using their email and password.
UC03	Activate Account	This use case allows customers to activate accounts by submitting required documents then staff reviews and approve activations.
UC04	Make Booking	This use case allows customers to create a booking by specifying pickup and return details and selecting available cars.
UC05	Make Payment	This use case allows customers to complete payment by uploading the payment receipt and submitting booking.
UC06	Manage Booking	This use case allows staff and admin to view all bookings, approve or reject the bookings.
UC07	Pickup	This use case allows customers to process car pickup by completing the agreement and inspection form and capturing pickup car photos.
UC08	Return	This use case allows customers to return the car by capturing return car photos including car key photo, providing feedback, and reporting an issue that occurred during the rental period (if any).
UC09	Manage Account	This use case allows all users to view their own profile information and change account password.
UC10	Manage Car	This use case allows staff and admin to add new vehicles to the rental fleet and update existing car information.
UC11	Manage User	This use case allows the admin to view all user accounts in the system, create new accounts for staff and update user status.
UC12	View Reporting	This use case allows admin to view comprehensive reports including rental statistics, revenue reports and customer analysis.

1.1.3.6 System Scope & Boundary

The system boundaries define the scope of the Hasta Car Rental System by differentiate the internal functions that are automated and handled within the system, and also the external operations that must be performed manually or outside the system. This explanation would provide a clear picture of capabilities of the system, limitations of the system and the responsibilities of the users.

1) Internal Scope

i) General access for all users

Page	Function
Register	• Customer account registration with form validation • Automatic email uniqueness check • Automatic NEWUSER10 voucher for first-time customers
Login	• User authentication for all roles including Customer, Staff and Admin • Role-based redirection to appropriate portal

ii) Customer Portal

Page	Function
Home	• Overview of account status and activation progress • Quick access to create booking function • Display of important notifications and reminders
Booking List	• View booking history and current bookings • Access to booking details including car and booking information, payment summary and booking timeline • Cancel booking

Make Booking	• Create new booking • Apply Voucher • Upload payment receipts • Receive email notifications once booking approved
Pickup	• Download agreement and inspection form • Upload signed forms • Upload 5 car photos (front, back, left, right, odometer) • Can upload additional 1 photo if found damage on car • Receive email reminder for pickup time
Return	• Upload 6 car photos (front, back, left, right, odometer, car key) • Report any issue encountered during rental • Submit feedback on rental experience • Provide bank details for deposit refund *For late return: • View penalty amount • Submit penalty payment with the receipt • Wait for staff approval
Profile	• View and edit personal information • Submit activation request if new user • Save bank detail for return use • View available vouchers • View stamps history • Change account password

iii) Staff Portal

Page	Function
Dashboard (Admin only)	• View rentals, customers, revenue analytics with time period filters • View booking trends graph, income growth chart, customer growth chart, customer distribution by faculty and college, and booking heatmap
Home	• Overview of pending tasks • Quick access to all management functions
Car	• View complete car list with details • Add new car • Edit car details • Change car status via dropdown menu • View all rental history for each car • Check car availability for specific dates
Customers	• View all customer list • View pending activation requests • Review request and choose to approve or reject the activation • Change customer status via dropdown
Rentals	• View all rental records • Access pending rentals for approval • Review all booking details and verify receipt • Approve or reject bookings • View car flagged for maintenance • Check customer feedback to manage issue • Change car status to Available after resolved issue • View pickup and return car photos • Issue deposit refund • Export rental reports to PDF or Excel format

Delivery Locations	• View list of available pickup and return locations • Edit details of delivery locations • Add new delivery locations
Payments	• View list of all payment record with receipt • Verify penalty payments and remove customers from blacklist • Export payment reports to PDF or Excel format
Vouchers	• View all vouchers usage status and usage history • Issue voucher to customers
Staff (Admin only)	• View list of all staff with details • Edit staff details • Add new staff to create account for them
Profile	• Change password

2) External Scope

Process	Manual Operations
Authentication and Verification	• Manual verification of the authenticity of IC/Passport, UTM cards and driving license that uploaded by customer • Cross-check document information with the registered data. • License expiry validation
Payment	• Manual verification of payment receipts with the bank statements • Manual refund processing to customer
Pickup	• Customers download the forms and sign them before uploading to the system. • Manual verification of the uploaded signed agreement and inspection form • Staff manually send car to the pickup location if not at Hasta
Return	• Physical car inspection by staff • Physical car collection if not at Hasta
Maintenance	• Scheduled car maintenance or repairs by third-party

3) Major User Views

i) Customer View

The customer portal gives functions to customers to manage their account and create a booking. They can create an account and log in, activate, and update their account. They can create a new booking by filling in booking information and choosing a car and pay. After that, they can see booking history and booking details, cancel their bookings at least 24 hours before pickup time, and track the stamps and vouchers received in their profile.

ii) Staff View

The staff portal supports various operational activities necessary to manage the car rental services. Staff can log in to the system to view pending bookings, pending customer activations, manage customer account and activation status. In addition, staff can also manage cars through add or edit cars, track returns of cars, rental history of cars, approval, or rejection of rentals, update the maintenance and resolve issues reported. Moreover, staff can also manage delivery locations and vouchers, issue refunds and issue penalties.

iii) Admin View

Admin view is an extended staff view that comprises system-level monitoring and administrative control. Admin can access various dashboard analytics such as total rental and customer statistics as well as managing staff accounts by viewing the list of staff and performing add/edit staff operations.

1.2 Overview of Project

This report documents the Database Logical Design for the Hasta Car Rental System. At this phase, the conceptual ERD of Phase 2 is converted to a logical database design by focusing on the entities and relationships to relational tables with defined primary and foreign keys. It is aimed to generate a logical data design, which will be accurate, consistent, and appropriate to be implemented physically in a relational database.

In addition, the logical ERD will be used to derive the relational database schemas and perform normalization up to BCNF in order to remove redundancy and update anomalies. The data dictionary will be revised with reference to the final normalized relations to make sure that all the attributes, constraints, and relationships are well documented.

Lastly, the proposed logical design will be verified against the system transaction requirements by making sure that all the data entry, data update, and data query operations are supported. It contains a series of proposed SQL statements (DDL and DML) to show the way the database structure could be built and manipulated to achieve the major operations of the system.

1.3 Global Conceptual Data Model (ERD)

1.3.1 Updated Business Rules

(a) Customer and Activation Rules

1. Account activation rule:

Before a customer can make a booking, the account needs to be activated such that the status = 'Active'.

2. Blacklist restriction rule:

A customer with status = 'Blacklisted' will be blocked to access the system.

3. Activation rejection reason rule:

When an activation is rejected, a rejected reason must be provided by staff.

4. Duplicate activation rule:

A customer cannot submit a new activation request while having a pending activation.

(b) Car Availability Rules

1. Car availability rule:

A car can only be booked if there is no overlapping rental for the selected pickup–return date/time.

2. Return period cooldown rule:

A car becomes available only after 1 hour of the rental's return time.

3. Maintenance exclusion rule:

A car with status 'Maintenance' will not be displayed in the available car list.

(c) Booking Rules

1. Booking rejection rule:

When staff reject a booking, the car's status is reset to 'Available' with a disapprove reason.

2. Booking period validation rule:

The date and time of return must be after the date and time of pickup.

(d) Cancellation Rules

1. 24-hour cancellation rule:

A booking can be cancelled only if there is more than 24 hours left before the scheduled pickup time and the status of that booking is either “Pending” or “Approved”.

(e) Payment and Refund Rules.

1. Refund bank details rule:

In order to process a refund, bank details must be provided.

2. Rejection refund rule:

When a booking is rejected or cancelled, a full refund will be issued by staff.

(f) Voucher Rules

1. New user voucher rule:

A 'NEWUSER10' voucher is automatically given to a customer upon registration.

2. Voucher validity rule:

A voucher can only be applied if usage = 'Unused' and expiry_date >= current date.

3. Voucher usage update rule:

Once a voucher is applied to a rental, its usage is updated to 'Used'.

4. Loyalty voucher claim rule:

A customer can claim a 'LOYALTY20' voucher only when they have at least 3 available loyalty stamps.

(g) Late Return and Penalty Rules**1. Late return cooldown period rule:**

A return is considered late only if it exceeds 30 minutes past the scheduled return time.

2. Late return penalty rule:

For late returns after the cooldown period, a penalty of RM50 per hour (minus RM50 deposit) will be charged.

3. Blacklist on penalty rule:

If a customer has a late penalty amount greater than zero, their status is set to 'Blacklisted'.

4. Deposit forfeiture rule:

The RM50 deposit is forfeited if the customer returns late.

(h) Loyalty Stamp Rule**1. Loyalty stamp rule:**

A customer can earn 1 loyalty stamp for each completed rental that is more than 12 hours.

(i) Car Maintenance Rule**1. Maintenance rule based on issues:**

When a customer reports an issue during car return, the car's status is changed to 'Maintenance'.

1.3.2 Global Conceptual Database Design (ERD)

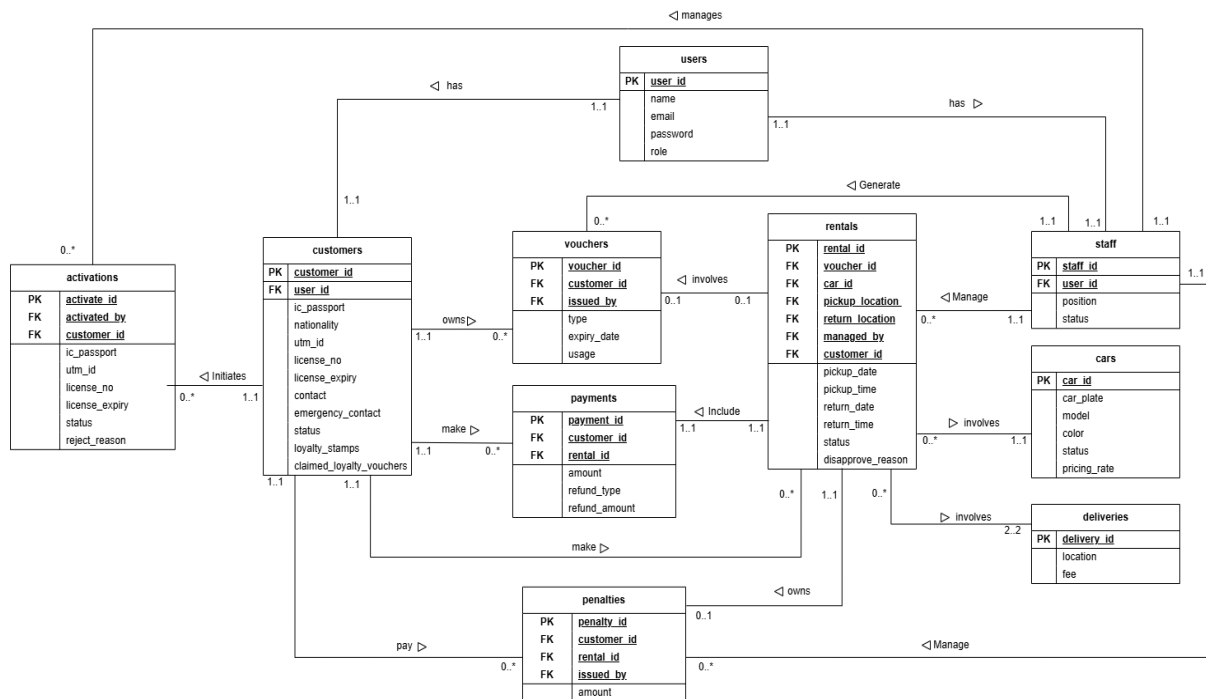


Figure above shows the conceptual ERD that provides a high-level overview of the system to identify the main entities involved including Users, Customers, Staff, Cars, Rentals, Payments, Vouchers, Deliveries, Activations and Penalties and the relationship among them. At this level, the diagram is concerned with the data that the system is working with and how the entities are related but not with the specifics of implementation. The conceptual ERD is used to make the stakeholders get the overall picture of the system and business rules in the system prior to venturing into the detailed database design.

1.3.3 Updated Data Dictionary of Global Conceptual ERD

1. Users

Field Name	Type	Constraint	Description
id (PK)	bigint	PRIMARY KEY, AUTO INCREMENT	Unique identifier for each user, uses auto increment as sequential numbers are efficient for indexing and searching, and also easy to manage and identify records.
name	varchar(50)	NOT NULL	Full name of user (NRIC), uses size of 50 because it is sufficient to store all names.
email	varchar(255)	NOT NULL, UNIQUE	Email address of the users to be used to login, uses varchar 255 as it is the industry standard for email fields. Each email only can be used to create one account so it must be unique.
role	varchar(20)	NOT NULL, DEFAULT 'customer'	Role of user in the system to manage user views and system boundaries. (e.g. staff, admin, customer) Uses varchar 20 as it is sufficient for any role name. Any account registration will create a user with the role of customer, only admin can create user with role of staff.
password	varchar(255)	NOT NULL	This is to store the user passwords for account login credentials. Passwords will be hashed using bcrypt to avoid brute forcing to crack passwords. VARCHAR(255) is safe and recommended for any potential changes to the hashing algorithm or if the library adds extra metadata to the string, such as Spring Security, which uses a {bcrypt} prefix.

2. Customers

Field Name	Type	Constraint	Description
id (PK)	bigint	PRIMARY KEY, AUTO INCREMENT	Unique identifier for each customer, uses auto increment as sequential numbers are efficient for indexing and searching, and also easy to manage and identify records.
user_id (FK)	bigint	FOREIGN KEY, UNIQUE, NOT NULL	Links to the user account to inherit the attributes from user.
ic_passport	varchar(20)	NULLABLE	IC (for Malaysian) or passport number (for foreigner) of the customer. It is null when the account is registered, and filled after the account is activated. VARCHAR 20 is used because it is sufficient to occupy all IC and passport numbers in world.
nationality	varchar(50)	NULLABLE	Nationality of customer, this is to form a composite key with ic_passport, as a customer from one country might have the same passport number with another customer from another country, but each country only can have one distinct passport number. It is null when the account is registered, and filled after the account is activated. Varchar 50 can be used for most of the country.
utm_id	varchar(20)	UNIQUE, NULLABLE	UTM student card or staff card number, it must be unique as one student/staff only can have one account, and varchar 20 is used as it's sufficient for all UTM ID (e.g. A24CS0001) It is null when the account is registered, and filled after the account is activated.

contact	varchar(20)	NULLABLE	Phone contact number of customer, using varchar 20 because it is enough for all telephone numbers. It is null after registration, and filled after the account is activated.
emergency_contact	varchar(20)	NULLABLE	Emergency phone contact number of customer, using varchar 20 because it is enough for all telephone numbers. It is null when the account is registered, and filled after the account is activated.
license_no	varchar(20)	NULLABLE	The car license number of customers as their eligibility to book a car, use varchar 20 as it is enough for all the licences. It is null when the account is registered, and filled after the account is activated. License number is not unique, as different countries might produce the same license no.
license_expiry	date	NULLABLE	The expiry date of car license. It is null when the account is registered, and filled after the account is activated.
college	varchar(30)	NULLABLE	The residential college of students or staff in UTM. For students/staff that live outside, their college will be “NON APPLICABLE”, else it will be UTM college such as KTDI. Varchar 30 is used as it is enough for all college names. It is null when the account is registered, and filled after the account is activated.

faculty	varchar(30)	NULLABLE	The faculty of the customer such as Faculty of Computing, Faculty of Civil Engineering, etc. It is null when the account is registered, and filled after the account is activated. Varchar 30 is used as it is enough for all faculty names.
address	varchar(255)	NULLABLE	The home address of the customer, varchar 255 is enough to occupy most of the address. It is null when the account is registered, and filled after the account is activated.
bank_name	varchar(20)	NULLABLE	The bank name of the customer bank account, such as RHB, Maybank, etc. This is used for refund purposes, and customers can decide to fill it or not in profile. Varchar 20 is used as it is enough for all bank names.
bank_account	varchar(20)	NULLABLE	The bank account number of the customer bank. This is used for refund purposes, and customers can decide to fill it or not in their profile page, so it is nullable in constraint. Varchar 20 is used as it is enough for account numbers.
status	varchar(20)	NOT NULL, DEFAULT 'New'	The status of the customer, by default the customer is New. Varchar20 is enough for all status (Active, Inactive, Blacklisted)
loyalty_stamps	int	NOT NULL, DEFAULT '0'	The current loyalty stamps owned by customers.
claimed_loyalty_vouchers	int	NOT NULL, DEFAULT '0'	The total loyalty vouchers claimed by customers to track how much stamps are used and how much vouchers obtained.

3. Staff

Field Name	Type	Constraint	Description
id (PK)	bigint	PRIMARY KEY, AUTO INCREMENT	Unique identifier for each staff, uses auto increment as sequential numbers are efficient for indexing and searching, and also easy to manage and identify records.
user_id (FK)	bigint	FOREIGN KEY, UNIQUE, NOT NULL	Links to the user account to inherit the attributes from user.
position	varchar(20)	NOT NULL, DEFAULT 'Staff'	The position of staff for managing the privilege and accessibility of certain functions. For example, staff can only do tasks, while admin can view analytics dashboard and download reporting.
status	varchar(20)	NOT NULL, DEFAULT 'Active'	The status of staff, e.g. Active or Inactive, this is important for the soft delete feature to trace back previous staff.

4. Cars

Field Name	Type	Constraint	Description
id (PK)	bigint	PRIMARY KEY, AUTO INCREMENT	Unique identifier for each vehicle, uses auto increment as sequential numbers are efficient for indexing and searching, and also easy to manage and identify records.
car_plate	varchar(20)	NOT NULL, UNIQUE	Plate number for each vehicle, it must be unique as there are no two vehicles with same plate number, varchar 20 is safe to cover all car plate variation.
model	varchar(30)	NOT NULL	The model name of the car, such as Proton X70, etc. Varchar 30 is just enough for a simple model name.
color	varchar(20)	NOT NULL	The color of the car, such as Gold, White, Black, etc. Varchar 20 is totally enough for all color names.
status	varchar(20)	NOT NULL, DEFAULT 'Active'	The status of the car, by default the car is Active, cars that are soft deleted will become Inactive, cars under booking will become Rented, and cars having issues will become Maintenance.
mileage	int	NOT NULL	The mileage of the car, updated by staff after the customer returns the car.
fuel_bar	int	NOT NULL	The fuel bar of the car, updated by staff after the customer returns the car.

pricing_rate	json	NOT NULL	The list of pricing rates of car rental for each hour. JSON is used as Hasta has a special way to calculate price, which they provide pricing rate of 1 hour, 3 hour, 5 hour, 7 hour, 9 hour, 12 hour, and 24 hour. And if a customer does a 10 hour booking, it will use a 12 hour booking rate. Hence, JSON format is suitable and avoids using another table to store pricing rate, it is also scalable and easy to manage as Hasta may change their unrealistic way of price calculation in future.
features	json	NULLABLE	The list of special features of the car, JSON format is used for its scalability feature, as we can add or delete features easily without modifying tables if we use the solution of applying different tables for car features only.

5. Activations

Field Name	Type	Constraint	Description
id (PK)	bigint	PRIMARY KEY, AUTO INCREMENT	Unique identifier for each activation record, uses auto increment as sequential numbers are efficient for indexing and searching, and also easy to manage and identify records.
customer_id (FK)	bigint	FOREIGN KEY, NOT NULL	Links to the customer ID that submit this activation request. A customer can submit multiple activation records as it might be rejected.
activated_by (FK)	bigint	FOREIGN KEY, NULLABLE	Links to the staff account that approve this activation request. It is filled once staff approve it.
ic_passport	varchar(20)	NOT NULL	IC (for Malaysian) or passport number (for foreigner) of the customer. It is not null as customers must fill this to submit an activation request. VARCHAR 20 is used because it is sufficient to occupy all IC and passport numbers in world.
nationality	varchar(50)	NOT NULL	Nationality of customer. It is not null as customers must fill this to submit an activation request. It uses varchar 2 as we are using two-letter codes (e.g. US, MY)
utm_id	varchar(20)	NOT NULL	UTM student card or staff card number, it must be unique as one student/staff only can have one account, and varchar 20 is used as it's sufficient for all UTM ID (e.g. A24CS0001) It is not null as customers must fill this to submit an activation.

contact	varchar(20)	NOT NULL	Phone contact number of customer, using varchar 20 because it is enough for all telephone numbers. It is not null as customers must fill this to submit an activation request.
emergency_ contact	varchar(20)	NOT NULL	Emergency phone contact number of customer, using varchar 20 because it is enough for all telephone numbers. It is not null as customers must fill this to submit an activation.
license_no	varchar(20)	NOT NULL	The car license number of customers as their eligibility to book a car, use varchar 20 as it is enough for all the licences. It is not null as customers must fill this to submit an activation.
license_expiry	date	NOT NULL	The expiry date of car license. It is not null as customers must fill this to submit an activation.
college	varchar(30)	NOT NULL	The residential college of students or staff in UTM. For students/staff that live outside, their college will be “NON APPLICABLE”, else it will be UTM college such as KTDI. Varchar 30 is used as it is enough for all college names. It is not null as customers must fill this to submit an activation.

faculty	varchar(30)	NOT NULL	The faculty of the customer such as Faculty of Computing, Faculty of Civil Engineering, etc. Varchar 30 is used as it is enough for all faculty names. It is not null as customers must fill this to submit an activation.
address	varchar(255)	NOT NULL	The home address of the customer, varchar 255 is enough to occupy most of the address. It is not null as customers must fill this to submit an activation.
status	varchar(20)	NOT NULL, DEFAULT 'Pending'	The status of activation record, by default is Pending, if approved turn Approved, if rejected turn Rejected. Varchar 20 is enough for all status.
reject_reason	varchar(255)	NULLABLE	The reason given by staff to reject the activation request. Varchar 255 is enough for text.

6. Vouchers

Field Name	Type	Constraint	Description
id (PK)	bigint	PRIMARY KEY, AUTO INCREMENT	Unique identifier for each voucher, uses auto increment as sequential numbers are efficient for indexing and searching, and also easy to manage and identify records.
customer_id (FK)	bigint	FOREIGN KEY, NOT NULL	Links to the customer that uses this voucher to make a booking payment.
issued_by (FK)	bigint	FOREIGN KEY, NULLABLE	Links to the staff that assign the voucher manually, if the voucher is auto created by system like NEWUSER10 created for new users, and LOYALTY20 created for customer that claim 3 stamps, the issued_by column is null.
type	varchar(20)	NOT NULL	The type of voucher such as BOOKING10, NEWUSER10, and LOYALTY20, each voucher has their different description and different rules of discount. Varchar 20 is enough for all the types needed for a voucher.
expiry_date	date	NOT NULL	The expiry date of the voucher, exceeding expiry date will change the status of Unused voucher to Expired.
status	varchar(20)	NOT NULL, DEFAULT 'Unused'	The status of the vouchers, Unused for voucher that has not been used, Used for voucher that has been used, Expired for voucher that exceeds the expiry date. Varchar 20 is enough for all status needed for a voucher.

7. Deliveries

Field Name	Type	Constraint	Description
id (PK)	bigint	PRIMARY KEY, AUTO INCREMENT	Unique identifier for each delivery location, uses auto increment as sequential numbers are efficient for indexing and searching, and also easy to manage and identify records.
location	varchar(50)	NOT NULL	The delivery locations provided by Hasta for pickup and return locations, varchar 50 is used as it is sufficient for all the locations needed,
fee	decimal(6,2)	NOT NULL	The delivery fees to deliver cars to pickup location and return location.

8. Rentals

Field Name	Type	Constraint	Description
id (PK)	bigint	PRIMARY KEY, AUTO INCREMENT	Unique identifier for each rental record, uses auto increment as sequential numbers are efficient for indexing and searching, and also easy to manage and identify records.
customer_id (FK)	bigint	FOREIGN KEY, NOT NULL	Links to the customer account that initiates this booking.
car_id (FK)	bigint	FOREIGN KEY, NOT NULL	Links to the car that is involved in this booking.
voucher_id (FK)	bigint	FOREIGN KEY, NULLABLE	Links to the voucher used in this booking, can be null if customers do not use any voucher during payment.
managed_by (FK)	bigint	FOREIGN KEY, NULLABLE	Links to the staff that manage this booking, the column become filled once any staff approve or reject booking.
pickup_date	date	NOT NULL	The selected date of car pickup by the customer.
pickup_time	time	NOT NULL	The selected time of car pickup by the customer.
return_date	date	NOT NULL	The selected date of car return by the customer.
return_time	time	NOT NULL	The selected time of car return by the customer.

pickup_location (FK)	bigint	FOREIGN KEY, NOT NULL	The selected location of car pickup by the customer. Links to the delivery location to obtain the delivery fee.
return_location (FK)	bigint	FOREIGN KEY, NOT NULL	The selected location of car return by the customer. Links to the delivery location to obtain the delivery fee.
disapprove_ reason	varchar(255)	NULLABLE	The reason to reject booking submitted by staff, this is filled if the status of booking becomes Rejected, so the customer can see the disapproval reason, else it will remain as null. Varchar 255 is enough for all reasons, and limits extremely long reasons given by staff.
feedback	varchar(255)	NULLABLE	The feedback submitted by customer after car return, it is filled if customer submitted any feedback. Varchar 255 is enough and limits extremely long feedback by customers.

9. Payments

Field Name	Type	Constraint	Description
id (PK)	bigint	PRIMARY KEY, AUTO INCREMENT	Unique identifier for each transaction record, uses auto increment as sequential numbers are efficient for indexing and searching, and also easy to manage and identify records.
customer_id (FK)	bigint	FOREIGN KEY, NOT NULL	Links to the customer account that make this transaction record.
rental_id (FK)	bigint	FOREIGN KEY, NOT NULL	Links to the rental record where this payment record belongs to.
amount	decimal(8,2)	NOT NULL	The total amount of payment of this booking, including delivery fees, deposit, full payment, and deducted by voucher discount.
refund_type	varchar(20)	NULLABLE	The refund type of the rental record, it can be Cancellation Refund to refund full if customer cancels booking before 24 hours, can be Rejection Refund to refund full if staff reject booking, can be Deposit Refund, which is RM50 after customer returns the car. Varchar 20 is enough for all types needed for refund.
refund_owner	varchar(50)	NULLABLE	The full name of the refund bank account owner, which is submitted by the customer. Varchar 50 is enough for all full names typically. It is filled once car is returned, as customers fill in return form.

refund_bank	varchar(20)	NULLABLE	The bank name of the account to be refunded. Varchar 20 is enough for all bank names. It is filled once car is returned, as customers fill in return form.
refund_account	varchar(20)	NULLABLE	The bank account number of the account to be refunded. Varchar 20 is enough for all account number. It is filled once car is returned, as customers fill in return form.
refund_amount	decimal(6,2)	NULLABLE	The total amount needs to be refunded for the customer, if the type is Cancellation Refund or Rejection, full refund will be given, else if the type is Deposit Refund, the refund will be RM50. This attribute is filled once the car is returned.

10. Penalty

Field Name	Type	Constraint	Description
id (PK)	bigint	PRIMARY KEY, AUTO INCREMENT	Unique identifier for each penalty record, uses auto increment as sequential numbers are efficient for indexing and searching, and also easy to manage and identify records.
customer_id (FK)	bigint	NOT NULL	Links to the customer account that is involved in this penalty record.
rental_id (FK)	bigint	NOT NULL	Links to the rental ID that causes this penalty.
issued_by (FK)	bigint	NULLABLE	Links to the staff account that manages this penalty, either approve or reject payment.
amount	decimal(6,2)	NOT NULL	The total amount of the penalty needs to be paid by the customer.

1.4 Logical Data Model (Logical ERD)

1.4.1 Logical Database Design (Logical ERD)

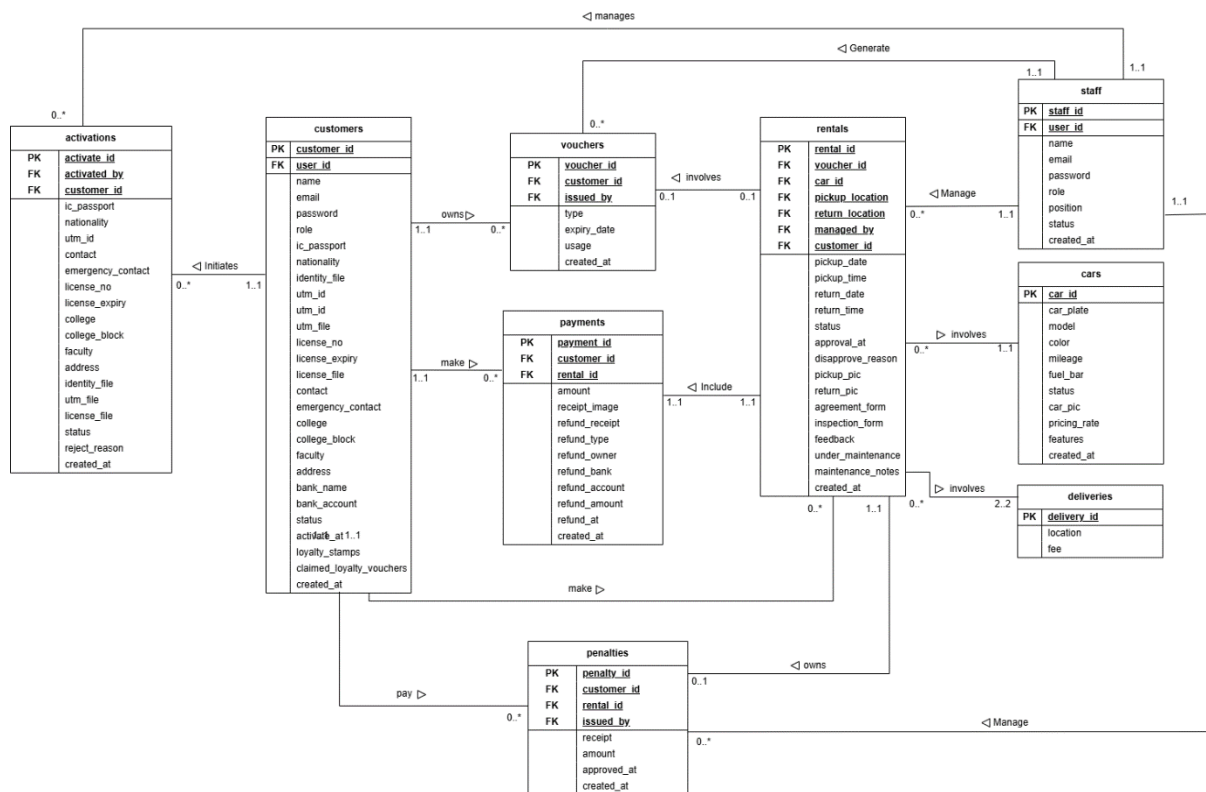


Figure above shows the logical ERD derived from the conceptual design and providing a more detailed representation of how the database should be organized after normalization. It displays primary keys, foreign keys and cardinalities that show how entities such as Rentals, Payments, Vouchers and Deliveries are related in the final relational schemas. This diagram has also included the deliverables of normalization to the BCNF level which includes a minimum amount of redundancy and the dependency management. This logical ERD is a connection between the conceptual design and physical database implementation.

1.4.2 Updated Data Dictionary for the Logical ERD

1. Users

Field Name	Type	Constraint	Description
id (PK)	bigint	PRIMARY KEY, AUTO INCREMENT	Unique identifier for each user, uses auto increment as sequential numbers are efficient for indexing and searching, and also easy to manage and identify records.
name	varchar(50)	NOT NULL	Full name of user (NRIC), uses size of 50 because it is sufficient to store all names.
email	varchar(255)	NOT NULL, UNIQUE	Email address of the users to be used to login, uses varchar 255 as it is the industry standard for email fields. Each email only can be used to create one account so it must be unique.
role	varchar(20)	NOT NULL, DEFAULT 'customer'	Role of user in the system to manage user views and system boundaries. (e.g. staff, admin, customer) Uses varchar 20 as it is sufficient for any role name. Any account registration will create a user with the role of customer, only admin can create user with role of staff.
password	varchar(255)	NOT NULL	This is to store the user passwords for account login credentials. Passwords will be hashed using bcrypt to avoid brute forcing to crack passwords. VARCHAR(255) is safe and recommended for any potential changes to the hashing algorithm or if the library adds extra metadata to the string, such as Spring Security, which uses a {bcrypt} prefix.
created_at	timestamp	NOT NULL	Timestamp when the record was created.

2. Customers

Field Name	Type	Constraint	Description
id (PK)	bigint	PRIMARY KEY, AUTO INCREMENT	Unique identifier for each customer, uses auto increment as sequential numbers are efficient for indexing and searching, and also easy to manage and identify records.
user_id (FK)	bigint	FOREIGN KEY, UNIQUE, NOT NULL	Links to the user account to inherit the attributes from user.
ic_passport	varchar(20)	NULLABLE	IC (for Malaysian) or passport number (for foreigner) of the customer. It is null when the account is registered, and filled after the account is activated. VARCHAR 20 is used because it is sufficient to occupy all IC and passport numbers in world.
nationality	varchar(50)	NULLABLE	Nationality of customer, this is to form a composite key with ic_passport, as a customer from one country might have the same passport number with another customer from another country, but each country only can have one distinct passport number. It is null when the account is registered, and filled after the account is activated. Varchar 50 can be used for most of the country.
utm_id	varchar(20)	UNIQUE, NULLABLE	UTM student card or staff card number, it must be unique as one student/staff only can have one account, and varchar 20 is used as it's sufficient for all UTM ID (e.g. A24CS0001) It is null when the account is registered, and filled after the account is activated.

contact	varchar(20)	NULLABLE	Phone contact number of customer, using varchar 20 because it is enough for all telephone numbers. It is null after registration, and filled after the account is activated.
emergency_ contact	varchar(20)	NULLABLE	Emergency phone contact number of customer, using varchar 20 because it is enough for all telephone numbers. It is null when the account is registered, and filled after the account is activated.
license_no	varchar(20)	NULLABLE	The car license number of customers as their eligibility to book a car, use varchar 20 as it is enough for all the licences. It is null when the account is registered, and filled after the account is activated. License number is not unique, as different countries might produce the same license no.
license_expiry	date	NULLABLE	The expiry date of car license. It is null when the account is registered, and filled after the account is activated.
college	varchar(30)	NULLABLE	The residential college of students or staff in UTM. For students/staff that live outside, their college will be “NON APPLICABLE”, else it will be UTM college such as KTDI. Varchar 30 is used as it is enough for all college names. It is null when the account is registered, and filled after the account is activated.

college_block	varchar(20)	NULLABLE	The residential college block of students or staff in UTM. Varchar 20 is used as it is enough for all college blocks. It is null when the account is registered, and filled after the account is activated if college is not NON APPLICABLE.
faculty	varchar(30)	NULLABLE	The faculty of the customer such as Faculty of Computing, Faculty of Civil Engineering, etc. It is null when the account is registered, and filled after the account is activated. Varchar 30 is used as it is enough for all faculty names.
address	varchar(255)	NULLABLE	The home address of the customer, varchar 255 is enough to occupy most of the address. It is null when the account is registered, and filled after the account is activated.
bank_name	varchar(20)	NULLABLE	The bank name of the customer bank account, such as RHB, Maybank, etc. This is used for refund purposes, and customers can decide to fill it or not in profile. Varchar 20 is used as it is enough for all bank names.
bank_account	varchar(20)	NULLABLE	The bank account number of the customer bank. This is used for refund purposes, and customers can decide to fill it or not in their profile page, so it is nullable in constraint. Varchar 20 is used as it is enough for account numbers.
identity_file	varchar(255)	NULLABLE	The file path of IC or passport documents stored in external storage. Varchar 255 is needed to be reserved for any future change in external storage type that requires a longer file path. It is null when the account is registered, and filled after the account is activated.

utm_file	varchar(255)	NULLABLE	The file path of UTM student card or staff card documents stored in external storage. Varchar 255 is needed to be reserved for any future change in external storage type that requires a longer file path. It is null when the account is registered, and filled after the account is activated.
license_file	varchar(255)	NULLABLE	The file path of license documents stored in external storage. Varchar 255 is needed to be reserved for any future change in external storage type that requires a longer file path. It is null when the account is registered, and filled after the account is activated.
activated_at	timestamp	NULLABLE	The timestamp of customer activation approved by staff. It is null when the account is registered, and filled after the account is activated.
status	varchar(20)	NOT NULL, DEFAULT 'New'	The status of the customer, by default the customer is New. Varchar20 is enough for all status (Active, Inactive, Blacklisted)
loyalty_stamps	int	NOT NULL, DEFAULT '0'	The current loyalty stamps owned by customers.
claimed_loyalty_vouchers	int	NOT NULL, DEFAULT '0'	The total loyalty vouchers claimed by customers to track how much stamps are used and how much vouchers obtained.
created_at	timestamp	NOT NULL	Timestamp when the record was created.

3. Staff

Field Name	Type	Constraint	Description
id (PK)	bigint	PRIMARY KEY, AUTO INCREMENT	Unique identifier for each staff, uses auto increment as sequential numbers are efficient for indexing and searching, and also easy to manage and identify records.
user_id (FK)	bigint	FOREIGN KEY, UNIQUE, NOT NULL	Links to the user account to inherit the attributes from user.
position	varchar(20)	NOT NULL, DEFAULT 'Staff'	The position of staff for managing the privilege and accessibility of certain functions. For example, staff can only do tasks, while admin can view analytics dashboard and download reporting.
status	varchar(20)	NOT NULL, DEFAULT 'Active'	The status of staff, e.g. Active or Inactive, this is important for the soft delete feature to trace back previous staff.
created_at	timestamp	NOT NULL	Timestamp when the record was created.

4. Cars

Field Name	Type	Constraint	Description
id (PK)	bigint	PRIMARY KEY, AUTO INCREMENT	Unique identifier for each vehicle, uses auto increment as sequential numbers are efficient for indexing and searching, and also easy to manage and identify records.
car_plate	varchar(20)	NOT NULL, UNIQUE	Plate number for each vehicle, it must be unique as there are no two vehicles with same plate number, varchar 20 is safe to cover all car plate variation.
model	varchar(30)	NOT NULL	The model name of the car, such as Proton X70, etc. Varchar 30 is just enough for a simple model name.
color	varchar(20)	NOT NULL	The color of the car, such as Gold, White, Black, etc. Varchar 20 is totally enough for all color names.
car_pic	varchar(255)	NOT NULL	File path stored at external storage of picture of the car. Varchar 255 is needed to be reserved for any future change in external storage mode, which results in a longer file path.
status	varchar(20)	NOT NULL, DEFAULT 'Active'	The status of the car, by default the car is Active, cars that are soft deleted will become Inactive, cars under booking will become Rented, and cars having issues will become Maintenance.
mileage	int	NOT NULL	The mileage of the car, updated by staff after the customer returns the car.
fuel_bar	int	NOT NULL	The fuel bar of the car, updated by staff after the customer returns the car.

pricing_rate	json	NOT NULL	The list of pricing rates of car rental for each hour. JSON is used as Hasta has a special way to calculate price, which they provide pricing rate of 1 hour, 3 hour, 5 hour, 7 hour, 9 hour, 12 hour, and 24 hour. And if a customer does a 10 hour booking, it will use a 12 hour booking rate. Hence, JSON format is suitable and avoids using another table to store pricing rate, it is also scalable and easy to manage as Hasta may change their unrealistic way of price calculation in future.
features	json	NULLABLE	The list of special features of the car, JSON format is used for its scalability feature, as we can add or delete features easily without modifying tables if we use the solution of applying different tables for car features only.
created_at	timestamp	NOT NULL	Timestamp when the record was created.

5. Activations

Field Name	Type	Constraint	Description
id (PK)	bigint	PRIMARY KEY, AUTO INCREMENT	Unique identifier for each activation record, uses auto increment as sequential numbers are efficient for indexing and searching, and also easy to manage and identify records.
customer_id (FK)	bigint	FOREIGN KEY, NOT NULL	Links to the customer ID that submit this activation request. A customer can submit multiple activation records as it might be rejected.
activated_by (FK)	bigint	FOREIGN KEY, NULLABLE	Links to the staff account that approve this activation request. It is filled once staff approve it.
ic_passport	varchar(20)	NOT NULL	IC (for Malaysian) or passport number (for foreigner) of the customer. It is not null as customers must fill this to submit an activation request. VARCHAR 20 is used because it is sufficient to occupy all IC and passport numbers in world.
nationality	varchar(50)	NOT NULL	Nationality of customer. It is not null as customers must fill this to submit an activation request. It uses varchar 2 as we are using two-letter codes (e.g. US, MY)
utm_id	varchar(20)	NOT NULL	UTM student card or staff card number, it must be unique as one student/staff only can have one account, and varchar 20 is used as it's sufficient for all UTM ID (e.g. A24CS0001) It is not null as customers must fill this to submit an activation.

contact	varchar(20)	NOT NULL	Phone contact number of customer, using varchar 20 because it is enough for all telephone numbers. It is not null as customers must fill this to submit an activation request.
emergency_contact	varchar(20)	NOT NULL	Emergency phone contact number of customer, using varchar 20 because it is enough for all telephone numbers. It is not null as customers must fill this to submit an activation.
license_no	varchar(20)	NOT NULL	The car license number of customers as their eligibility to book a car, use varchar 20 as it is enough for all the licences. It is not null as customers must fill this to submit an activation.
license_expiry	date	NOT NULL	The expiry date of car license. It is not null as customers must fill this to submit an activation.
college	varchar(30)	NOT NULL	The residential college of students or staff in UTM. For students/staff that live outside, their college will be “NON APPLICABLE”, else it will be UTM college such as KTDI. Varchar 30 is used as it is enough for all college names. It is not null as customers must fill this to submit an activation.
college_block	varchar(20)	NOT NULL	The residential college block of students or staff in UTM. Varchar 20 is used as it is enough for all college blocks. It is not null as customers must fill this to submit an activation.

faculty	varchar(30)	NOT NULL	The faculty of the customer such as Faculty of Computing, Faculty of Civil Engineering, etc. Varchar 30 is used as it is enough for all faculty names. It is not null as customers must fill this to submit an activation.
address	varchar(255)	NOT NULL	The home address of the customer, varchar 255 is enough to occupy most of the address. It is not null as customers must fill this to submit an activation.
identity_file	varchar(255)	NOT NULL	The file path of IC or passport documents stored in external storage. Varchar 255 is needed to be reserved for any future change in external storage type that requires a longer file path. It is not null as customers must fill this to submit an activation.
utm_file	varchar(255)	NOT NULL	The file path of UTM student card or staff card documents stored in external storage. Varchar 255 is needed to be reserved for any future change in external storage type that requires a longer file path. It is not null as customers must fill this to submit an activation.
license_file	varchar(255)	NOT NULL	The file path of license documents stored in external storage. Varchar 255 is needed to be reserved for any future change in external storage type that requires a longer file path. It is not null as customers must fill this to submit an activation.
status	varchar(20)	NOT NULL, DEFAULT 'Pending'	The status of activation record, by default is Pending, if approved turn Approved, if rejected turn Rejected. Varchar 20 is enough for all status.
reject_reason	varchar(255)	NULLABLE	The reason given by staff to reject the activation request. Varchar 255 is enough for text.
created_at	timestamp	NOT NULL	Timestamp when the record was created.

6. Vouchers

Field Name	Type	Constraint	Description
id (PK)	bigint	PRIMARY KEY, AUTO INCREMENT	Unique identifier for each voucher, uses auto increment as sequential numbers are efficient for indexing and searching, and also easy to manage and identify records.
customer_id (FK)	bigint	FOREIGN KEY, NOT NULL	Links to the customer that uses this voucher to make a booking payment.
issued_by (FK)	bigint	FOREIGN KEY, NULLABLE	Links to the staff that assign the voucher manually, if the voucher is auto created by system like NEWUSER10 created for new users, and LOYALTY20 created for customer that claim 3 stamps, the issued_by column is null.
type	varchar(20)	NOT NULL	The type of voucher such as BOOKING10, NEWUSER10, and LOYALTY20, each voucher has their different description and different rules of discount. Varchar 20 is enough for all the types needed for a voucher.
expiry_date	date	NOT NULL	The expiry date of the voucher, exceeding expiry date will change the status of Unused voucher to Expired.
status	varchar(20)	NOT NULL, DEFAULT 'Unused'	The status of the vouchers, Unused for voucher that has not been used, Used for voucher that has been used, Expired for voucher that exceeds the expiry date. Varchar 20 is enough for all status needed for a voucher.
created_at	timestamp	NOT NULL	Timestamp when the record was created.

7. Deliveries

Field Name	Type	Constraint	Description
id (PK)	bigint	PRIMARY KEY, AUTO INCREMENT	Unique identifier for each delivery location, uses auto increment as sequential numbers are efficient for indexing and searching, and also easy to manage and identify records.
location	varchar(50)	NOT NULL	The delivery locations provided by Hasta for pickup and return locations, varchar 50 is used as it is sufficient for all the locations needed,
fee	decimal(6,2)	NOT NULL	The delivery fees to deliver cars to pickup location and return location.
created_at	timestamp	NOT NULL	Timestamp when the record was created.

8. Rentals

Field Name	Type	Constraint	Description
id (PK)	bigint	PRIMARY KEY, AUTO INCREMENT	Unique identifier for each rental record, uses auto increment as sequential numbers are efficient for indexing and searching, and also easy to manage and identify records.
customer_id (FK)	bigint	FOREIGN KEY, NOT NULL	Links to the customer account that initiates this booking.
car_id (FK)	bigint	FOREIGN KEY, NOT NULL	Links to the car that is involved in this booking.
voucher_id (FK)	bigint	FOREIGN KEY, NULLABLE	Links to the voucher used in this booking, can be null if customers do not use any voucher during payment.
managed_by (FK)	bigint	FOREIGN KEY, NULLABLE	Links to the staff that manage this booking, the column become filled once any staff approve or reject booking.
pickup_date	date	NOT NULL	The selected date of car pickup by the customer.
pickup_time	time	NOT NULL	The selected time of car pickup by the customer.
return_date	date	NOT NULL	The selected date of car return by the customer.
return_time	time	NOT NULL	The selected time of car return by the customer.

pickup_location (FK)	bigint	FOREIGN KEY, NOT NULL	The selected location of car pickup by the customer. Links to the delivery location to obtain the delivery fee.
return_location (FK)	bigint	FOREIGN KEY, NOT NULL	The selected location of car return by the customer. Links to the delivery location to obtain the delivery fee.
approved_at	timestamp	NULLABLE	The timestamp of when the booking is approved, it is filled once staff approve it.
disapprove_reason	varchar(255)	NULLABLE	The reason to reject booking submitted by staff, this is filled if the status of booking becomes Rejected, so the customer can see the disapproval reason, else it will remain as null. Varchar 255 is enough for all reasons, and limits extremely long reasons given by staff.
agreement_form	varchar(255)	NULLABLE	File path stored at external storage of agreement form submitted by customers. Varchar 255 is needed to be reserved for any future change in external storage mode, which results in a longer file path.
inspection_form	varchar(255)	NULLABLE	File path stored at external storage of inspection form submitted by customers. Varchar 255 is needed to be reserved for any future change in external storage mode, which results in a longer file path.

pickup_pic	json	NULLABLE	The list of inspection pictures file paths submitted by the customer during car pickup. JSON format is suitable and avoids using another table to store pickup picture file paths, it is also scalable and easy to manage as Hasta may require customers to submit more inspection pictures in future, which avoids having huge changes in the database.
return_pic	json	NULLABLE	The list of inspection pictures file paths submitted by the customer during car return. JSON format is suitable and avoids using another table to store return picture file paths, it is also scalable and easy to manage as Hasta may require customers to submit more inspection pictures in future, which avoids having huge changes in the database.
feedback	varchar(255)	NULLABLE	The feedback submitted by customer after car return, it is filled if customer submitted any feedback. Varchar 255 is enough and limits extremely long feedback by customers.
created_at	timestamp	NOT NULL	Timestamp when the record was created.

9. Payments

Field Name	Type	Constraint	Description
id (PK)	bigint	PRIMARY KEY, AUTO INCREMENT	Unique identifier for each transaction record, uses auto increment as sequential numbers are efficient for indexing and searching, and also easy to manage and identify records.
customer_id (FK)	bigint	FOREIGN KEY, NOT NULL	Links to the customer account that make this transaction record.
rental_id (FK)	bigint	FOREIGN KEY, NOT NULL	Links to the rental record where this payment record belongs to.
amount	decimal(8,2)	NOT NULL	The total amount of payment of this booking, including delivery fees, deposit, full payment, and deducted by voucher discount.
payment_receipt	varchar(255)	NOT NULL	File path stored at external storage of payment receipt submitted by customers during booking. Varchar 255 is needed to be reserved for any future change in external storage mode, which results in a longer file path.
refund_receipt	varchar(255)	NULLABLE	File path stored at external storage of refund receipt submitted by staff. This attribute is filled after staff issue the refund and submit a refund receipt in the system. Varchar 255 is needed to be reserved for any future change in external storage mode, which results in a longer file path.

refund_type	varchar(20)	NULLABLE	The refund type of the rental record, it can be Cancellation Refund to refund full if customer cancels booking before 24 hours, can be Rejection Refund to refund full if staff reject booking, can be Deposit Refund, which is RM50 after customer returns the car. Varchar 20 is enough for all types needed for refund.
refund_owner	varchar(50)	NULLABLE	The full name of the refund bank account owner, which is submitted by the customer. Varchar 50 is enough for all full names typically. It is filled once car is returned, as customers fill in return form.
refund_bank	varchar(20)	NULLABLE	The bank name of the account to be refunded. Varchar 20 is enough for all bank names. It is filled once car is returned, as customers fill in return form.
refund_account	varchar(20)	NULLABLE	The bank account number of the account to be refunded. Varchar 20 is enough for all account number. It is filled once car is returned, as customers fill in return form.
refund_amount	decimal(6,2)	NULLABLE	The total amount needs to be refunded for the customer, if the type is Cancellation Refund or Rejection, full refund will be given, else if the type is Deposit Refund, the refund will be RM50. This attribute is filled once the car is returned.
refund_at	timestamp	NULLABLE	The timestamp of the refund is issued. This attribute is filled once staff finish refund by submit a refund receipt.
created_at	timestamp	NOT NULL	Timestamp when the record was created.

10. Penalty

Field Name	Type	Constraint	Description
id (PK)	bigint	PRIMARY KEY, AUTO INCREMENT	Unique identifier for each penalty record, uses auto increment as sequential numbers are efficient for indexing and searching, and also easy to manage and identify records.
customer_id (FK)	bigint	NOT NULL	Links to the customer account that is involved in this penalty record.
rental_id (FK)	bigint	NOT NULL	Links to the rental ID that causes this penalty.
issued_by (FK)	bigint	NULLABLE	Links to the staff account that manages this penalty, either approve or reject payment.
receipt	varchar(255)	NULLABLE	File path stored at external storage of penalty receipts that are pending to be approved by staff. Varchar 255 is needed to be reserved for any future change in external storage mode, which results in a longer file path.
amount	decimal(6,2)	NOT NULL	The total amount of the penalty needs to be paid by the customer.
approved_at	timestamp	NULLABLE	The timestamp when the penalty payment is approved by staff.
created_at	timestamp	NOT NULL	Timestamp when the record was created.

1.4.3 Normalization

1.4.3.1 Unnormalized Form (UNF)

HastaCarRental (

user_id, name, email, password, role, customer_id, customer_ic_passport,
customer_nationality, customer_utm_id, customer_contact, customer_emergency_contact,
customer_license_no, customer_license_expiry, customer_college, customer_college_block,
customer_faculty, customer_address, customer_bank_name, customer_bank_account,
customer_identity_file, customer_utm_file, customer_license_file, customer_activated_at,
customer_status, customer_loyalty_stamps, customer_claimed_loyalty_vouchers,
activation_id, activation_ic_passport, activation_nationality, activation_utm_id,
activation_contact, activation_emergency_contact, activation_license_no,
activation_license_expiry, activation_college, activation_college_block, activation_faculty,
activation_address, activation_identity_file, activation_utm_file, activation_license_file,
activation_status, activation_reject_reason, car_id, car_plate, model, color, car_pic,
car_status, mileage, fuel_bar, pricing_rate, features, staff_id, staff_position, staff_status,
rental_id, pickup_date, pickup_time, pickup_location, return_date, return_time,
return_location, pickup_pic, return_pic, rental_status, rental_approved_at, agreement_form,
inspection_form, feedback, rental_disapprove_reason, payment_id, payment_amount,
receipt_image, refund_receipt, refund_type, refund_owner, refund_bank, refund_account,
refund_amount, refund_at, voucher_id, voucher_type, voucher_usage, voucher_expiry_date,
penalty_id, penalty_receipt, penalty_amount, penalty_approved_at, delivery_id,
delivery_location, delivery_fee

)

Assumptions:

The attributes pricing_rate, features, pickup_pic, and return_pic (stored as JSON) are treated as atomic strings for the purpose of this normalization.

Dependency:

FD1: user_id → name, email, password, roles (Primary Key)

FD2: email → user_id (Candidate Key)

FD3: customer_id → customer_ic_passport, customer_nationality, customer_utm_id
customer_contact, customer_emergency_contact,
customer_license_no, customer_license_expiry,
customer_college,
customer_college_block, customer_faculty, customer_address,
customer_bank_name, customer_bank_account,
customer_identity_file, customer_utm_file,
customer_license_file,
customer_activated_at, customer_status,
customer_loyalty_stamps,
customer_claimed_loyalty_vouchers (Primary Key)

FD4: user_id → customer_id (Candidate Key)

FD5: customer_utm_id → customer_id (Candidate Key)

FD6: activation_id → activation_ic_passport, activation_nationality,
activation_utm_id,
activation_contact, activation_emergency_contact,
activation_license_no, activation_license_expiry,
activation_college,
activation_college_block, activation_faculty,
activation_address,
activation_identity_file, activation_utm_file,
activation_license_file,
activation_status, activation_reject_reason (Primary Key)

FD7: activation_id → customer_id (Relationship Dependency)

FD8: car_id → car_plate, model, color, car_pic, car_status, mileage, fuel_bar,
pricing_rate, features (Primary Key)

FD9: car_plate → car_id (Candidate Key)

FD10: staff_id → staff_position, staff_status (Primary Key)

FD11: user_id → staff_id (Candidate Key)

FD12: rental_id → customer_id, car_id, staff_id, pickup_date, pickup_time,
pickup_location, return_date, return_time, return_location,
pickup_pic, return_pic, rental_status, rental_approved_at,
agreement_form, inspection_form, feedback,
rental_disapprove_reason (Primary Key)

FD13: payment_id → rental_id, payment_amount, receipt_image, refund_receipt,
refund_type, refund_owner, refund_bank, refund_account,
refund_amount, refund_at (Primary Key)

FD14: voucher_id → voucher_type, voucher_usage, voucher_expiry_date
(Primary Key)

FD15: voucher_id → customer_id (Relationship Dependency)

FD16: penalty_id → penalty_receipt, penalty_amount, penalty_approved_at
(Primary Key)

FD17: penalty_id → rental_id (Relationship Dependency)

FD18: delivery_id → delivery_location, delivery_fee (Primary Key)

FD19: pickup_location → delivery_fee (Transitive Dependency)

FD20: return_location → delivery_fee (Transitive Dependency)

1.4.3.2 First Normal Form (1NF)

Users (

user_id (PK),

name,

email,

password,

role,

)

Customers (

customer_id (PK),

user_id (FK),

customer_ic_passport,

customer_nationality,

customer_utm_id,

customer_contact,

customer_emergency_contact,

customer_license_no,

customer_license_expiry,

customer_college,

customer_college_block,

customer_faculty,

customer_address,

customer_bank_name,

customer_bank_account,

customer_identity_file,

customer_utm_file,

customer_license_file,

customer_activated_at,

customer_status,

customer_loyalty_stamps,

customer_claimed_loyalty_vouchers

)

Activations (

activation_id (PK),

customer_id (FK),

activated_by (FK),

activation_ic_passport,

activation_nationality,

activation_utm_id,

activation_contact,

activation_emergency_contact,

activation_license_no,

activation_license_expiry,

activation_college,

activation_college_block,

activation_faculty,

activation_address,

activation_identity_file,

activation_utm_file,

activation_license_file,

activation_status,

activation_reject_reason

)

Cars (

cars_id (PK),

car_plate,

model,

color,

car_pic,

car_status,

mileage,

fuel_bar,

pricing_rate,

features

)

Staff (

staff_id (PK),

user_id (FK),

staff_position,

staff_status

)

Rentals (

rental_id (PK),

customer_id (FK),

car_id (FK),

staff_id (FK),

voucher_id (FK),

pickup_date,

pickup_time,

pickup_location,

return_date,

return_time,

return_location,

pickup_pic,

return_pic,

rental_status,

rental_approved_at,

agreement_form,

inspection_form,

feedback,

rental_disapprove_reason

)

Payments (

payment_id (PK),

rental_id (FK),

customer_id (FK),

payment_amount,

receipt_image,

refund_receipt,

refund_type,

refund_owner,

refund_bank,

refund_account,

refund_amount,

refund_at

)

Vouchers (

voucher_id (PK),

customer_id (FK),

staff_id (FK),

voucher_type,

voucher_usage,

voucher_expiry_date

)

Penalties (

penalty_id (PK),

customer_id (FK),

rental_id (FK),

staff_id (FK),

penalty_amount,

penalty_receipt,

penalty_approved_at

)

Deliveries (

delivery_id (PK),

delivery_location,

delivery_fee

)

1.4.3.3 Second Normal Form (2NF)

Objectives: Identify Partial Dependency

All relations are using single-attribute as primary keys. Since a partial dependency can only occur when there is a composite primary key, therefore no further decomposition of the table is required. All the relations remain unchanged as in 1NF and satisfy 2NF.

1.4.3.4 Third Normal Form (3NF)

Objectives: Identify Transitive Dependency

FD19: pickup_location → delivery_fee (Transitive Dependency)

FD20: return_location → delivery_fee (Transitive Dependency)

The delivery fee does not depend directly on the rental. A rental only specifies the delivery location while the delivery fee is determined by that location. Hence, this creates a transitive dependency with rental_id → pickup_location → delivery_fee which violates the 3NF.

In the final design, this dependency is resolved by ensuring the delivery fee is stored in the Deliveries table, and the Rental table references the delivery locations using foreign key pickup_location and return_location.

Deliveries (

delivery_id (PK),

delivery_location,

delivery_fee

)

Rentals (

rental_id (PK),

customer_id (FK),

car_id (FK),

staff_id (FK),

voucher_id (FK),

pickup_date,

pickup_time,

pickup_location (FK),

return_date,

return_time,

return_location (FK),

pickup_pic,

return_pic,

rental_status,

rental_approved_at,

agreement_form,

inspection_form,

feedback,

rental_disapprove_reason

)

1.4.3.5 Boyce-Codd Normal Form (BCNF)

Users

FD1: user_id → name, email, password, roles (Primary Key)

FD2: email → user_id (Candidate Key)

Customers

FD3: customer_id → customer_ic_passport, customer_nationality, customer_utm_id
customer_contact, customer_emergency_contact,
customer_license_no, customer_license_expiry,
customer_college,
customer_college_block, customer_faculty, customer_address,
customer_bank_name, customer_bank_account,
customer_identity_file, customer_utm_file,
customer_license_file,
customer_activated_at, customer_status,
customer_loyalty_stamps,
customer_claimed_loyalty_vouchers (Primary Key)

FD4: user_id → customer_id (Candidate Key)

FD5: customer_utm_id → customer_id (Candidate Key)

Activations

FD6: activation_id → activation_ic_passport, activation_nationality,
activation_utm_id,
activation_contact, activation_emergency_contact,
activation_license_no, activation_license_expiry,
activation_college, activation_college_block,
activation_faculty, activation_address,
activation_identity_file, activation_utm_file,
activation_license_file,
activation_status, activation_reject_reason (Primary Key)

FD7: activation_id → customer_id (Relationship Dependency)

Cars

FD8: car_id → car_plate, model, color, car_pic, car_status,
mileage, fuel_bar, pricing_rate, features (Primary Key)

FD9: car_plate → car_id (Candidate Key)

Staff

FD10: staff_id → staff_position, staff_status (Primary Key)

FD11: user_id → staff_id (Candidate Key)

Rentals

FD12: rental_id → customer_id, car_id, staff_id, pickup_date, pickup_time,
pickup_location, return_date, return_time, return_location,
pickup_pic, return_pic, rental_status, rental_approved_at,
agreement_form, inspection_form, feedback,
rental_disapprove_reason (Primary Key)

Payments

FD13: payment_id → rental_id, payment_amount, receipt_image, refund_receipt,
refund_type, refund_owner, refund_bank, refund_account, refund_amount, refund_at
(Primary Key)

Vouchers

FD14: voucher_id → voucher_type, voucher_usage, voucher_expiry_date
(Primary Key)

FD15: voucher_id → customer_id (Relationship Dependency)

Penalties

FD16: penalty_id → penalty_receipt, penalty_amount, penalty_approved_at
(Primary Key)

FD17: penalty_id → rental_id (Relationship Dependency)

Deliveries

FD18: $\text{delivery_id} \rightarrow \text{delivery_location}, \text{delivery_fee}$ (Primary Key)
FD19: $\text{pickup_location} \rightarrow \text{delivery_fee}$ (Transitive Dependency)
FD20: $\text{return_location} \rightarrow \text{delivery_fee}$ (Transitive Dependency)\

All the relations above satisfy the requirements of BCNF where the determinant of each functional dependency is a candidate key. Therefore, no more decomposition is needed, and the final database design is free from the redundant data created by non-key determinants.

1.5 Relational Database Schema(s)

1. Users (**user id**, name, email, password, roles)
2. Customers (**customer id**, customer_ic_passport, customer_nationality, customer_utm_id, customer_contact, customer_emergency_contact, customer_license_no, customer_license_expiry, customer_college, customer_college_block, customer_faculty, customer_address, customer_bank_name, customer_bank_account, customer_identity_file, customer_utm_file, customer_license_file, customer_activated_at, customer_status, customer_loyalty_stamps, customer_claimed_loyalty_vouchers)
3. Staff (**staff id**, staff_position, staff_status)
4. Activations (**activation id**, activation_ic_passport, activation_nationality, activation_utm_id, activation_contact, activation_emergency_contact, activation_license_no, activation_license_expiry, activation_college, activation_college_block, activation_faculty, activation_address, activation_identity_file, activation_utm_file, activation_license_file, activation_status, activation_reject_reason)
5. Cars (**car id**, car_plate, model, color, car_pic, car_status, mileage, fuel_bar, pricing_rate, features)
6. Deliveries (**delivery id**, delivery_location, delivery_fee)
7. Rentals (**rental id**, customer_id, car_id, staff_id, pickup_date, pickup_time, pickup_location, return_date, return_time, return_location, pickup_pic, return_pic, rental_status, rental_approved_at, agreement_form, inspection_form, feedback, rental_disapprove_reason)

8. Payments (**payment_id**, rental_id, payment_amount, receipt_image,
refund_receipt, refund_type, refund_owner, refund_bank,
refund_account, refund_amount, refund_at)
9. Vouchers (**voucher_id**, voucher_type, voucher_usage, voucher_expiry_date)
10. Penalties (**penalty_id**, penalty_receipt, penalty_amount, penalty_approved_at)

1.6 Data Definition Language (DDL)

```
CREATE DATABASE hastasystem;
```

```
-- Users Table
```

```
CREATE TABLE `users` (  
  `id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,  
  `name` varchar(50) NOT NULL,  
  `email` varchar(255) NOT NULL,  
  `role` varchar(20) NOT NULL DEFAULT 'customer',  
  `password` varchar(255) NOT NULL,  
  `created_at` timestamp NULL DEFAULT NULL,  
  `updated_at` timestamp NULL DEFAULT NULL,  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `users_email_unique` (`email`)  
);
```

```

-- Customers Table
CREATE TABLE `customers` (
  `id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
  `user_id` bigint(20) unsigned NOT NULL,
  `ic_passport` varchar(20) DEFAULT NULL,
  `nationality` varchar(50) DEFAULT NULL,
  `utm_id` varchar(20) DEFAULT NULL,
  `contact` varchar(20) DEFAULT NULL,
  `emergency_contact` varchar(20) DEFAULT NULL,
  `license_no` varchar(20) DEFAULT NULL,
  `license_expiry` date DEFAULT NULL,
  `college` varchar(30) DEFAULT NULL,
  `college_block` varchar(20) DEFAULT NULL,
  `faculty` varchar(30) DEFAULT NULL,
  `address` varchar(255) DEFAULT NULL,
  `bank_name` varchar(20) DEFAULT NULL,
  `bank_account` varchar(20) DEFAULT NULL,
  `identity_file` varchar(255) DEFAULT NULL,
  `utm_file` varchar(255) DEFAULT NULL,
  `license_file` varchar(255) DEFAULT NULL,
  `activated_at` timestamp NULL DEFAULT NULL,
  `status` varchar(20) NOT NULL DEFAULT 'New',
  `loyalty_stamps` int(11) NOT NULL DEFAULT '0',
  `claimed_loyalty_vouchers` int(11) NOT NULL DEFAULT '0',
  `created_at` timestamp NULL DEFAULT NULL,
  `updated_at` timestamp NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `customers_utm_id_unique` (`utm_id`),
  CONSTRAINT `customers_user_id_foreign` FOREIGN KEY
  (`user_id`) REFERENCES `users` (`id`) ON DELETE CASCADE
);

```

```

-- Staff Table
CREATE TABLE `staff` (
  `id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
  `user_id` bigint(20) unsigned NOT NULL,
  `position` varchar(20) NOT NULL DEFAULT 'staff',
  `status` varchar(20) NOT NULL DEFAULT 'Active',
  `created_at` timestamp NULL DEFAULT NULL,
  `updated_at` timestamp NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  CONSTRAINT `staff_user_id_foreign` FOREIGN KEY (`user_id`)
REFERENCES `users` (`id`) ON DELETE CASCADE
);

```

```

-- Cars Table
CREATE TABLE `cars` (
  `id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
  `car_plate` varchar(20) NOT NULL,
  `model` varchar(30) NOT NULL,
  `color` varchar(20) NOT NULL,
  `car_pic` varchar(255) DEFAULT NULL,
  `status` varchar(20) NOT NULL DEFAULT 'Available',
  `mileage` int(11) NOT NULL DEFAULT '0',
  `fuel_bar` int(11) NOT NULL DEFAULT '0',
  `pricing_rate` json DEFAULT NULL,
  `features` json DEFAULT NULL,
  `created_at` timestamp NULL DEFAULT NULL,
  `updated_at` timestamp NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `cars_car_plate_unique` (`car_plate`)
);

```

```

-- Activations Table
CREATE TABLE `activations` (
  `id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
  `ic_passport` varchar(20) NOT NULL,
  `nationality` varchar(50) NOT NULL,
  `utm_id` varchar(20) NOT NULL,
  `contact` varchar(20) NOT NULL,
  `emergency_contact` varchar(20) NOT NULL,
  `license_no` varchar(20) NOT NULL,
  `license_expiry` date NOT NULL,
  `college` varchar(30) NOT NULL,
  `college_block` varchar(20) NOT NULL,
  `faculty` varchar(30) NOT NULL,
  `address` varchar(255) NOT NULL,
  `identity_file` varchar(255) NOT NULL,
  `utm_file` varchar(255) NOT NULL,
  `license_file` varchar(255) NOT NULL,
  `customer_id` bigint(20) unsigned NOT NULL,
  `activated_by` bigint(20) unsigned DEFAULT NULL,
  `status` varchar(20) NOT NULL DEFAULT 'Pending',
  `reject_reason` varchar(255) DEFAULT NULL,
  `created_at` timestamp NULL DEFAULT NULL,
  `updated_at` timestamp NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  CONSTRAINT `activations_customer_id_foreign` FOREIGN KEY
(`customer_id`) REFERENCES `customers` (`id`) ON DELETE
CASCADE,
  CONSTRAINT `activations_activated_by_foreign` FOREIGN KEY
(`activated_by`) REFERENCES `staff` (`id`) ON DELETE CASCADE
);

```



```

-- Vouchers Table
CREATE TABLE `vouchers` (
  `id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
  `customer_id` bigint(20) unsigned NOT NULL,
  `type` varchar(20) NOT NULL,
  `issued_by` bigint(20) unsigned DEFAULT NULL,
  `expiry_date` date NOT NULL,
  `usage` varchar(20) NOT NULL DEFAULT 'Unused',
  `created_at` timestamp NULL DEFAULT NULL,
  `updated_at` timestamp NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  CONSTRAINT `vouchers_customer_id_foreign` FOREIGN KEY
(`customer_id`) REFERENCES `customers` (`id`) ON DELETE
CASCADE,
  CONSTRAINT `vouchers_issued_by_foreign` FOREIGN KEY
(`issued_by`) REFERENCES `staff` (`id`) ON DELETE CASCADE
);

-- Deliveries Table
CREATE TABLE `deliveries` (
  `id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
  `location` varchar(50) NOT NULL,
  `fee` decimal(6,2) NOT NULL,
  `created_at` timestamp NULL DEFAULT NULL,
  `updated_at` timestamp NULL DEFAULT NULL,
  PRIMARY KEY (`id`)
);

```

```

-- Rentals Table
CREATE TABLE `rentals` (
  `id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
  `customer_id` bigint(20) unsigned NOT NULL,
  `car_id` bigint(20) unsigned NOT NULL,
  `pickup_date` date NOT NULL,
  `pickup_time` time NOT NULL,
  `return_date` date NOT NULL,
  `return_time` time NOT NULL,
  `pickup_location` bigint(20) NOT NULL,
  `return_location` bigint(20) NOT NULL,
  `pickup_pic` json DEFAULT NULL,
  `return_pic` json DEFAULT NULL,
  `status` varchar(20) NOT NULL DEFAULT 'Pending',
  `approved_at` timestamp NULL DEFAULT NULL,
  `agreement_form` varchar(255) DEFAULT NULL,
  `inspection_form` varchar(255) DEFAULT NULL,
  `voucher_id` bigint(20) unsigned DEFAULT NULL,
  `managed_by` bigint(20) unsigned DEFAULT NULL,
  `feedback` varchar(255) DEFAULT NULL,
  `disapprove_reason` varchar(255) DEFAULT NULL,
  `created_at` timestamp NULL DEFAULT NULL,
  `updated_at` timestamp NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  CONSTRAINT `rentals_customer_id_foreign` FOREIGN KEY
(`customer_id`) REFERENCES `customers` (`id`) ON DELETE
CASCADE,
  CONSTRAINT `rentals_car_id_foreign` FOREIGN KEY (`car_id`)
REFERENCES `cars` (`id`) ON DELETE CASCADE,
  CONSTRAINT `rentals_voucher_id_foreign` FOREIGN KEY
(`voucher_id`) REFERENCES `vouchers` (`id`) ON DELETE SET
NULL,
  CONSTRAINT `rentals_managed_by_foreign` FOREIGN KEY
(`managed_by`) REFERENCES `staff` (`id`) ON DELETE CASCADE );

```

```

-- Payments Table
CREATE TABLE `payments` (
  `id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
  `rental_id` bigint(20) unsigned NOT NULL,
  `customer_id` bigint(20) unsigned NOT NULL,
  `amount` decimal(8,2) NOT NULL,
  `receipt_image` varchar(255) DEFAULT NULL,
  `refund_receipt` varchar(255) DEFAULT NULL,
  `refund_type` varchar(20) DEFAULT NULL,
  `refund_owner` varchar(50) DEFAULT NULL,
  `refund_bank` varchar(20) DEFAULT NULL,
  `refund_account` varchar(20) DEFAULT NULL,
  `refund_amount` decimal(6,2) DEFAULT NULL,
  `refund_at` timestamp NULL DEFAULT NULL,
  `created_at` timestamp NULL DEFAULT NULL,
  `updated_at` timestamp NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  CONSTRAINT `payments_rental_id_foreign` FOREIGN KEY
(`rental_id`) REFERENCES `rentals` (`id`) ON DELETE CASCADE,
  CONSTRAINT `payments_customer_id_foreign` FOREIGN KEY
(`customer_id`) REFERENCES `customers` (`id`) ON DELETE
CASCADE
);

```

```

-- Penalties Table
CREATE TABLE `penalties` (
  `id` bigint(20) unsigned NOT NULL AUTO_INCREMENT,
  `customer_id` bigint(20) unsigned NOT NULL,
  `rental_id` bigint(20) unsigned NOT NULL,
  `issued_by` bigint(20) unsigned DEFAULT NULL,
  `receipt` varchar(255) DEFAULT NULL,
  `amount` decimal(6,2) NOT NULL,
  `approved_at` timestamp NULL DEFAULT NULL,
  `created_at` timestamp NULL DEFAULT NULL,
  `updated_at` timestamp NULL DEFAULT NULL,
  PRIMARY KEY (`id`),
  CONSTRAINT `penalties_customer_id_foreign` FOREIGN KEY
(`customer_id`) REFERENCES `customers` (`id`) ON DELETE
CASCADE,
  CONSTRAINT `penalties_rental_id_foreign` FOREIGN KEY
(`rental_id`) REFERENCES `rentals` (`id`) ON DELETE CASCADE,
  CONSTRAINT `penalties_issued_by_foreign` FOREIGN KEY
(`issued_by`) REFERENCES `users` (`id`) ON DELETE CASCADE
);

```

1.7 Interface Design and SQL Implementation

1.7.1 Register & Login

The image displays two mobile application screens for the Hasta Car Rental System. The left screen, titled 'Create Account', prompts users to 'Join us to start your journey' and includes input fields for a full name (e.g., 'Evelyn Ang'), email (e.g., 'evelynang@gradaute.utm.my'), and two passwords. A 'Register' button is at the bottom, with a link to 'Log in' for existing users. The right screen, titled 'Welcome Back', prompts users to 'Please sign in to your account' and includes input fields for email (e.g., 'evelynang@gradaute.utm.my') and password. It features a 'Remember me' checkbox, a 'Forgot password?' link, a 'Log in' button, and a link to 'Sign up now' for new users.

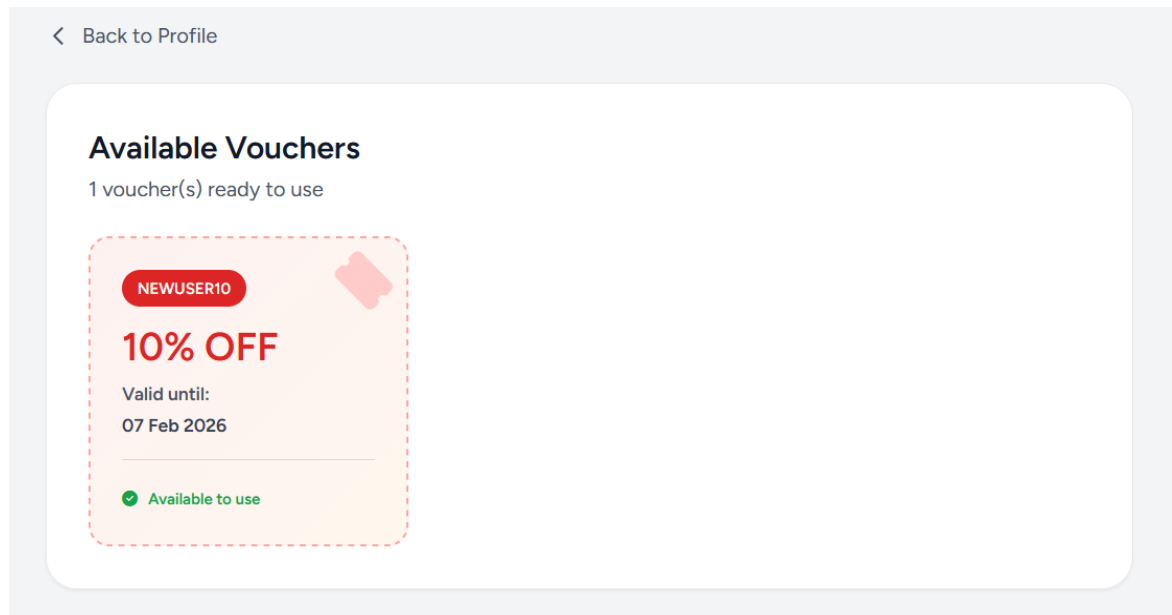
To start booking, customer need to create an account first. These two interfaces show the Register and Login screen for the Hasta Car Rental System, where customers can create a new account by providing basic information such as full name, email, and password. After that, customers can login using their email and password to access the customer portal.

SQL Statements:

```
INSERT INTO users (name, email, role, password, created_at,
updated_at)
VALUES (?, ?, 'customer', ?, NOW(), NOW());
INSERT INTO customers (user_id, status, loyalty_stamps,
created_at, updated_at)
VALUES (?, 'New', 0, NOW(), NOW());
```

Explanation:

This statement inserts a new user record into the users table that stores the user's name, email, hashed password, and the role is set to 'customer' by default for all registrations. After a user registers, a customer profile is automatically created in the customer's table. The user_id is linked to the newly created user, with the status set to 'New', and loyalty stamps start at 0.



SQL Statements:

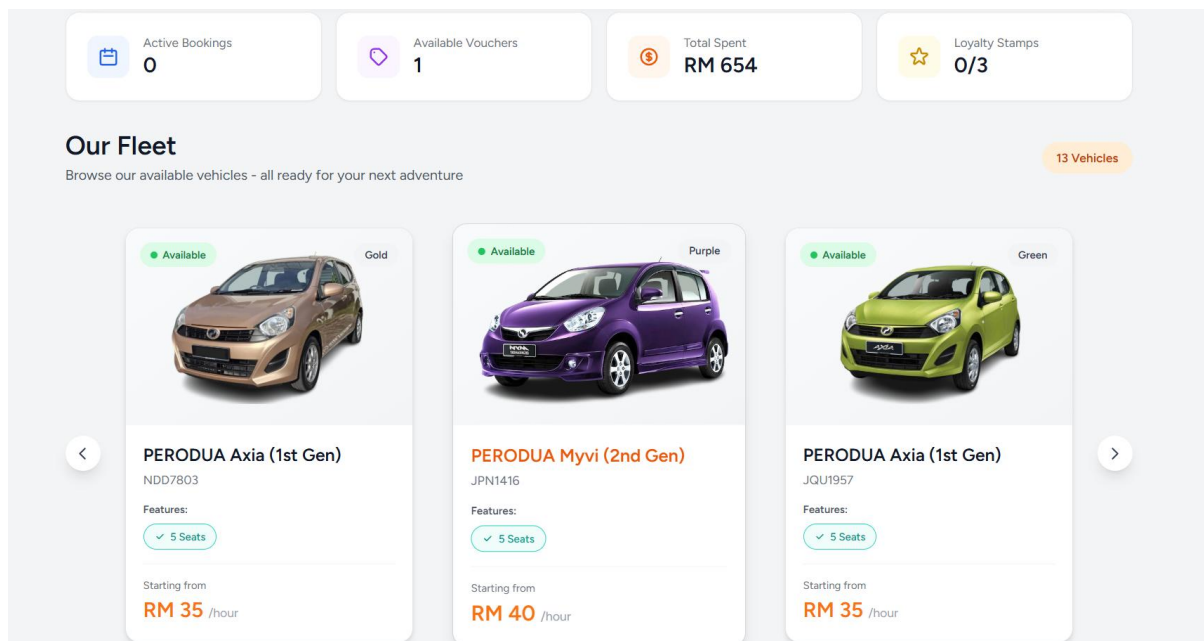
```
INSERT INTO vouchers (customer_id, type, issued_by,
expiry_date, usage, created_at, updated_at)
VALUES (?, 'NEWUSER10', NULL, DATE_ADD(NOW(), INTERVAL 30
DAY), 'Unused', NOW(), NOW());
```

```
SELECT id, customer_id, type, expiry_date, usage
FROM vouchers
WHERE customer_id = ? AND usage = 'Unused' AND expiry_date >=
CURDATE()
ORDER BY expiry_date ASC;
```

Explanation:

After the account creation, a welcome discount voucher (NEWUSER10) is automatically issued to new customers. The voucher is valid for 30 days from registration and has no staff issued since it is system generated. Then, the select statement will retrieve all vouchers that are available to use.

1.7.2 Customer Homepage



After customers login, they will be redirected to homepage, which is shown in interface above. The statistics of customers account is displayed, with the list of cars available.

SQL Statements:

```
-- Count active bookings
SELECT COUNT(*) AS active_bookings
FROM rentals
WHERE customer_id = ?
AND status IN ('Pending', 'Approved', 'Rented');

-- Count available vouchers
SELECT COUNT(*) AS available_vouchers
FROM vouchers
WHERE customer_id = ?
AND usage = 'Unused' AND expiry_date >= CURDATE();

-- Get total spent
SELECT COALESCE(SUM(amount), 0) AS total_spent
FROM payments
WHERE customer_id = ?;
```

```
-- Get loyalty stamps
SELECT loyalty_stamps, claimed_loyalty_vouchers
FROM customers
WHERE customer_id = ?;
```

Explanation:

These queries populate the dashboard statistics shown at the customer homepage including active bookings count, available vouchers, total spending, and loyalty stamp count.

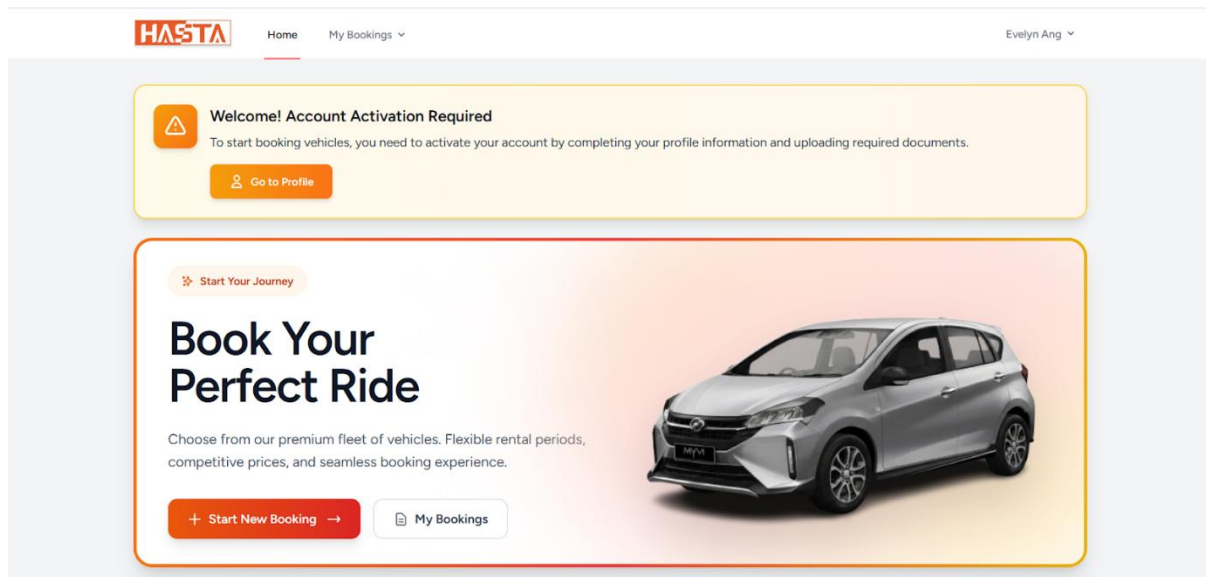
SQL Statements:

```
SELECT
    id,
    model,
    car_plate,
    color,
    car_pic,
    status,
    pricing_rate,
    features
FROM cars
WHERE status = 'Available'
ORDER BY model ASC;
```

Explanation:

The query selects all the available cars for booking to be displayed for customers.

1.7.3 Customer Activation



This screen shows that customers are not yet allowed to do booking if they haven't activated their account. They must complete their profile details and upload the required documents before they are allowed to proceed with booking.

SQL Statements:

```
SELECT id, user_id, status, activated_at
FROM customers
WHERE user_id = ?
LIMIT 1;
```

Explanation:

This query retrieves the customer's profile to check their activation status. Before allowing a customer to start a new booking, the system checks if their status is 'Active'. If the status is 'New', then the system displays the "Account Activation Required" banner prompting the customer to complete their activation request. If the status is 'Pending', it will show the activation request is under process, and customer still cannot book.

Activate Your Account

Please complete your profile information to activate your account and start booking.

Personal Information

IC / Passport Number *

Nationality *

UTM ID *

College Information

College *

College Block *

Faculty *

Home Address *

Contact Information

Contact Number *

Emergency Contact Number *

Driver's License Information

License Number *

License Expiry Date *

Document Uploads

Please upload the following documents in PDF format only.

UTM Student Card *

PDF format only (Max 5MB)

IC / Passport *

PDF format only (Max 5MB)

Driver's License *

PDF format only (Max 5MB)

☒ I confirm that all the information provided is accurate and I agree to the [terms and conditions](#). I understand that my account will be reviewed by staff before activation.

Interface above is the Account Activation Form where the customers are required to fill in their personal information, contact information, driver's license information, college information as well as upload all the required documents including UTM card, IC/ Passport and License before submitting the request for staff verification.

SQL Statements:

```
SELECT id, customer_id, status
FROM activations
WHERE customer_id = ? AND status = 'Pending'
LIMIT 1;
```

Explanation:

Before submitting a new activation request, the system will check if the customer already has a pending activation to prevent duplicate submissions while waiting for staff review.

SQL Statements:

```
INSERT INTO activations (  
customer_id, ic_passport, nationality, utm_id, contact,  
emergency_contact, license_no, license_expiry, college,  
college_block, faculty, address, utm_file, identity_file,  
license_file, status, created_at, updated_at)  
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?,  
'Pending', NOW(), NOW());
```

Explanation:

This statement inserts a customer's activation request with all form data and uploaded document paths. Status defaults to 'Pending' for staff review.

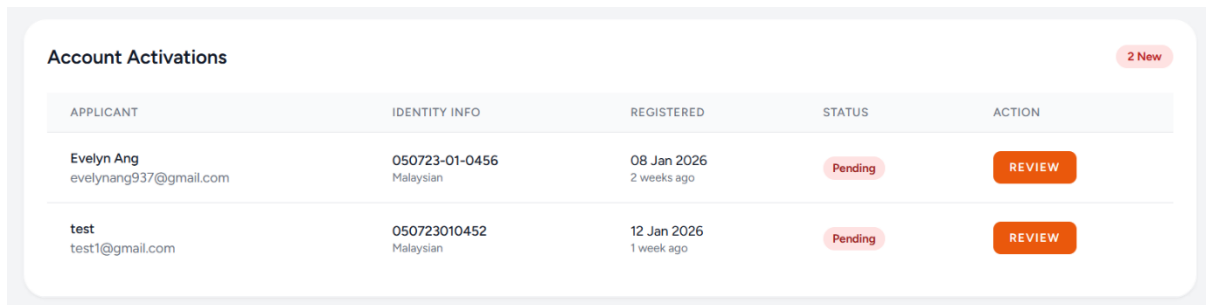
SQL Statements:

```
UPDATE customers  
SET status = 'Pending', updated_at = NOW()  
WHERE id = ?;
```

Explanation:

This statement updates customer status from 'New' to 'Pending' after form submission, indicating their account awaits verification.

1.7.4 Manage Activation Request



APPLICANT	IDENTITY INFO	REGISTERED	STATUS	ACTION
Evelyn Ang evelynang937@gmail.com	050723-01-0456 Malaysian	08 Jan 2026 2 weeks ago	Pending	REVIEW
test test1@gmail.com	050723010452 Malaysian	12 Jan 2026 1 week ago	Pending	REVIEW

This screen is for staff to view all the pending activation requests and click “Review” to verify the customer’s information and uploaded documents.

SQL Statements:

```
SELECT c.id, c.status, c.created_at, u.name, u.email,  
a.ic_passport, a.nationality  
FROM customers c  
INNER JOIN users u ON c.user_id = u.id  
LEFT JOIN activations a ON a.customer_id = c.id  
WHERE c.status = 'Pending'  
ORDER BY c.created_at ASC;
```

Explanation:

The statement above retrieves all customers with 'Pending' status along with their user info and activation details. Staff can review the applicant’s name, email, IC/Passport, nationality, and registration date.

Applicant Details

PERSONAL INFO

Full Name
Evelyn Ang

Email
evelynang937@gmail.com

IC / Passport
050723-01-0456

Nationality
Malaysian

Contact
016-7606839

Emergency
016-7156604

ACADEMIC INFO

UTM ID
A24CS0069

College
KTDI

Faculty
Faculty of Computing

LICENSE INFO

License No
A2345GTHB

Expiry
2028-01-30 00:00:00

APPROVE AND ACTIVATE

REJECT APPLICATION

Submitted Documents

Review carefully before approving

Identity Document

PDF OPEN

1 of 1



The screen images above are the activation review page for staff which display the applicant details with the submitted documents at the right. Staff can choose to approve or reject the activation after review. If staff want to reject the activation, he/she must provide a reason before they can confirm rejection.

SQL Statements:

```
SELECT
    a.id AS activation_id,
    u.name AS full_name,
    u.email,
    a.ic_passport,
    a.nationality,
    a.contact,
    a.emergency_contact,
    a.utm_id,
    a.college,
    a.faculty,
    a.license_no,
    a.license_expiry,
    a.identity_file,
    a.utm_file,
    a.license_file,
    a.status
FROM activations a
JOIN customers c ON a.customer_id = c.id
JOIN users u ON c.user_id = u.id
WHERE a.id = ?;
```

Explanation:

The statement above retrieves the customer profile and user information for the review page. Using JOIN to link the activation request to the customer record and link the customer record to the main user account to get the name and email.

SQL Statements:

```
UPDATE customers
SET ic_passport = ?, nationality = ?, utm_id = ?, contact = ?,
emergency_contact = ?, license_no = ?, license_expiry = ?,
college = ?, college_block = ?, faculty = ?, address = ?,
identity_file = ?, utm_file = ?, license_file = ?,
status = 'Active', activated_at = NOW(), updated_at = NOW()
WHERE id = ?;
```

```
UPDATE activations
```

```
SET status = 'Approved', activated_by = ?, updated_at = NOW()
WHERE id = ?;
```

Explanation:

If staff approved, the system would copy all verified information from activation record to customer profile and set status to 'Active', enabling the customer to start booking. After that, the system will mark the activation requests as 'Approved' and record the staff who approved.

SQL Statements:

```
UPDATE activations
```

```
SET status = 'Rejected', reject_reason = ?, activated_by = ?,
updated_at = NOW()
WHERE id = ?;
```

```
UPDATE customers
```

```
SET status = 'New', updated_at = NOW()
WHERE id = ?;
```

Explanation:

If staff reject the request, the system will mark activation as 'Rejected' with the staff's reason and records who rejected it. After that, the customer status will be set to 'New' so they can submit a new activation request after addressing the rejection reason.

1.7.5 Profile Page

My Profile

Full Name

Name cannot be changed

IC/Passport Number

IC/Passport cannot be changed

UTM ID

UTM ID cannot be changed

License Number

License Expiry Date

College/Residential

Faculty

Block/Unit

Full Address

SAVE CHANGES

SQL Statements:

SELECT

```
    u.name AS full_name,  
    c.ic_passport, c.utm_id, c.nationality, c.contact,  
    c.emergency_contact, c.license_no, c.license_expiry,  
    c.college, c.college_block, c.faculty, c.address  
FROM customers c  
JOIN users u ON c.user_id = u.id  
WHERE c.user_id = ?;
```

UPDATE customers

SET

```
    nationality = ?, contact = ?, emergency_contact = ?,  
    license_no = ?, license_expiry = ?, college = ?,  
    faculty = ?, college_block = ?, address = ?,  
    updated_at = NOW()  
WHERE user_id = ?;
```

Explanation:

The first select query is used to retrieve the customer's profile information to be displayed on the "My Profile" page. The second query updates the editable fields of the customer's profile.

This screen shows the page where customers can save their bank account for refund deposit later. However, they can also use different bank accounts during the return process. This can speed up the return process because they do not need to enter the bank details every time.

SQL Statements:

```
SELECT id, user_id, bank_name, bank_account
FROM customers
WHERE user_id = ?;
```

```
UPDATE customers
SET bank_name = ?, bank_account = ?, updated_at = NOW()
WHERE user_id = ?;
```

Explanation:

This first statement retrieves customer's existing bank details to pre-fill the form. If no bank details are saved, the form shows empty fields. The second statement saves the customer's bank name and account number for future refund deposits after completing return.

1.7.6 Booking Process

1 Make a Reservation
Step 1 of 3 - Select your journey dates and location

Pickup Details

Pickup Date *
dd/mm/yyyy

Pickup Hour *
-- Select Hour --

Note: You can pickup the car 30 minutes before your scheduled time. Late returns will incur a penalty.

Pickup Location *
-- Select Location --

Return Details

Return Date *
dd/mm/yyyy

Return Hour *
-- Select Hour --

Return Location *
-- Select Location --

-- Back Reset Next: Select Car

This interface shows the Step 1 of the booking process, where customers need to select pickup and return date, time, and location. If the location is outside of UTM, then delivery fees will be charged during payment. After completing all the required fields, customers can click “Next” to proceed to car selection.

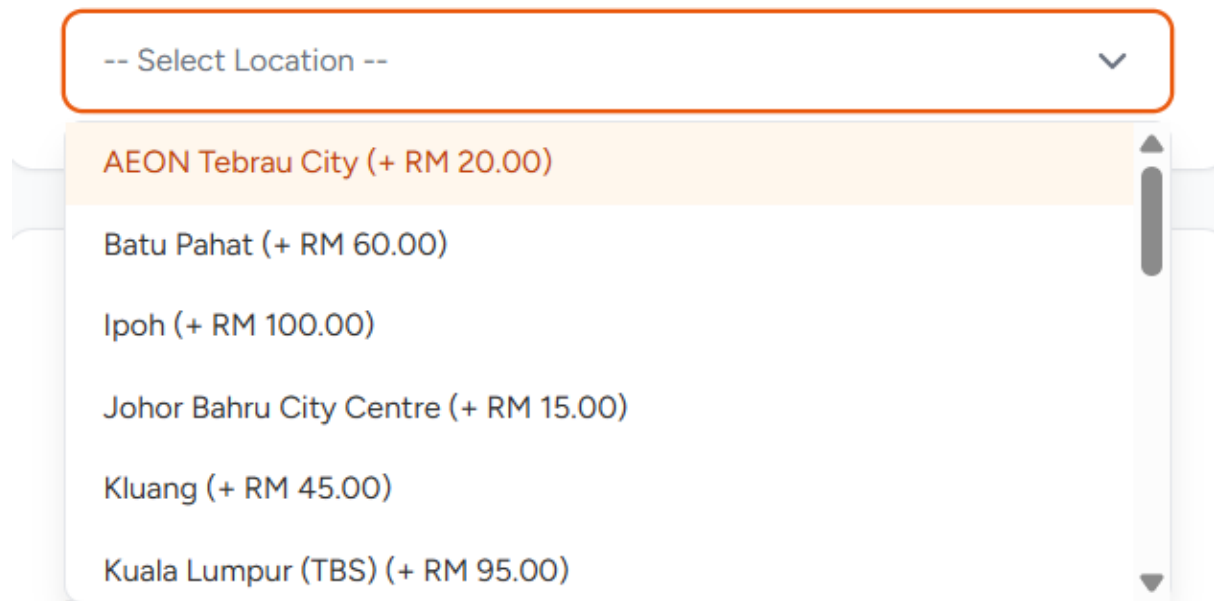
SQL Statements:

```
SELECT id, user_id, status
FROM customers
WHERE user_id = ?;
```

Explanation:

This statement verifies the customer exists and has 'Active' status before allowing booking. Redirects to profile activation if status is 'New' or 'Pending'.

Pickup Location *



-- Select Location --

- AEON Tebrau City (+ RM 20.00)
- Batu Pahat (+ RM 60.00)
- Ipoh (+ RM 100.00)
- Johor Bahru City Centre (+ RM 15.00)
- Kluang (+ RM 45.00)
- Kuala Lumpur (TBS) (+ RM 95.00)

SQL Statements:

```
SELECT id, location, fee
FROM deliveries
ORDER BY location ASC;
```

Explanation:

This statement retrieves all available pickup and return locations for the dropdown menus, ordered alphabetically.

SQL Statements:

```
SELECT location, fee
FROM deliveries
WHERE location = ?;
```

Explanation:

After the customer select the pickup / return location, this statement retrieves the delivery fee for selected pickup and return location. If the location is 'Hasta Travel', the fee is RM0.

2

Select Your Car

Step 2 of 3 - Choose your preferred vehicle

Your Booking Details

Pickup:

22 Jan 2026, 01:00 PM

Location: UTM Main Gate

Return:

23 Jan 2026, 01:00 PM

Location: Skudai Town

Duration: 24 hours

Available Cars (12)



HONDA VARIO 160 CC

JWD9496

Colour:

Yellow

Fuel Level:

8 bar

Mileage:

1,111,809 km

Features:

✓ DASH 125

Price per day

RM 43.00

View all rates →

Total Cost

RM 43.00

for 1 day



PERODUA Axia (1st Gen)

NDD7803

Colour:

Gold

Fuel Level:

8 bar

Mileage:

94,095 km

Features:

✓ 5 Seats

Price per day

RM 120.00

View all rates →

Total Cost

RM 120.00

for 1 day



PERODUA Axia (1st Gen)

JQU1957

Colour:

Green

Fuel Level:

8 bar

Mileage:

316,957 km

Features:

✓ 5 Seats

Price per day

RM 120.00

View all rates →

Total Cost

RM 120.00

for 1 day

← Back

Continue to Payment →

This screen image shows Step 2 of the booking flow, where the system will filter available cars based on the selected booking period and display the list to customers. Car details including mileage, special features and pricing rates will be displayed to customers. Customers can select their preferred car and then proceed to the payment page.

SQL Statements:

```
SELECT id, car_plate, model, color, car_pic, status, mileage,
fuel_bar, pricing_rate, features
FROM cars
WHERE status IN ('Available', 'Rented')
    AND id NOT IN (
        SELECT car_id FROM rentals
        WHERE (
            (
                status IN ('Pending', 'Approved', 'Picked Up')
                AND CONCAT(pickup_date, ' ', pickup_time) <= ?
                AND DATE_ADD(CONCAT(return_date, ' ', return_time)
, INTERVAL 1 HOUR) > ?
            )
            OR (
                status IN ('Returned', 'Late Returned')
                AND DATE_ADD(updated_at, INTERVAL 1 HOUR) > ?
                AND updated_at >= DATE_SUB(?, INTERVAL 1 DAY)
            )
        )
    )
ORDER BY model ASC;
```

Explanation:

This statement will fetch the cars with 'Available' or 'Rented' status that don't have conflicting bookings. The system will apply a 1-hour cooldown after return time to allow for car cleaning or inspection.

3

Payment & Confirmation

Step 3 of 3 - Review and submit your booking

Booking Summary

Vehicle:	PERODUA Axia (1st Gen) (MCP6113)
Pickup:	22 Jan 2026, 01:00 PM
Pickup Location:	UTM Main Gate
Return:	23 Jan 2026, 01:00 PM
Return Location:	Skudai Town
Duration:	1 day 0 hours
Rental Cost:	RM 120.00
Delivery Fee:	Return only RM 10.00
Deposit:	RM 50.00

Note: RM50.00 deposit will be refunded to your bank account within 7 working days after the rental is completed and the car is returned in good condition and on time.

Apply Voucher

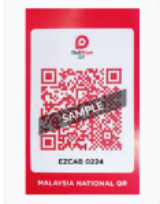
NEWUSER10 (10% OFF)

Rental Cost:	RM 120.00
Delivery Fee:	RM 10.00
Voucher Discount (10%):	- RM 13.00
Deposit (Refundable):	RM 50.00
Total Amount:	RM 167.00

Payment Instructions

Please complete your payment using one of the methods below, then upload your receipt to confirm the booking.

Scan QR Code to Pay



OR

Bank Transfer Details

Bank Name:	Maybank
Account Name:	Hasta Travel & Rental Sdn Bhd
Account Number:	123456789876
Amount to Pay:	RM 180.00

Upload Payment Receipt

After making the payment, please upload your receipt below

Browse...

Receipt-Example.pdf

Accepted formats: JPG, PNG, PDF (Max 5MB)

I confirm that I have made the payment and uploaded the correct receipt. I agree to the Terms and Conditions and understand that this booking is subject to payment verification.

Back

Submit Booking ✓

This screen is Step 3: Payment & Confirmation where customers can review the booking summary and apply an available voucher to get a discount for the rental cost. Then they can choose the payment method either DuitNow QR or bank transfer. After that, the customers need to upload the payment receipt and agree to the Terms and Conditions before submitting the booking.

SQL Statements:

```
SELECT id, car_plate, model, color, car_pic, pricing_rate,
features
FROM cars
WHERE id = ?;
```

Explanation:

This statement retrieves the selected car details in step 2 to display in the booking summary and calculate pricing based on tier rates.

SQL Statements:

```
SELECT id, customer_id, type, expiry_date, usage
FROM vouchers
WHERE customer_id = ? AND usage = 'Unused'
AND expiry_date >= CURDATE();
```

Explanation:

This statement is used to retrieve all valid, unused vouchers in a dropdown format for the customer to apply for a discount.

```
SELECT COUNT(*) > 0 AS has_conflict
FROM rentals
WHERE car_id = ?
    AND status IN ('Pending', 'Approved',
    'Picked Up', 'Returned')
    AND CONCAT(pickup_date, ' ', pickup_time)
        < CONCAT(?, ' ', ?)
    AND DATE_ADD(CONCAT(return_date, ' ', return_time),
        INTERVAL 1 HOUR) > CONCAT(?, ' ', ?);
```

Explanation:

This statement is to double-check car availability before processing payment to prevent race conditions when multiple users book the same car.

SQL Statements:

```
INSERT INTO rentals (  
customer_id, car_id, pickup_date, pickup_time, pickup_location,  
return_date, return_time, return_location, status, voucher_id,  
created_at, updated_at)  
VALUES (?, ?, ?, ?, ?, ?, ?, ?, 'Pending', ?, NOW(), NOW());
```

Explanation:

This statement creates a new rental record with 'Pending' status. It also includes the voucher ID involved if a customer applied a discount.

SQL Statements:

```
UPDATE cars  
SET status = 'Rented', updated_at = NOW()  
WHERE id = ?;
```

Explanation:

After that, this statement marks car as 'Rented' to prevent other customers from selecting it.

SQL Statements:

```
INSERT INTO payments (  
rental_id, customer_id, amount, receipt_image, refund_bank,  
refund_owner, refund_account, created_at, updated_at)  
VALUES (?, ?, ?, ?, ?, ?, ?, NOW(), NOW());
```

Explanation:

This statement creates a payment record storing the amount paid and uploaded receipt filename.

SQL Statements:

```
UPDATE vouchers
SET usage = 'Used', updated_at = NOW()
WHERE id = ?;
```

Explanation:

If a customer applied a voucher during payment, this statement update the voucher status as 'Used' so it cannot be reused.

The screenshot displays a user interface for a car rental booking. At the top, there is a navigation link "Back to My Bookings". The main heading is "Booking #295" with a "Pending" status tag and a link to "View booking details and status". Below this, the "Vehicle Details" section shows a white Perodua Axia (1st Gen) with license plate MCP6113, 83,060 km, 8/8 fuel, and 5 seats. The "Payment Summary" section lists Payment ID #295 and Amount Paid RM 135.00, with a link to "View Payment Receipt". The "Rental Period" section shows a pickup on 12 Jan 2026 at 11:00 AM from UTM Main Gate and a return on 12 Jan 2026 at 8:00 PM to UTM Library, with a total duration of 9 hours. The "Booking Timeline" section shows the booking was created on 12 Jan 2026 at 11:31 AM. An "Actions" section at the bottom contains a "Cancel Booking" button.

← Back to My Bookings

Booking #295 Pending
View booking details and status

Vehicle Details

 **PERODUA Axia (1st Gen)**
MCP6113
83,060 km 8/8
5 Seats

Payment Summary

Payment ID #295
Amount Paid RM 135.00
View Payment Receipt

Rental Period

PICKUP
12 Jan 2026
11:00 AM
UTM Main Gate

RETURN
12 Jan 2026
8:00 PM
UTM Library

Total Duration: 9 hours (9 hours)

Booking Timeline

Booking Created
12 Jan 2026, 11:31 AM

Actions

Cancel Booking

This screen shows that the booking is still pending after the customer submits the booking and needs to wait for the staff approval. The customer can click the “Details” to view all the booking details including car information, rental period, payment summary and booking timeline. The customer can only cancel the booking at least 24 hours before the scheduled pickup time.

SQL Statements:

```
SELECT r.id, r.customer_id, r.car_id, r.pickup_date,  
r.pickup_time, r.pickup_location, r.return_date,  
r.return_time, r.return_location, r.status, r.created_at  
FROM rentals r  
WHERE r.id = ? AND r.customer_id = ?;
```

Explanation:

This statement retrieves the rental record selected that belongs to the logged-in customer.

SQL Statements:

```
SELECT id, car_plate, model, color, car_pic, mileage,  
fuel_bar, features  
FROM cars  
WHERE id = ?;
```

Explanation:

This statement retrieves car details for the Vehicle Details section including mileage, fuel level, and features.

SQL Statements:

```
SELECT p.id, p.rental_id, p.amount, p.refund_amount,  
p.refund_at, p.refund_receipt, p.receipt_image  
FROM payments p  
WHERE p.rental_id = ?;
```

Explanation:

This statement retrieves the payment info for the Payment Summary section showing payment ID, amount paid and payment receipt. It will also show the refund amount, refund time and refund receipt after staff have issued a refund.

1.7.7 Manage Booking

Rental Details

CUSTOMER

Evelyn Ang

evelynang937@gmail.com

MANAGED BY

N/A

CAR

PERODUA Axia (1st Gen)

MCP6113

CURRENT STATUS

Pending

RENTAL SCHEDULE

Pickup

DATE & TIME

22 Jan 2026 @ 01:00 PM

LOCATION

UTM Main Gate

Return

DATE & TIME

23 Jan 2026 @ 01:00 PM

LOCATION

Skudai Town

PAYMENT INFO

TOTAL AMOUNT

RM 167.00

Payment Status

Paid

APPROVE RENTAL

REJECT RENTAL

Supporting Documents

Review carefully before approving

Payment Receipt

PDFOPEN

1 of 1

Detail

-RM200.00

Transaction Type	Transfer to Wallet
Transfer To	Wong Wai Long
Transaction Details	PS4 deposit.
Payment Method	eWallet Balance
Date/Time	12/02/2021 12:15:33
eWallet Ref No.	20210212111217000101017179 28039999191
Status	Successful
Transaction No.	1a98ee8f 6c13 4a10 a73a 8625d5cb4528

This screen is the rental approval page where staff can review all the rental details and verify the uploaded payment receipt. After that, staff can choose to approve or reject the rental after review. If staff want to reject, they must provide a reason before they can confirm rejection.

SQL Statements:

```
SELECT r.id, r.customer_id, r.car_id, r.pickup_date,  
r.pickup_time, r.pickup_location, r.return_date, r.return_time,  
r.return_location, r.status, r.managed_by, r.approved_at  
FROM rentals r  
WHERE r.id = ?;
```

```
SELECT c.id, c.user_id, u.name, u.email  
FROM customers c  
INNER JOIN users u ON c.user_id = u.id  
WHERE c.id = ?;
```

```
SELECT id, car_plate, model, color, car_pic  
FROM cars  
WHERE id = ?;
```

```
SELECT id, rental_id, amount, receipt_image  
FROM payments  
WHERE rental_id = ?;
```

Explanation:

The statement retrieves rental details of rental record involved/selected, with the customer info, car info, and payment receipt for staff to review before approval.

SQL Statement:

```
UPDATE rentals
SET status = 'Approved', managed_by = ?,
approved_at = NOW(), updated_at = NOW()
WHERE id = ?;
```

Explanation:

If staff approve the rental, then this statement updates the rental status to 'Approved' and records which staff member approved it.

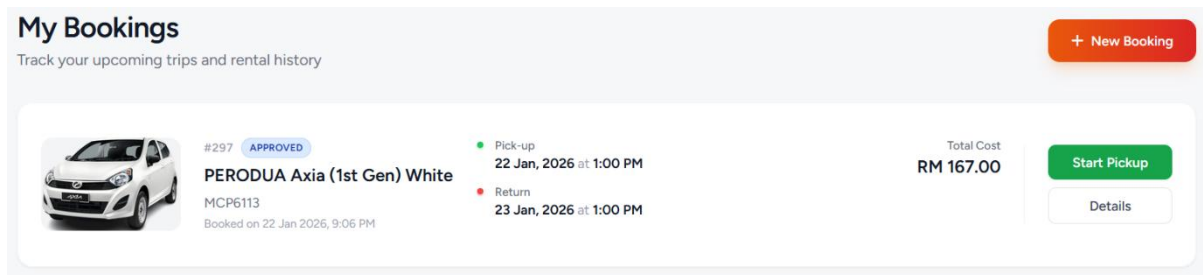
SQL Statements:

```
UPDATE rentals
SET status = 'Rejected', disapprove_reason = ?,
managed_by = ?, approved_at = NOW(), updated_at = NOW()
WHERE id = ?;
```

Explanation:

If staff reject the rental, then this statement updates the rental status to 'Rejected' with a rejection reason provided by staff.

1.7.8 View Booking List



The screens above show the booking timeline is updated after approval. The customer can also click “Pickup” 30 minutes before the scheduled pickup time to collect the car.

SQL Statements:

```
SELECT r.id, r.customer_id, r.car_id, r.pickup_date,  
r.pickup_time, r.pickup_location, r.return_date, r.return_time,  
r.return_location, r.status, r.created_at, r.approved_at  
FROM rentals r  
WHERE r.customer_id = ?  
ORDER BY r.created_at DESC  
LIMIT 10 OFFSET ?;
```

Explanation:

The statement retrieves a paginated list of customer bookings ordered by creation date (newest first).

SQL Statements:

```
SELECT id, car_plate, model, color, car_pic  
FROM cars  
WHERE id = ?;
```

```
SELECT id, rental_id, amount  
FROM payments  
WHERE rental_id = ?;
```

Explanation:

For each rental, retrieve the car details and payment amount to display in the booking card.

1.7.9 Pickup Process

The image displays two screenshots of a car rental application interface.

Left Screenshot: Review Pickup & Booking Details

This screen is titled "Review Pickup & Booking Details" with a subtitle "Confirm your booking information before continuing". It features a car icon at the top. The content is organized into three sections:

- Booking Details:** Includes Booking ID (#297), Status (Approved), and Customer (Evelyn Ang).
- Vehicle Details:** Includes Make & Model (PERODUA Axia (1st Gen)), License Plate (MCP6113), and Color (White).
- Pickup & Return:** Includes Pickup Date & Time (January 22, 2026 at 01:00 PM), Pickup Location (UTM Main Gate), Return Date & Time (January 23, 2026 at 01:00 PM), and Return Location (Skudai Town).

Below these sections is a "Before You Proceed" section with two instructions: "Inspect the vehicle with staff before signing documents" and "Note existing scratches/dents; take clear photos." A "Next Step →" button is at the bottom right.

Right Screenshot: Documents & Forms

This screen is titled "Documents & Forms" with a subtitle "Download, sign, and upload the required documents". It features a document icon at the top. The content is organized into two sections:

- Rental Agreement Form:** Includes a "Rental Agreement" section with a "Download" button and a "Document uploaded" status (5_6280648389109488353.pdf).
- Vehicle Inspection Form:** Includes an "Inspection Checklist" section with a "Download" button and a "Document uploaded" status (5_6280648389109488354.pdf).

At the bottom, there are "← Previous" and "Next Step →" buttons.

These screens show the Step 1 and Step 2 of the pickup process. First the customer will review the pickup and booking details. Next, the customer is required to download the agreement and inspection forms, sign them, and upload the signed forms before proceeding to the next step.

SQL Statements:

```
SELECT r.id, r.customer_id, r.car_id, r.pickup_date,
r.pickup_time, r.pickup_location, r.return_date, r.return_time,
r.return_location, r.status
FROM rentals r
WHERE r.id = ? AND r.customer_id = ? AND r.status IN ('Approved',
'Late Returned');
```

Explanation:

The statement retrieves the rental that is approved and ready for pickup. Only allows pickups before 30 minutes of scheduled pickup time.

SQL Statements:

```
SELECT id, car_plate, model, color, car_pic  
FROM cars  
WHERE id = ?;
```

Explanation:

The statement retrieves car details to display in the booking summary including Model, License Plate and Color.

SQL Statements:

```
UPDATE rentals  
SET agreement_form = ?, updated_at = NOW()  
WHERE id = ?;
```

```
UPDATE rentals  
SET inspection_form = ?, updated_at = NOW()  
WHERE id = ?;
```

Explanation:

The statement will save the file path of the signed agreement form and inspection form uploaded by the customer.



Vehicle Documentation

Take photos of the vehicle from all angles for your protection

Photography Tips

- Ensure good lighting and clear visibility
- Capture the entire vehicle in each photo
- Document any existing damages or scratches
- Make sure the odometer reading is clearly visible

1 Front View *
Capture the front of the vehicle



✓ Photo uploaded

2 Back View *
Capture the rear of the vehicle



✓ Photo uploaded

3 Left Side View *
Capture the left side of the vehicle



✓ Photo uploaded

4 Right Side View *
Capture the right side of the vehicle



✓ Photo uploaded

5 Dashboard (Odometer) *
Capture the odometer reading clearly



✓ Photo uploaded

The screen above is the pickup process Step 3 where the customer needs to upload required car photos from multiple angles including front, back, left and right and also the car odometer as proof of the car's condition during pickup. An optional section is provided if the customer finds any damage to the car at pickup.

SQL Statements:

```
UPDATE rentals
SET pickup_pic = ?, updated_at = NOW()
WHERE id = ?;
```

Explanation:

The statement saves all the uploaded photos including front, back, left, right, dashboard, and damage which are stored in a JSON object in the pickup_pic column.



Almost Done!

Review your submission and complete the pickup process

Pickup Summary

Vehicle Details

Model: PERODUA Alza (2nd Gen)

License Plate: UUM1095

Color: Silver

Rental Period

Pickup: Jan 12, 2026 10:00 AM

Return: Jan 12, 2026 09:00 PM

Duration: 11 hours

Documents Status



Agreement Form
Signed and uploaded

Complete



Inspection Form
Completed and uploaded

Complete

Vehicle Photos



Front View



Back View



Left Side



Right Side



Dashboard

Final Confirmation

By completing this pickup process, you acknowledge that:

-  You have inspected the vehicle and documented its current condition
-  You agree to return the vehicle in the same condition
-  You will be responsible for any damages during the rental period
-  You have read and agree to the rental terms and conditions
-  All provided information and documents are accurate

Important Reminders

-  Collect your vehicle keys and all necessary documents from staff
-  Note the current fuel level and return with the same level
-  Keep emergency contact numbers handy during your rental
-  Return the vehicle on time to avoid additional charges

Late Return Policy:
Deposit forfeited due to late return exceeding 30 minutes.
Additional penalty charges apply at RM 50.00 per hour.
The forfeited deposit is deducted from the total penalty.

☒ I confirm that I have reviewed all information above and agree to proceed with the vehicle pickup.

← Previous

➡ Complete Pickup

The interfaces above show the Step 4 of the pickup process where the customer has to review the pickup summary and confirm that all the required information is correct before completing the pickup. After submission, the system will update the booking timeline as Picked Up.

SQL Statements:

```
UPDATE rentals
SET status = 'Picked Up', updated_at = NOW()
WHERE id = ?;
```

Explanation:

The statement updates rental status from 'Approved' to 'Picked Up' after the customer confirms and completes the pickup process.

1.7.10 Return Process



Review Return Details

Please review the information below before proceeding

 **Vehicle Information**

Model	PERODUA Axia (1st Gen)	Car Plate	MCP6113
Color	White		

 **Rental Period**

Pickup Date	January 22, 2026	Pickup Time	01:00 PM
Return Date	January 23, 2026	Return Time	01:00 PM
Total Duration	1 days		

 **Return Location**

Location	Skudai Town
----------	-------------

 **Important Return Checklist**

- Ensure the vehicle is clean and in good condition
- Check for all personal belongings before returning
- Fuel level should match the pickup condition
- Report any damages or issues that occurred during rental
- Have your car keys ready for handover

Next Step →

This interface shows the return process Step 1 where the customer can review the vehicle information, rental period, and location as well as the checklist before proceed to the next step.

SQL Statement:

```
SELECT    r.id,      r.customer_id,      r.car_id,      r.pickup_date,
r.pickup_time, r.pickup_location,
r.return_date, r.return_time, r.return_location, r.status
FROM rentals r
WHERE r.id = ? AND r.customer_id = ? AND r.status IN
('Picked Up', 'Late Returned');
```

Explanation:

The statement retrieves the rental record with status 'Picked Up' or 'Late Returned' to begin the return process. Only those with these two status can initiate return process.

SQL Statement:

```
SELECT
    r.id AS rental_id,
    c.model AS car_model,
    c.car_plate,
    c.color AS car_color,
    c.car_pic,
    r.pickup_date,
    r.pickup_time,
    r.return_date,
    r.return_time,
    DATEDIFF(r.return_date, r.pickup_date)
    AS total_duration_days,
    d.location AS return_location_name
FROM rentals r
JOIN cars c ON r.car_id = c.id
LEFT JOIN deliveries d ON r.return_location = d.id
WHERE r.id = ?;
```

Explanation:

The statement retrieves the rental information of car involved and location involved using join.



Vehicle Return Documentation

Take photos of the vehicle from all angles including the car key

Photography Tips

- Ensure good lighting and clear visibility
- Document any damages that occurred during rental
- Make sure the odometer reading is clearly visible
- Capture the car key with any tags or accessories

1 Front View
Capture the front of the vehicle



✓ Photo uploaded

2 Back View
Capture the rear of the vehicle



✓ Photo uploaded

3 Left Side View
Capture the left side of the vehicle



✓ Photo uploaded

4 Right Side View
Capture the right side of the vehicle



✓ Photo uploaded

5 Dashboard (Odometer)
Capture the odometer reading clearly



✓ Photo uploaded

6 Car Key
Capture the car key and any accessories



✓ Photo uploaded

← Previous

Upload & Continue →

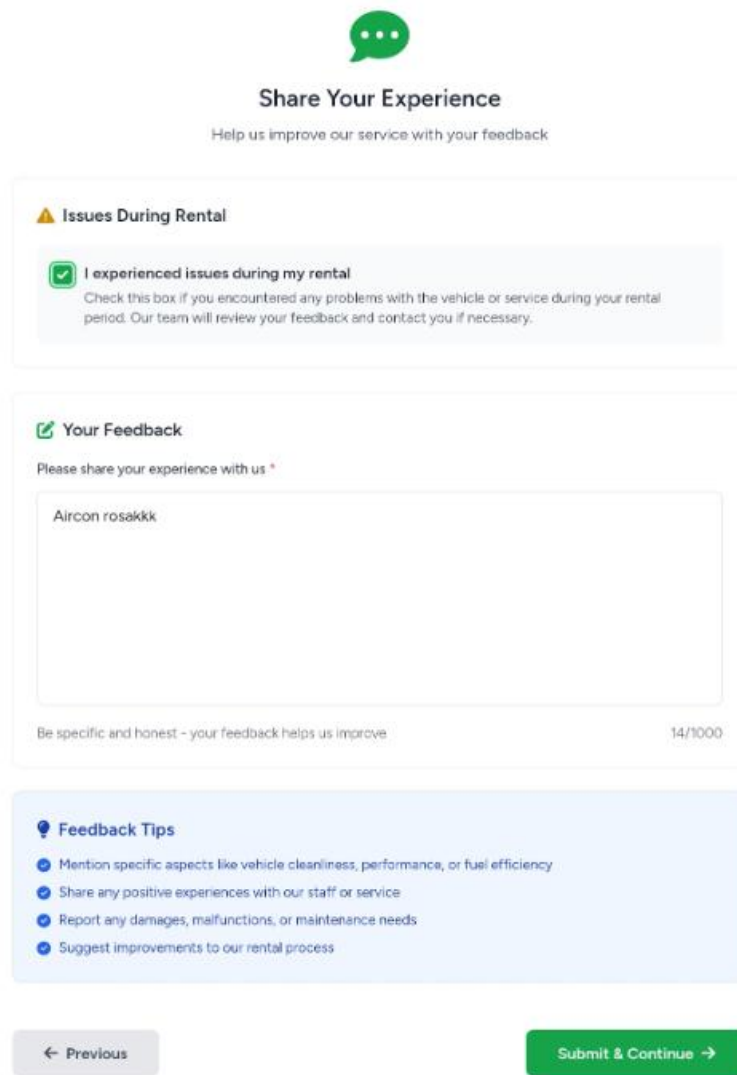
The screen above is the return process Step 2 where the customer needs to upload required car photos from multiple angles including front, back, left, and right, the car odometer and also the car key as proof of the car's condition before return.

SQL Statement:

```
UPDATE rentals
SET return_pic = ?, updated_at = NOW()
WHERE id = ?;
```

Explanation:

The statement saves all the uploaded return photos which are stored in a JSON object in the return_pic column.



The form is titled "Share Your Experience" with a subtitle "Help us improve our service with your feedback". It features a green speech bubble icon at the top. The first section, "Issues During Rental", includes a checkbox labeled "I experienced issues during my rental" with a green checkmark icon and a note: "Check this box if you encountered any problems with the vehicle or service during your rental period. Our team will review your feedback and contact you if necessary." The second section, "Your Feedback", has a green checkmark icon and the text "Please share your experience with us *". Below this is a text input field containing "Aircon rosakkk". At the bottom of this section is a character count "14/1000" and the instruction "Be specific and honest - your feedback helps us improve". The third section, "Feedback Tips", is a light blue box with a lightbulb icon and four bullet points: "Mention specific aspects like vehicle cleanliness, performance, or fuel efficiency", "Share any positive experiences with our staff or service", "Report any damages, malfunctions, or maintenance needs", and "Suggest improvements to our rental process". At the bottom are two buttons: "Previous" (grey) and "Submit & Continue" (green).

This screen shows the Step 3 in return process where the customer can report an issue if it occurred during rental and submit feedback in order to help improve the services. After completing the feedback, the customer can click submit & continue to proceed to the last step.

SQL Statements:

```
UPDATE rentals
SET feedback = ?, updated_at = NOW()
WHERE id = ?;
```

Explanation:

This statement saves the customer's feedback after the rental.



Refund & Complete Return

Review your information and set up refund details

Return Summary

Vehicle Details

Vehicle: PERODUA Alza (2nd Gen)

Car Plate: UUM1095

Return Location: Paradigm Mall JB

Payment Details

Rental Amount: RM 203.00

Refund Amount: RM 50.00

Duration: 11 hours

Return Photos








Your Feedback

⚠ Issues reported during rental

"Aircan rosakkk"

Refund Account Details

☒ Use Existing Bank Account

Account Holder: Evelyn Ang
 Bank Name: Maybank
 Account Number: 9876****2111

☐ Use Different Bank Account

Important Information

- Refund will be processed within 5-7 business days
- Please ensure bank account details are accurate
- You will receive an email confirmation once refund is processed
- Contact support if you don't receive refund within 7 days

☒ I confirm that:

- All provided information is accurate and complete
- I have returned the vehicle in good condition
- I have handed over all keys and accessories
- I agree to the refund terms and conditions

← Previous
Complete Return

Interfaces above show the Step 4: Refund & Complete Return where the customer can review the return summary and select either the saved bank account or a different bank account for the deposit refund. If the car is returned late, then a penalty will be charged to the customer. After completion, the system will update the booking timeline.

SQL Statements:

```
SELECT    r.id,      r.customer_id,      r.car_id,      r.pickup_date,
r.pickup_time, r.return_date,r.return_time, r.return_location,
r.status, r.return_pic, r.feedback
FROM rentals r
WHERE r.id = ? AND r.customer_id = ?;
```

```
SELECT id, rental_id, amount, refund_amount
FROM payments
WHERE rental_id = ?;
```

```
SELECT id, user_id, bank_name, bank_account
FROM customers
WHERE user_id = ?;
```

Explanation:

This statement retrieves rental details, payment info, and saved bank account for displaying return summary and refund account details.

SQL Statements:

```
UPDATE payments
SET refund_owner = ?, refund_bank = ?, refund_account = ?,
refund_amount = 50.00, refund_at = NULL, updated_at = NOW()
WHERE rental_id = ?;
```

```
UPDATE rentals
SET status = 'Returned', updated_at = NOW()
WHERE id = ?;
```

Explanation:

This statement sets the refund amount to RM50 if returned on time which means return within a 30-minute cooldown period and updates rental status to 'Returned' for on-time returns.

SQL Statements:

```
UPDATE cars
SET status = ?, updated_at = NOW()
WHERE id = ?;
```

Explanation:

This statement sets car status to 'Available' if normal return or 'Maintenance' if customer reported issues.

SQL Statements:

```
UPDATE customers
SET loyalty_stamps = loyalty_stamps + 1, updated_at = NOW()
WHERE id = ?;
```

Explanation:

This statement will award 1 loyalty stamp to the customer if the booking duration exceeds 12 hours.

SQL Statements:

```
UPDATE payments
SET refund_owner = ?, refund_bank = ?, refund_account = ?,
    refund_amount = 0, refund_at = NULL, updated_at = NOW()
WHERE rental_id = ?;
```

```
INSERT INTO penalties
(customer_id, rental_id, amount, receipt,
issued_by, approved_at, created_at, updated_at)
VALUES (?, ?, ?, NULL, NULL, NULL, NOW(), NOW());
```

```
UPDATE customers
SET status = 'Blacklisted', updated_at = NOW()
WHERE id = ?;
```

```
UPDATE rentals
SET status = 'Late Returned', updated_at = NOW()
WHERE id = ?;
```

Explanation:

If late return, then set the refund amount to RM0 and create a penalty record for late returns. Then the customer account will become blacklisted if they incur a late return penalty. Then rental status will also be set to 'Late Returned'.

1.7.11 Staff Manage Return

Return Photos

Front View

Back View

Left View

Right View

Dashboard / Odometer

Customer Feedback

Issue Reported

Aircon rosakkk

MARK RESOLVED

Update Car Status

Mileage (km)

100010

Fuel (0-8)

7 Bars

UPDATE DETAILS

This screen shows how the staff is going to manage the return process. Staff need to refer to the uploaded return photos to update the mileage and fuel level of the car. They also need to check the reported issue and feedback, staff can click “Mark Resolved” if the issue is already solved.

SQL Statements:

```
SELECT r.id, r.customer_id, r.car_id, r.pickup_date,  
r.pickup_time, r.return_date, r.return_time, r.return_pic,  
r.feedback  
FROM rentals r  
WHERE r.id = ?;
```

```
SELECT id, car_plate, model, mileage, fuel_bar  
FROM cars  
WHERE id = ?;
```

```
SELECT c.id, u.name  
FROM customers c  
INNER JOIN users u ON c.user_id = u.id  
WHERE c.id = ?;
```

Explanation:

This statement retrieves rental details with return photos, customer feedback, and current car details for staff to verify.

SQL Statement:

```
UPDATE cars  
SET mileage = ?, fuel_bar = ?, updated_at = NOW()  
WHERE id = ?;
```

Explanation:

Staff updates the car's mileage from odometer photo and fuel level from dashboard based on return photos.

SQL Statement:

```
UPDATE cars
SET status = 'Available', updated_at = NOW()
WHERE id = ?;
```

Explanation:

Staff marks the reported issue as resolved after addressing customer feedback and set the car status back to 'Available'.





Issue Refund

Review details and upload receipt.

REFUND TYPE	Deposit Refund
AMOUNT	RM 50.00
CUSTOMER BANK DETAILS	
Bank Name:	Maybank
Bank Account:	Copy 98765432111
Account Holder:	Evelyn Ang

Browse...

Receipt-Example.pdf

Cancel

Confirm Issue

This interface shows the issue refund window for staff. After staff inspect the car, they need to refund the deposit to the customer if the customer returns on time by uploading the refund receipt as the proof. After refunding the deposit, the rental status will become Completed.

SQL Statements:

```
SELECT p.id, p.rental_id, p.customer_id, p.amount,  
p.refund_amount, p.refund_owner, p.refund_bank,  
p.refund_account, p.refund_type  
FROM payments p  
WHERE p.rental_id = ?;
```

Explanation:

This statement retrieves payment record with customer's bank details for refund purposes.

SQL Statements:

```
UPDATE payments  
SET refund_at = NOW(), refund_receipt = ?,  
refund_amount = 50.00, updated_at = NOW()  
WHERE rental_id = ?;
```

Explanation:

This statement records refund issuance with receipt filename.

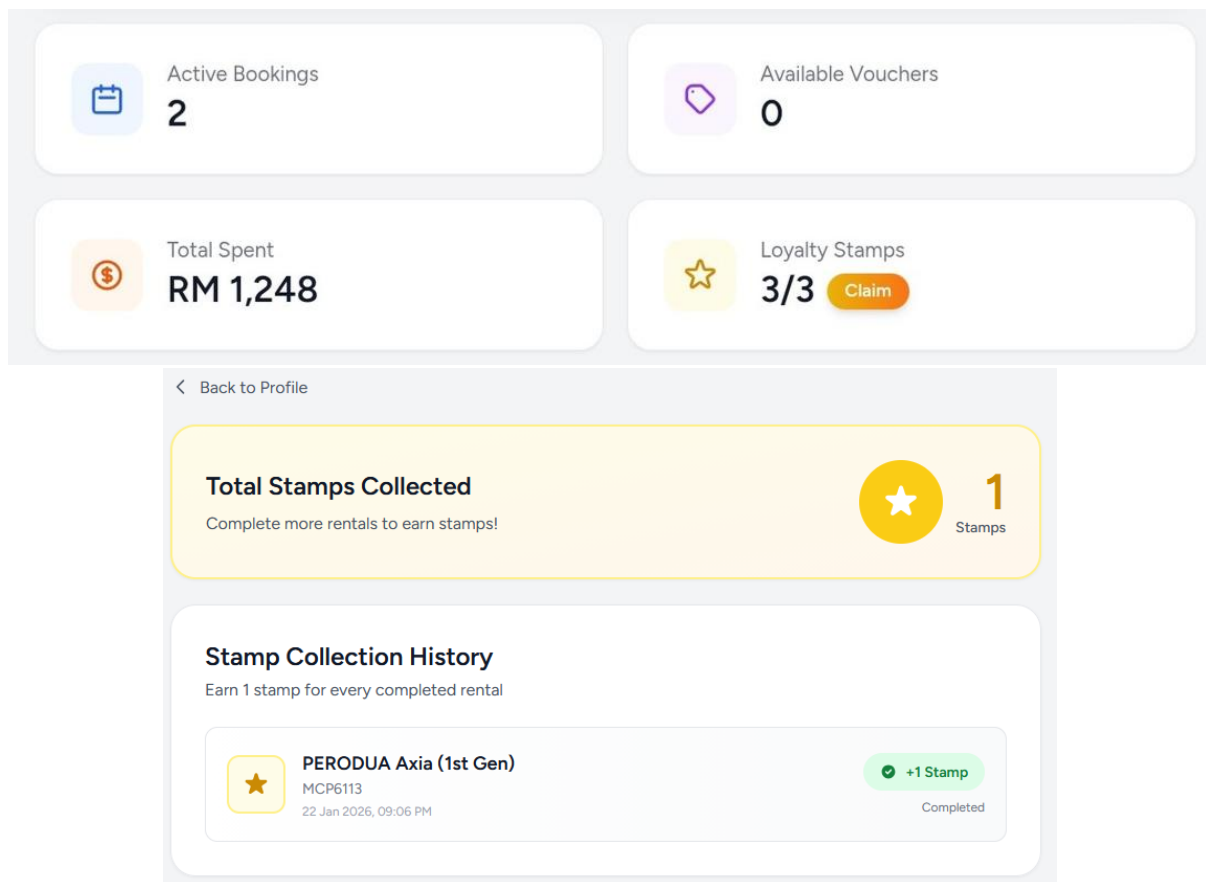
SQL Statements:

```
UPDATE rentals  
SET status = 'Completed', updated_at = NOW()  
WHERE id = ?;
```

Explanation:

This statement updates the rental status to 'Completed' after refund is issued.

1.7.12 Loyalty Stamps and Vouchers



After completing the rental, customers can view the stamps earned and the stamp collection history in their profile. Once the customer already collects 3 stamps, then they can use 3 stamps to redeem one loyalty voucher and use it in the future booking.

SQL Statements:

```
SELECT id, user_id, loyalty_stamps, claimed_loyalty_vouchers
FROM customers
WHERE user_id = ?;
```

Explanation:

This statement retrieves the customer's total loyalty stamps and claimed voucher count for the summary card.

SQL Statements:

```
SELECT r.id, r.customer_id, r.car_id, r.status, r.created_at
FROM rentals r
WHERE r.customer_id = ? AND r.status = 'Completed'
ORDER BY r.created_at DESC;
```

```
SELECT id, car_plate, model
FROM cars
WHERE id = ?;
```

Explanation:

This statement retrieves all completed rentals to display stamp collection history. Each completed rental with a duration more than 12 hours will earn 1 stamp.

SQL Statements:

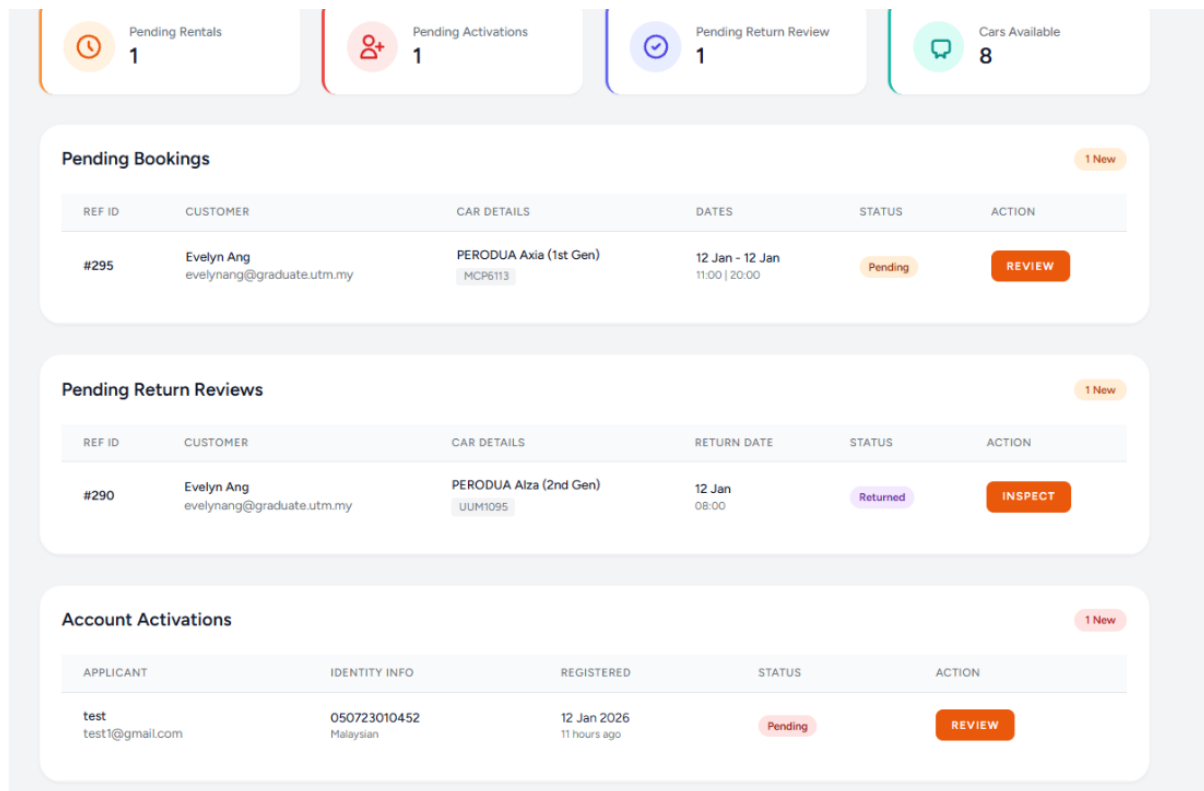
```
INSERT INTO vouchers (customer_id, type, issued_by, expiry_date,
usage, created_at, updated_at)
VALUES (?, 'LOYALTY20', NULL, DATE_ADD(NOW(), INTERVAL 365 DAY),
'Unused', NOW(), NOW());
```

```
UPDATE customers
SET claimed_loyalty_vouchers = claimed_loyalty_vouchers + 1,
updated_at = NOW()
WHERE id = ?;
```

Explanation:

This statement will create a LOYALTY20 voucher (20% OFF) valid for 1 year when customer claims with 3 stamps. After that, increase the number of claimed vouchers in customer profile.

1.7.13 Staff Dashboard



After staff login, they will redirect to the staff dashboard homepage. This is where the staff can quickly navigate and start doing their tasks with an easy click. At the top section, staff can see the numbers of pending booking reviews, pending activations reviews, pending return reviews, and the current car available as well. For the pending booking section, staff can see all the booking requests of the customer which show the customer's name, car details and rental period. Staff can just click review to view the request to approve it or disapprove it. For pending return reviews, staff can also see customer name, car details and return date. They can also click Inspect to redirect to the three tasks they need to do, which are check feedback with issue, manage refund and update mileage and fuel bar. For account activation, it will also show the applicant's name, IC number and submitted timestamp, while staff can click review to decide to approve or reject activation.

SQL Statement:

```
SELECT COUNT(*)
FROM rentals
WHERE status = 'Pending';

SELECT COUNT(*)
FROM activations
WHERE status = 'Pending';

SELECT COUNT(*)
FROM rentals
WHERE status = 'Returned';

SELECT COUNT(*)
FROM cars
WHERE status = 'Available';
```

Explanation:

These queries get the count of pending activations, pending bookings, returned rentals, available cars to be displayed on the statistics card section on top.

SQL Statement:

```
SELECT
r.id AS ref_id, u.name AS customer_name,
u.email AS customer_email, c.model AS car_model,
c.car_plate AS car_plate,
r.pickup_date, r.pickup_time,
r.return_date, r.return_time, r.status
FROM rentals r
JOIN customers cu ON r.customer_id = cu.id
JOIN users u ON cu.user_id = u.id
JOIN cars c ON r.car_id = c.id
WHERE r.status = 'Pending'
ORDER BY r.created_at ASC;
```

Explanation:

List of bookings with 'Pending' status, with the oldest record comes first.

SQL Statement:

```
SELECT
    r.id AS ref_id, u.name AS customer_name,
    u.email AS customer_email,
    c.model AS car_model,
    c.car_plate AS car_plate,
    r.return_date, r.return_time, r.status
FROM rentals r
JOIN customers cu ON r.customer_id = cu.id
JOIN users u ON cu.user_id = u.id
JOIN cars c ON r.car_id = c.id
WHERE r.status = 'Returned'
ORDER BY r.return_date ASC;
```

Explanation:

List of rentals that were returned but not yet inspected/completed, older record comes first.

SQL Statement:

```
SELECT
    u.name AS applicant_name,
    u.email AS applicant_email,
    a.ic_passport AS identity_info,
    a.nationality,
    a.created_at AS registered_at,
    a.status
FROM activations a
JOIN customers cu ON a.customer_id = cu.id
JOIN users u ON cu.user_id = u.id
WHERE a.status = 'Pending'
ORDER BY a.created_at ASC;
```

Explanation:

List of new customers requesting account activation, older record comes first.

1.7.14 Customer List

Customer Records						
Filter customers by account status						
Search by name, IC, or ID...				Status	All	Search
NAME	EMAIL	COLLEGE	FACULTY	CONTACT	STATUS	ACTION
Evelyn Ang	evelynang@graduate.utm.my	KTDI	FC	0172617271	Active	
Parthiv Gunalan	lightchaser1125@gmail.com	KDOJ	FABU	01156231172	Active	
Mirza	mirzaas-siddiq@graduate.utm.my	KTDI	FC	01156231121	Active	
Aliff Helmi	aliff-05@graduate.utm.my	KTDI	FKM	01156231123	Active	
Chun Yao Ting	chun Yao.ting@graduate.utm.my	KTDI	FC	01156231125	Active	
Aleeya Najihah binti Bakar	aleeya.najihah.bakar@graduate.utm.my	K9/K10	FEST	0144410150	Active	
Sarah Natasha binti Azman	sarah.natasha.azman@graduate.utm.my	KDOJ	FEST	0119515596	Active	

In the customer tab, we can view the customer's name, email, college, faculty, contact and their status sorted by the activation time. We can see that the status is in dropdown, meaning staff can directly change the status, for example from Active to Blacklisted, and vice versa. There is also a filter section on top, where you can search by customer name, and select by status.

SQL Statement:

```
SELECT
    c.id AS customer_id,
    u.name,
    u.email,
    c.college,
    c.faculty,
    c.contact,
    c.status
FROM customers c
JOIN users u ON c.user_id = u.id
ORDER BY c.created_at DESC;
```

Explanation:

The statement retrieves all customers as default, as there is no filter selected by staff. Earliest created account record comes first in the list.

Customer Records Filter customers by account status		Q Evel	Status	All	Search	Clear
NAME	EMAIL	COLLEGE	FACULTY	CONTACT	STATUS	ACTION
Evelyn Ang	evelynang@graduate.utm.my	KTDI	FC	0172617271	Active	

SQL Statement:

```
SELECT
    c.id AS customer_id, u.name, u.email,
    c.college, c.faculty, c.contact, c.status
FROM customers c
JOIN users u ON c.user_id = u.id
WHERE u.name LIKE CONCAT('%', ?, '%')
ORDER BY c.created_at DESC;
```

Explanation:

Staff can search by customer name substring, so the statement uses LIKE to detect pattern of substring to retrieve the customer fulfilling the substring pattern. Earliest created account record comes first in the list.

Customer Records Filter customers by account status		Q Search by name, IC, or ID...	Status	Blacklisted	Search	Clear
NAME	EMAIL	COLLEGE	FACULTY	CONTACT	STATUS	ACTION
Sarah Natasha binti Azman	sarah.natasha.azman@graduate.utm.my	KDOJ	FEST	0119515596	Blacklisted	

SQL Statement:

```
SELECT
    c.id AS customer_id, u.name, u.email,
    c.college, c.faculty, c.contact, c.status
FROM customers c
JOIN users u ON c.user_id = u.id
WHERE c.status = ?
ORDER BY c.created_at DESC;
```

Explanation:

Staff can filter by customer under certain status, so the statement retrieves all the customer under the selected status, such as Blacklisted. Earliest created account comes first in the list.

NAME	EMAIL	COLLEGE	FACULTY	CONTACT	STATUS	ACTION
Sarah Natasha binti Azman	sarah.natasha.azman@graduate.utm.my	KDOJ	FEST	0119515596	Blacklisted	
Nasuha Aina binti Firdaus	nasuha.aina.firdaus@graduate.utm.my	KTHO	SPACE	0166757137	New	
					Pending	
					Active	
					Blacklisted	
					Inactive	

SQL Statement:

```
UPDATE customers
SET status = ?, updated_at = NOW()
WHERE id = ?;
```

Explanation:

The statement updates the selected customer status into selected status in dropdown UI.

Applicant Details

PERSONAL INFO

Full Name
Evelyn Ang

Email
evelynang@graduate.utm.my

IC / Passport
050723012222

Nationality
Malaysian

Contact
0172617271

Emergency
018275172

ACADEMIC INFO

UTM ID
A23CS0123

College
Kolej Tun Dr. Ismail (KTDI)

Faculty
Faculty of Computing

LICENSE INFO

License No
192761

Expiry
2026-12-31 00:00:00

Application Approved

Submitted Documents *Review carefully before approving*

Identity Document PDF OPEN

By clicking the view button, we can see the full information of customer details, and also their submitted documents during activation phase, such as IC, UTM card, and license.

SQL Statement:

```
SELECT
    a.id AS activation_id,
    u.name AS full_name,
    u.email,
    a.ic_passport,
    a.nationality,
    a.contact,
    a.emergency_contact,
    a.utm_id,
    a.college,
    a.faculty,
    a.license_no,
    a.license_expiry,
    a.identity_file,
    a.utm_file,
    a.license_file,
    a.status
FROM activations a
JOIN customers c ON a.customer_id = c.id
JOIN users u ON c.user_id = u.id
WHERE a.id = ?;
```

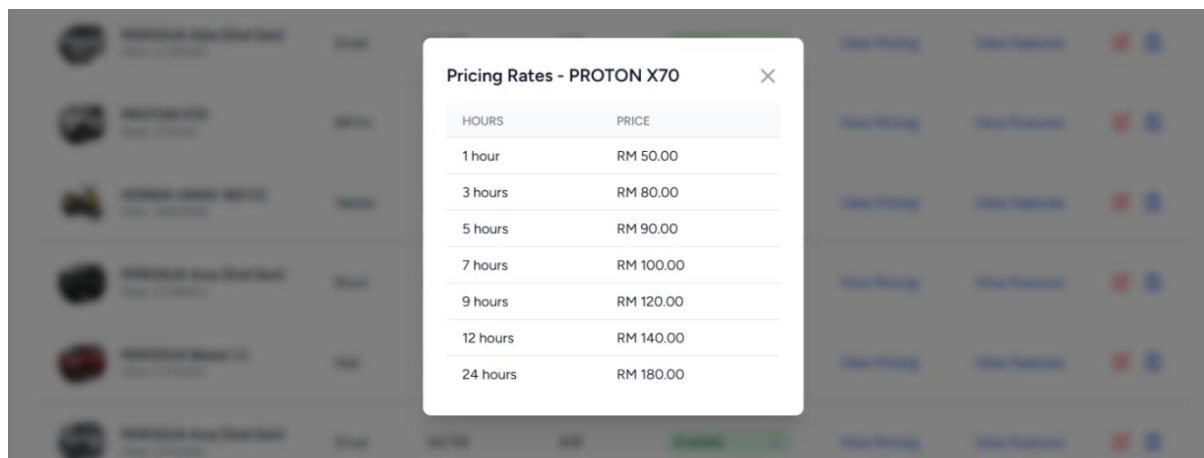
Explanation:

The statement uses JOIN to link the activation request to the customer record and link the customer record to the main user account to retrieve all the details as shown in interface.

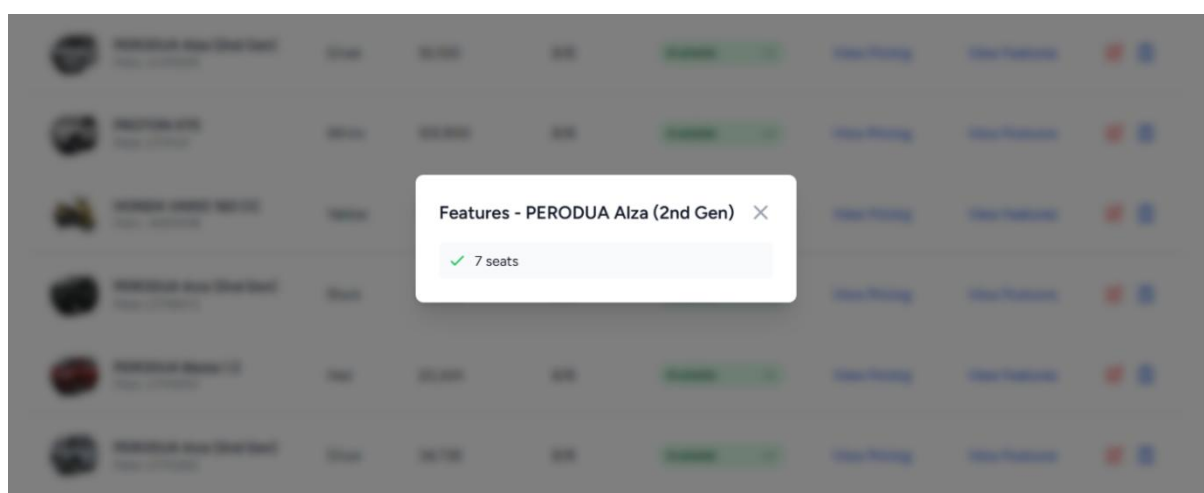
1.7.15 Car List

VEHICLE	COLOR	MILEAGE (KM)	FUEL BAR	STATUS	PRICING RATE	FEATURES	ACTIONS
 PERODUA Alza (2nd Gen) Plate: UUM1095	Silver	10,100	8/8	Available ▼	View Pricing	View Features	 
 PROTON X70 Plate: UTM137	White	102,850	8/8	Available ▼	View Pricing	View Features	 
 HONDA VARIO 160 CC Plate: JWD9496	Yellow	1,111,809	8/8	Available ▼	View Pricing	View Features	 
 PERODUA Axia (2nd Gen) Plate: UTM9473	Black	20,469	8/8	Available ▼	View Pricing	View Features	 
 PERODUA Bezza 1.3 Plate: UTM3057	Red	20,504	8/8	Available ▼	View Pricing	View Features	 

In the car tab, staff can view all the vehicle details, such as its model, plate number, mileage, fuel bar, pricing rate and features. For the status of the car, it is also in dropdown, meaning that the staff can change it directly to Maintenance and back to Available easily. Besides, staff can view pricing rates and view features as shown below.



HOURS	PRICE
1 hour	RM 50.00
3 hours	RM 80.00
5 hours	RM 90.00
7 hours	RM 100.00
9 hours	RM 120.00
12 hours	RM 140.00
24 hours	RM 180.00



✓ 7 seats

SQL Statement:

```
SELECT
    id, car_pic, model, car_plate, color, mileage,
    fuel_bar, status, pricing_rate, features
FROM cars
ORDER BY created_at DESC;
```

Explanation:

The statement retrieves all cars as default, as there is no filter selected by staff. Earliest created car record comes first in the list.

SQL Statement:

```
SELECT
    id, car_pic, model, car_plate, color, mileage,
    fuel_bar, status, pricing_rate, features
FROM cars
WHERE car_plate LIKE CONCAT('%', ?, '%')
ORDER BY created_at DESC;
```

Explanation:

The statement retrieves cars with plate number that follow the substring pattern of user input. Newer created car record comes first.

SQL Statement:

```
SELECT
    id, car_pic, model, car_plate, color, mileage,
    fuel_bar, status, pricing_rate, features
FROM cars
WHERE status = ?
ORDER BY created_at DESC;
```

Explanation:

The statement retrieves cars with status selected by customer, newer created car comes first.

	PERODUA Alza (2nd Gen) Plate: UUM1095	Silver	10,100	8/8	Available	View Pricing	View Features	Edit	Copy
	PROTON X70 Plate: UTM137	White	102,850	8/8	Maintenance	View Pricing	View Features	Edit	Copy

SQL Statement:

```
UPDATE cars
    SET status = ?, updated_at = NOW()
WHERE id = ?;
```

Explanation:

The statement updates the car status into selected status by staff in dropdown UI.

Car Plate *

UUM1095

Model *

PERODUA Alza (2nd Gen)

Color *

Silver

Status *

Available

Mileage (km) *

10100

Fuel Bar (0-8) *

8

Car Picture

Current Image:



Pricing Rate *

1 hour	RM 50
3 hours	RM 80
5 hours	RM 100
7 hours	RM 110
9 hours	RM 130
12 hours	RM 150
24 hours	RM 180

Features

7 seats

+ ADD NEW FEATURE

Cancel

Update Car

The staff can also update the car details after clicking edit button as shown in interface above.

SQL Statement:

```
SELECT
    id, car_pic, model, car_plate, color, mileage,
    fuel_bar, status, pricing_rate, features
FROM cars
WHERE id = ?;
```

```
UPDATE cars
SET
    model = ?, color = ?, car_pic = ?, mileage = ?,
    fuel_bar = ?, pricing_rate = ?, features = ?,
    updated_at = NOW()
WHERE id = ?;
```

Explanation:

The first statement fetches the data by selecting certain car ID for user to edit previously entered data. Then, after clicking submit, the second statement is executed to update the car details in database.

Car Information

Model: PERODUA Alza (2nd Gen) Plate: UUM1095
Color: Silver Mileage: 10,100 km
Fuel Level: 8/8 Status: Available



Rental History

RENTAL ID	CUSTOMER	RENTAL PERIOD	PHOTOS	STATUS	FEEDBACK
#289	Chun Yao Ting	Jan 08, 2026 → Jan 09, 2026		Completed	The booking is nice.
#290	Chun Yao Ting	Jan 08, 2026 → Jan 09, 2026		Completed	Nice booking
#291	Aliff Helmi	Jan 08, 2026 → Jan 08, 2026		Late Returned	No problems
#82	Harith Danish bin Saiful	Dec 20, 2025 → Dec 23, 2025		Completed	No feedback

Meanwhile, by clicking the View History icon at the Action column, staff will be redirected to this page. In this page, staff can view all the car information, as well as all rental records of that car. The history record contains the customer's name, rental period, status, and feedback. If the feedback is marked with issue, there will be an alert icon at the side of the rental record. Furthermore, staff also can view the pickup car picture and return car picture of the car submitted by the customer, by clicking the photo icon as shown below.

Car Information

Model: PERODUA Bezza 1.3 Plate: UTM3655
Color: Black Mileage: 64,447 km
Fuel Level: 8/8

Rental History

RENTAL ID	CUSTOMER	STATUS	FEEDBACK
 #295	Chun Yao Ting	Completed	Aircond rosak
#13	Harith Danish bin Saiful	Completed	No feedback
#50	Adrian	Completed	No feedback
#148	Lucy	Completed	No feedback
#29	Aimee	Completed	No feedback



Car Photos - PERODUA Bezza 1.3 (UTM3655)

Pickup Photos



Return Photos



SQL Statement:

```
SELECT
    id, car_pic, model, car_plate, color, mileage,
    fuel_bar, status, pricing_rate, features
FROM cars
WHERE id = ?;
```

Explanation:

The statement fetches the data by selecting certain car ID for user after clicking History icon.

SQL Statement:

```
SELECT
    r.id AS rental_id,
    u.name AS customer_name,
    r.pickup_date,
    r.return_date,
    r.pickup_pic,
    r.return_pic,
    r.status,
    r.feedback
FROM rentals r
JOIN customers c ON r.customer_id = c.id
JOIN users u ON c.user_id = u.id
WHERE r.car_id = ?
ORDER BY r.created_at DESC;
```

Explanation:

The statement uses JOIN to query and retrieve all the rentals historical record of the selected car, latest record comes first.

Required Information

Car Plate *

e.g., WXY1234

Model *

e.g., Perodua Bezza 1.3

Color *

e.g., Red

Status *

Available

Mileage (km) *

0

Fuel Bar (0-8) *

8

Car Picture

Choose File

No file chosen

Pricing Rate *

1 hour	RM 0.00
3 hours	RM 0.00
5 hours	RM 0.00
7 hours	RM 0.00
9 hours	RM 0.00
12 hours	RM 0.00
24 hours	RM 0.00

Features

+ ADD NEW FEATURE

Cancel

Add Car

To add a new car, staff need to fill in the car plate, model, color, status, initial mileage, fuel bar, car picture, pricing rate for each hour, and any available features.

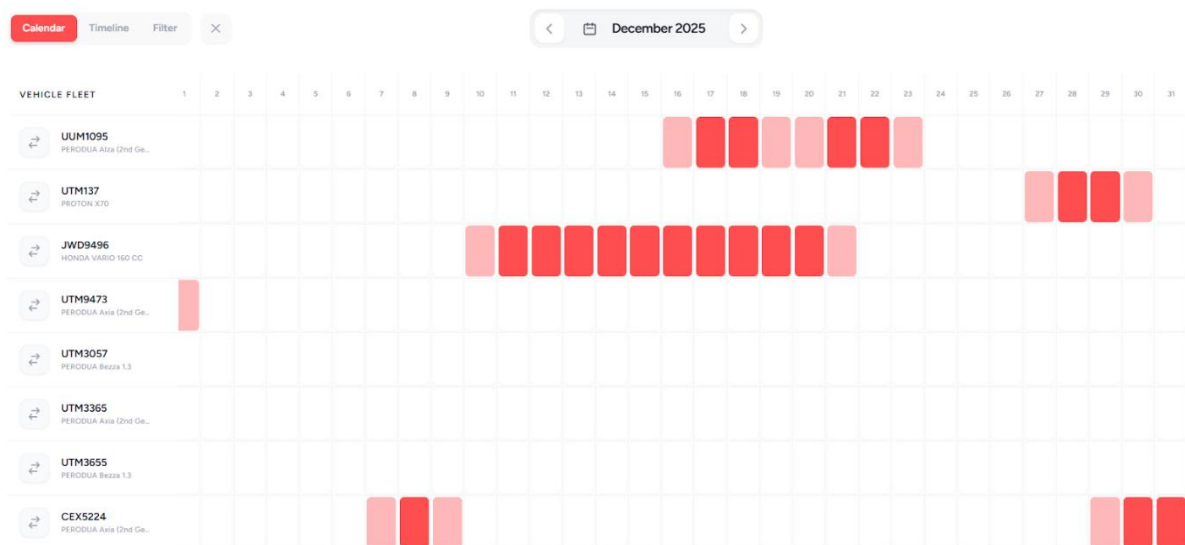
SQL Statement:

```
INSERT INTO cars (
    car_plate, model, color, car_pic,
    mileage, fuel_bar, pricing_rate, features,
    status, created_at, updated_at)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, 'Available', NOW(), NOW());
```

Explanation:

The statement executes insertion to create a new car record in database.

1.7.16 View Car Availability



There are three views in the view car availability tab, which are calendar, timeline, and filter. First, the staff can view the availability of every car on each day, even for previous records by clicking Calendar. Staff can filter by month by clicking the calendar to select months, and go to previous or next months by clicking previous or next button. The color of the box has their distinct meaning, red indicates fully booked, light red indicates partially booked, and white means available. By clicking the box, staff will be redirected to the timeline view of that day.

SQL Statement:

```
SELECT
    id, model, car_plate, car_pic, status
FROM cars
ORDER BY created_at DESC;
```

Explanation:

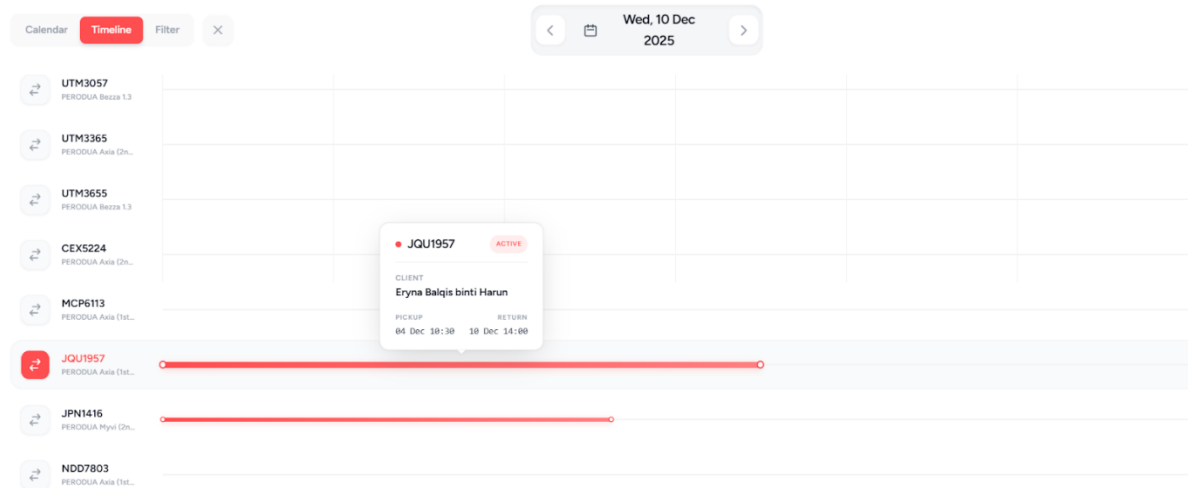
The statement fetches all the car record to be displayed, newly created car come first.

SQL Statement:

```
SELECT
    r.id AS rental_id,
    r.car_id,
    r.pickup_date,
    r.pickup_time,
    r.return_date,
    r.return_time,
    r.status,
    u.name AS customer_name
FROM rentals r
JOIN customers c ON r.customer_id = c.id
JOIN users u ON c.user_id = u.id
WHERE
    r.pickup_date <= ?
    AND r.return_date >= ?
    AND r.status NOT IN ('Cancelled', 'Rejected')
ORDER BY r.pickup_date ASC;
```

Explanation:

The statement queries and retrieve rentals that overlap with the selected timeframe. For example, '2025-12-01' to '2025-12-31'. The statement includes pickup_time and return_time to allow distinguishing between Full Day (Red) and Partial Day (Light Red) occupancy. The statement also excludes Cancelled/Rejected bookings which do not block availability.



In the timeline view, staff can see the rented period of each car on the selected date. Staff can change the date by selecting from calendar or go to previous or next day by clicking previous or next. By hovering the timeline, staff also can view the booking details, which are the customer's name, and the pickup date and time and return date and time.

SQL Statement:

```
SELECT
    r.id AS rental_id, r.car_id, r.pickup_date, r.pickup_time,
    r.return_date, r.return_time, r.status,
    u.name AS customer_name
FROM rentals r
JOIN customers c ON r.customer_id = c.id
JOIN users u ON c.user_id = u.id
WHERE
    r.pickup_date <= ?
    AND r.return_date >= ?
    AND r.status NOT IN ('Cancelled', 'Rejected')
ORDER BY r.pickup_date ASC;
```

Explanation:

The statement retrieves rentals that overlap with the selected timeline view range. The overlaps condition `pickup_date <= end AND return_date >= start` correctly identifies all rentals that touch the view range. Pickup and return time are included to support the timeline visualization. The statement also excluded Cancelled and Rejected rentals.

In the filter view, staff have to enter the pickup date and pickup time as well as return date and time to filter the available car during desired rental period. By clicking the search availability button, the system will show all the available cars. This is very useful to check car availability when there is a walk-in customer, so staff do not need to use any customer account to login and check the car availability from the customer side.

On top of the section, it will show the number of total available cars. Staff can see the car pictures, car model, car plate, color, fuel, mileage, and features in a card UI component. Staff can also press the next and previous button to view other cars available for walk-in customers to choose their preferred car.

SQL Statements:

```
SELECT id, car_plate, model, color, car_pic, status, mileage,
fuel_bar, pricing_rate, features
FROM cars
WHERE status IN ('Available', 'Rented')
    AND id NOT IN (
        SELECT car_id FROM rentals
        WHERE (
            (
                status IN ('Pending', 'Approved', 'Picked Up')
                AND CONCAT(pickup_date, ' ', pickup_time) <= ?
                AND DATE_ADD(CONCAT(return_date, ' ', return_time)
                    , INTERVAL 1 HOUR) > ?
            )
            OR (
                status IN ('Returned', 'Late Returned')
                AND DATE_ADD(updated_at, INTERVAL 1 HOUR) > ?
                AND updated_at >= DATE_SUB(?, INTERVAL 1 DAY)
            )
        )
    )
ORDER BY model ASC;
```

Explanation:

This statement will fetch the cars with 'Available' or 'Rented' status that don't have conflicting bookings. The system will apply a 1-hour cooldown after return time to allow for car cleaning or inspection.

1.7.17 Rental List

Generate ReportExport as Excel						
Rental Records Search by name, rental ID, or filter by car plate Search by name or rental ID... All Years All Months Car Plate All Plates Search						
NAME	CAR PLATE	RENTAL DURATION	STATUS	RENTAL DETAILS	PHOTOS	ACTION
Aliff Helmi	UUM1095	13 Jan 2026 → 13 Jan 2026	Picked Up	View Details		Inspect
Parthiv Gunalan	NDD7803	10 Jan 2026 → 11 Jan 2026	Returned	View Details		Inspect Refund
Chun Yao Ting	UTM3655	08 Jan 2026 → 09 Jan 2026	Completed	View Details		
Parthiv Gunalan	NDD7803	08 Jan 2026 → 08 Jan 2026	Completed	View Details		
Mirza	MCP6113	22 Jan 2026 → 29 Jan 2026	Cancelled	View Details		Refund
Mirza	MCP6113	09 Jan 2026 → 16 Jan 2026	Rejected	View Details		Refund
Aliff Helmi	UUM1095	08 Jan 2026 → 08 Jan 2026	Late Returned	View Details		—
Chun Yao Ting	UUM1095	08 Jan 2026 → 09 Jan 2026	Completed	View Details		

In the rental tab, staff can view all the rental records. For each rental record, name, car plate, rental duration, and status is shown clearly. At the top section, staff can filter by searching customer names, by years, by months, and by car plates as well.

SQL Statement:

```
SELECT
    r.id AS rental_id, u.name AS customer_name, c.car_plate,
    r.pickup_date, r.return_date, r.status,
    r.pickup_pic, r.return_pic
FROM rentals r
JOIN customers cu ON r.customer_id = cu.id
JOIN users u ON cu.user_id = u.id
JOIN cars c ON r.car_id = c.id
ORDER BY r.created_at DESC;
```

Explanation:

The statement retrieves the list of all rental records by default, ordered by latest first.

Rental Records Search by name, rental ID, or filter by car plate		Chun	All Years ▾	All Months ▾	Car Plate	All Plates ▾	Search	Clear
NAME	CAR PLATE	RENTAL DURATION	STATUS	RENTAL DETAILS	PHOTOS	ACTION		
Chun Yao Ting	UTM3655	08 Jan 2026 → 09 Jan 2026	Completed	View Details				
Chun Yao Ting	UUM1095	08 Jan 2026 → 09 Jan 2026	Completed	View Details				
Chun Yao Ting	UUM1095	08 Jan 2026 → 09 Jan 2026	Completed	View Details				

SQL Statement:

```

SELECT
    r.id AS rental_id, u.name AS customer_name,
    c.car_plate, r.pickup_date, r.return_date,
    r.status, r.pickup_pic, r.return_pic
FROM rentals r
JOIN customers cu ON r.customer_id = cu.id
JOIN users u ON cu.user_id = u.id
JOIN cars c ON r.car_id = c.id
WHERE
    u.name LIKE CONCAT('%', ?, '%')
ORDER BY r.created_at DESC;

```

Explanation:

Staff can search by customer name substring, so the statement uses LIKE to detect pattern of substring to retrieve the rental record with involved customer fulfilling the substring pattern. Newly created rental record comes first in the list.

Rental Records Search by name, rental ID, or filter by car plate Search by name or rental ID... 2023 ▼ March ▼ Car Plate All Plates ▼ Search Clear						
NAME	CAR PLATE	RENTAL DURATION	STATUS	RENTAL DETAILS	PHOTOS	ACTION
Liyana Sofiya binti...	UUM1095	25 Mar 2023 → 01 Apr 2023	Completed	View Details		
Azlan Qayyum bin Azm...	JPN1416	25 Mar 2023 → 03 Apr 2023	Completed	View Details		
Iris Natalie binti F...	JQU1957	22 Mar 2023 → 24 Mar 2023	Completed	View Details		
Azlan Qayyum bin Azm...	CEX5224	20 Mar 2023 → 29 Mar 2023	Completed	View Details		
Adib Ilyas bin Razal...	UTM9473	19 Mar 2023 → 27 Mar 2023	Completed	View Details		

SQL Statement:

```

SELECT
    r.id AS rental_id,
    u.name AS customer_name,
    c.car_plate,
    r.pickup_date,
    r.return_date,
    r.status,
    r.pickup_pic,
    r.return_pic
FROM rentals r
JOIN customers cu ON r.customer_id = cu.id
JOIN users u ON cu.user_id = u.id
JOIN cars c ON r.car_id = c.id
WHERE
    YEAR(r.pickup_date) = ?
    AND MONTH(r.pickup_date) = ?
ORDER BY r.created_at DESC;

```

Explanation:

The statement queries and retrieves all the rental record that fulfil the year and month filter selected by staff. The newer record comes first.

Rental Records Search by name, rental ID, or filter by car plate Search by name or rental ID... All Years All Months Car Plate UTM3655 Search Clear						
NAME	CAR PLATE	RENTAL DURATION	STATUS	RENTAL DETAILS	PHOTOS	ACTION
Chun Yao Ting	UTM3655	08 Jan 2026 → 09 Jan 2026	Completed	View Details		
Haziq Zafran bin Isk...	UTM3655	20 Aug 2025 → 23 Aug 2025	Completed	View Details		
Adriana Syahirah bin...	UTM3655	31 Jul 2025 → 09 Aug 2025	Completed	View Details		
Luqmanul Hakim bin A...	UTM3655	12 Apr 2025 → 15 Apr 2025	Completed	View Details		
Aiman Syazwan bin Ro...	UTM3655	10 Feb 2025 → 11 Feb 2025	Completed	View Details		

SQL Statement:

```

SELECT
    r.id AS rental_id,
    u.name AS customer_name,
    c.car_plate,
    r.pickup_date,
    r.return_date,
    r.status,
    r.pickup_pic,
    r.return_pic
FROM rentals r
JOIN customers cu ON r.customer_id = cu.id
JOIN users u ON cu.user_id = u.id
JOIN cars c ON r.car_id = c.id
WHERE
    c.car_plate = ?
ORDER BY r.created_at DESC;

```

Explanation:

The statement queries and retrieves all the rental record that is involved under the car plate number selected by staff. The newer record comes first.

CUSTOMER
Chun Yao Ting
chun.yao.ting@graduate.utm.my
MANAGED BY
N/A

CAR
PERODUA Bezza 1.3
UTM3655
CURRENT STATUS
Completed

RENTAL SCHEDULE

● Pickup

DATE & TIME
08 Jan 2026 @ 04:00 PM
LOCATION
UTM Library

● Return

DATE & TIME
09 Jan 2026 @ 10:00 AM
LOCATION
Skudai Town

PAYMENT INFO

TOTAL AMOUNT
RM 194.00

Payment Status
Paid

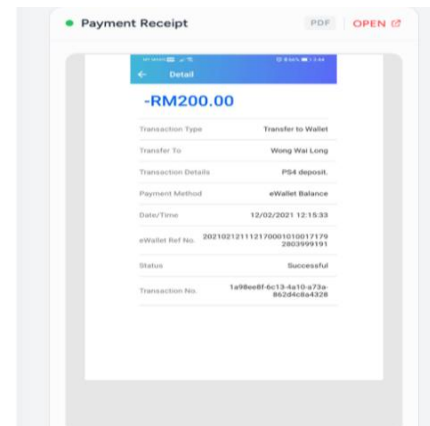
FORMS & DOCUMENTS



Agreement Form



Inspection Form



Staff can view the details of bookings after clicking view icon as well.

SQL Statement:

SELECT

```
r.id AS rental_id, u.name AS customer_name,  
u.email AS customer_email, c.model AS car_model,  
c.car_plate, r.status AS rental_status,  
s.name AS managed_by, r.pickup_date, r.pickup_time,  
r.pickup_location, r.return_date, r.return_time,  
r.return_location, r.agreement_form, r.inspection_form
```

FROM rentals r

JOIN customers cu ON r.customer_id = cu.id

JOIN users u ON cu.user_id = u.id

JOIN cars c ON r.car_id = c.id

LEFT JOIN staff s ON r.managed_by = s.id

LEFT JOIN users us ON s.user_id = us.id

WHERE r.id = ?;

SELECT

```
amount AS total_amount, receipt_image,
```

```
created_at AS payment_date, 'Paid' AS payment_status
```

FROM payments

WHERE rental_id = ?;

Explanation:

The first statement retrieves the rental details and the second retrieves payment details.

1.7.18 Payment List

Generate Report

Export as Excel

Transaction Records

Search by payment ID, rental ID, or customer name

Q

Search by payment ID, rental ID, or name...

All Years

All Months

Search

PAYMENT ID	RENTAL ID	CUSTOMER NAME	RENTAL DURATION	PAYMENT AMOUNT	RECEIPT
#297	#297	Aliff Helmi	13 Jan 2026 → 13 Jan 2026	RM130.00	
#296	#296	Parthiv Gunalan	10 Jan 2026 → 11 Jan 2026	RM170.00	
#295	#295	Chun Yao Ting	08 Jan 2026 → 09 Jan 2026	RM194.00	
#294	#294	Parthiv Gunalan	08 Jan 2026 → 08 Jan 2026	RM85.00	
#293	#293	Mirza	22 Jan 2026 → 29 Jan 2026	RM915.00	
#292	#292	Mirza	09 Jan 2026 → 16 Jan 2026	RM1,105.00	
#291	#291	Aliff Helmi	08 Jan 2026 → 08 Jan 2026	RM100.00	

In the payment tab, staff can view all the transaction records. For each transaction record, customer name, rental duration, and payment amount is shown clearly.

SQL Statement:

```
SELECT
    p.id AS payment_id,
    p.rental_id,
    u.name AS customer_name,
    r.pickup_date,
    r.return_date,
    p.amount AS payment_amount,
    p.receipt_image
FROM payments p
JOIN rentals r ON p.rental_id = r.id
JOIN customers c ON p.customer_id = c.id
JOIN users u ON c.user_id = u.id
ORDER BY p.created_at DESC;
```

Explanation:

The statement retrieves all transaction records by default, newly created record comes first.

Transaction Records					
Search by payment ID, rental ID, or customer name			Q Ting	All Years	All Months
				Search	Clear
PAYMENT ID	RENTAL ID	CUSTOMER NAME	RENTAL DURATION	PAYMENT AMOUNT	RECEIPT
#295	#295	Chun Yao Ting	08 Jan 2026 → 09 Jan 2026	RM194.00	
#290	#290	Chun Yao Ting	08 Jan 2026 → 09 Jan 2026	RM230.00	
#289	#289	Chun Yao Ting	08 Jan 2026 → 09 Jan 2026	RM230.00	

SQL Statement:

```
SELECT
    p.id AS payment_id,
    p.rental_id,
    u.name AS customer_name,
    r.pickup_date,
    r.return_date,
    p.amount AS payment_amount,
    p.receipt_image
FROM payments p
JOIN rentals r ON p.rental_id = r.id
JOIN customers c ON p.customer_id = c.id
JOIN users u ON c.user_id = u.id
WHERE
    u.name LIKE CONCAT('%', ?, '%')
ORDER BY p.created_at DESC;
```

Explanation:

Staff can search by customer name substring, so the statement uses LIKE to detect pattern of substring to retrieve the transaction record with involved customer fulfilling the substring pattern. Newly created transaction record comes first in the list.

Transaction Records

Search by payment ID, rental ID, or customer name



Search by payment ID, rental ID, or name...

2023



March



Search

Clear

PAYMENT ID	RENTAL ID	CUSTOMER NAME	RENTAL DURATION	PAYMENT AMOUNT	RECEIPT
#77	#77	Liyana Sofiya binti Bakar	25 Mar 2023 → 01 Apr 2023	RM1,033.00	
#159	#159	Azlan Qayyum bin Azmi	25 Mar 2023 → 03 Apr 2023	RM1,222.00	
#164	#164	Iris Natalie binti Firdaus	22 Mar 2023 → 24 Mar 2023	RM363.00	
#162	#162	Azlan Qayyum bin Azmi	20 Mar 2023 → 29 Mar 2023	RM1,348.00	
#99	#99	Adib Ilyas bin Razali	19 Mar 2023 → 27 Mar 2023	RM720.00	

SQL Statement:

```
SELECT
    p.id AS payment_id,
    p.rental_id,
    u.name AS customer_name,
    r.pickup_date,
    r.return_date,
    p.amount AS payment_amount,
    p.receipt_image
FROM payments p
JOIN rentals r ON p.rental_id = r.id
JOIN customers c ON p.customer_id = c.id
JOIN users u ON c.user_id = u.id
WHERE
    YEAR(p.created_at) = ?
    AND MONTH(p.created_at) = ?
ORDER BY p.created_at DESC;
```

Explanation:

The statement queries and retrieves all the transaction record that fulfil the year and month filter selected by staff. The newer record comes first.

1.7.19 Delivery Location List

+ Add Delivery Location

All Delivery Locations

Total Locations: 21

#	LOCATION	FEE (RM)	CREATED AT	ACTIONS
1	 AEON Tebrau City	RM 20.00	08 Jan 2026	 
2	 Batu Pahat	RM 60.00	08 Jan 2026	 
3	 Ipoh	RM 100.00	08 Jan 2026	 
4	 Johor Bahru City Centre	RM 15.00	08 Jan 2026	 
5	 Kluang	RM 45.00	08 Jan 2026	 
6	 Kuala Lumpur (TBS)	RM 95.00	08 Jan 2026	 
7	 Larkin Sentral	RM 22.00	08 Jan 2026	 

In the delivery tab, staff can view all the delivery locations provided by Hasta for pickup and return location. This includes the location name and delivery fee. Staff can edit the location and delete the location at the actions tab.

SQL Statement:

```
SELECT COUNT(*) AS total_locations FROM deliveries;
SELECT
    id, location, fee, created_at
FROM deliveries
ORDER BY location ASC;
```

Explanation:

The first statement retrieve the total count of deliveries and the second statement retrieve all delivery location from database.

SQL Statement:

```
UPDATE deliveries
SET
    location = ?, fee = ?, updated_at = NOW()
WHERE id = ?;
```

Explanation:

The statements save the changes and update the delivery location involved in database.

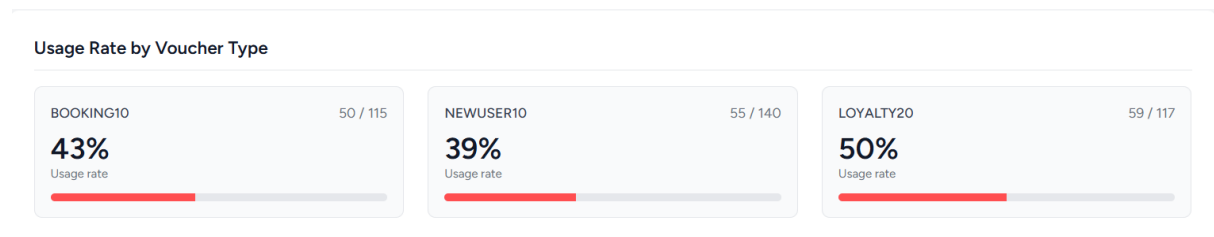
SQL Statement:

```
INSERT INTO deliveries (
    location, fee, created_at, updated_at
)
VALUES (?, ?, NOW(), NOW());
```

Explanation:

The statement creates a new record of delivery locations in database.

1.7.20 Voucher List



At the top section, staff can view the summary of the usage rate by voucher type. Staff can see the usage rate, and the used amount over the total voucher distributed for customers.

Usage History
Total Used Vouchers: 164

All Years All Months Filter

VOUCHER	CUSTOMER	EXPIRY	USED AT	ISSUED BY
NEWUSER10 #366	Chun Yao Ting Customer #88	2026-02-07	2026-01-08 16:05	—
BOOKING10 #122	Reyhan Shafiq bin Mansor Customer #30	2026-03-07	2026-01-07 13:56	—
NEWUSER10 #25	Amirah Najwa binti Sulaiman Customer #7	2026-07-06	2026-01-06 18:22	—
LOYALTY20 #326	Asyraf Hakim bin Halim Customer #78	2026-06-06	2026-01-06 15:03	—
NEWUSER10 #339	Aleeya Najihah binti Bakar Customer #81	2026-06-06	2026-01-06 13:39	—
LOYALTY20 #220	Balqis Amira binti Rashid Customer #55	2026-03-06	2026-01-06 09:33	—
NEWUSER10 #318	Irfan Haziq bin Mansor Customer #76	2026-04-05	2026-01-05 17:37	—
BOOKING10 #310	Aiman Akmal bin Rashid Customer #74	2026-06-04	2026-01-04 17:34	—
BOOKING10 #290	Qashrina Irdina binti Iskandar Customer #71	2026-07-04	2026-01-04 14:31	—
LOYALTY20 #77	Rifqi Danesh bin Yusof Customer #20	2026-08-03	2026-01-03 16:22	—

Showing 1 to 10 of 164 results

< 1 2 3 4 5 6 7 8 9 10 ... 16 17 >

Staff can also view all usage history in this page, while each history record contains the voucher type, voucher ID, customer name, expiry date and when the voucher is used. For example, if a customer just used a voucher for booking purposes, the new usage record will be shown here. The list is also sorted by descending timestamp of usage, which means that the latest usage of vouchers will come first. Besides, staff can also filter by years and months to view certain period voucher usage only.

SQL Statement:

```
SELECT
    type AS voucher_code,
    COUNT(*) AS total_issued,
    SUM(CASE WHEN usage = 'Used' THEN 1 ELSE 0 END)
    AS total_used,
    ROUND((SUM(CASE WHEN usage = 'Used' THEN 1 ELSE 0 END)
        / COUNT(*)) * 100)
AS usage_percentage
FROM vouchers
GROUP BY type;
```

Explanation:

The statement uses GROUP BY techniques to calculate the usage rate for each voucher type by count and sum.

SQL Statement:

```
SELECT COUNT(*) AS total_used
FROM rentals
WHERE
    voucher_id IS NOT NULL
    AND YEAR(created_at) = ?
    AND MONTH(created_at) = ?;
```

Explanation:

The statement retrieves the total count of used voucher that fulfil the year and month filter, by default it will show all record counts.

SQL Statement:

```
SELECT
    v.type AS voucher_code,
    v.id AS voucher_id,
    u.name AS customer_name,
    c.id AS customer_id,
    v.expiry_date,
    r.created_at AS used_at,
    COALESCE(s.name, '-') AS issued_by
FROM vouchers v
JOIN rentals r ON r.voucher_id = v.id
JOIN customers c ON v.customer_id = c.id
JOIN users u ON c.user_id = u.id
LEFT JOIN staff st ON v.issued_by = st.id
LEFT JOIN users s ON st.user_id = s.id
WHERE
    v.usage = 'Used'
ORDER BY r.created_at DESC;
```

Explanation:

The statement uses JOIN to retrieve the history of used vouchers, including customer and rental details. The voucher that is used most recently will come first (latest rental). By default it will show all vouchers usage history, as filter is not selected yet.

Usage History

Total Used Vouchers: 2

2025



January



Filter

Clear

VOUCHER	CUSTOMER	EXPIRY	USED AT	ISSUED BY
NEWUSER10 #257	Amirul Zikri bin Rahman Customer #64	2025-08-24	2025-01-24 11:52	—
LOYALTY20 #185	Damia Batrisyia binti Shahrul Customer #47	2025-04-06	2025-01-06 17:06	—

SQL Statement:

```
SELECT
    v.type AS voucher_code,
    v.id AS voucher_id,
    u.name AS customer_name,
    c.id AS customer_id,
    v.expiry_date,
    r.created_at AS used_at,
    COALESCE(s.name, '-') AS issued_by
FROM vouchers v
JOIN rentals r ON r.voucher_id = v.id
JOIN customers c ON v.customer_id = c.id
JOIN users u ON c.user_id = u.id
LEFT JOIN staff st ON v.issued_by = st.id
LEFT JOIN users s ON st.user_id = s.id
WHERE
    v.usage = 'Used'
    AND YEAR(r.created_at) = ?
    AND MONTH(r.created_at) = ?
ORDER BY r.created_at DESC;
```

Explanation:

The statement uses JOIN to retrieve the history of used vouchers, including customer and rental details. The voucher that is used most recently will come first (latest rental). The voucher retrieved will be filtered by year and month selected by staff.

Available Voucher Types

NEWUSER10

New User Discount: 10% off on first booking. Perfect for welcoming new customers to our service.

LOYALTY20

Loyalty Reward: 20% off for returning customers. Show appreciation for customer loyalty and encourage repeat bookings.

BOOKING10

Booking Reward: 10% off for general bookings. Useful for seasonal promotions or special events.

Assign to Customer *

Uses customer_id foreign key

Search by name, email, or ID...

Voucher Code *

Expiry Date *

Select voucher code

22/02/2026

DISCOUNT10, REBATE15, WEEKEND20, PROMO25...

Must be today or later.

Usage Status

Issued By

Unused

Current staff account

New vouchers start as Unused.

Captured automatically on save.

Cancel

Create Voucher

By clicking Issue Vouchers button at voucher list, staff will be redirected to this page. At the top section, the interface shows the available voucher types and their description. Staff may search the customer's name or find it by dropdown. Staff can also select the voucher code they wish to select by dropdown as well. Besides, staff also need to set expiry date for the voucher, after expiry date the voucher status will turn "Expired" and cannot be used anymore. By clicking create voucher, the voucher will be received at the customer side immediately. This is extremely useful for Hasta if they want to rebate for those customers that face issues during the rental period such as problematic cars, and rebate for customers who are very loyal and do a lot of bookings in the history.

177

Assign to Customer * Uses customer_id foreign key

#92 — Evelyn Ang
evelynang@graduate.utm.my

Select voucher code ▼

03/03/2026

DISCOUNT10, REBATE15, WEEKEND20, PROMO25... Must be today or later.

First, the staff need to search the customer to be assigned by entering their name substring.

SQL Statement:

```
SELECT
    u.name, u.email, c.id AS customer_id, c.ic_passport
FROM customers c
JOIN users u ON c.user_id = u.id
WHERE
    u.name LIKE CONCAT('%', ?, '%')
    OR u.email LIKE CONCAT('%', ?, '%')
    OR c.id LIKE CONCAT('%', ?, '%')
LIMIT 10;
```

Explanation:

The statement uses LIKE technique to retrieve all customer with their name, email address or ID matching the substring pattern of the input.

SQL Statement:

```
INSERT INTO vouchers (
    customer_id, type, expiry_date,
    issued_by, usage, created_at, updated_at)
VALUES (?, ?, ?, ?, 'Unused', NOW(), NOW());
```

Explanation:

The statement insert voucher into the customer account selected by staff.

1.7.21 Staff List

NAME	EMAIL	POSITION	JOINED DATE	ACTIONS
John Doe	john@gmail.com	Staff	Jan 08, 2026	 
Amirul Hakim bin Ismail	amirul.hakim@hasta.utm.my	Staff	Dec 26, 2024	 
Nadia Syafiqah binti Ahmad	nadia.syafiqah@hasta.utm.my	Staff	Dec 18, 2024	 
Khairul Azman bin Zainuddin	khairul.azman@hasta.utm.my	Staff	Apr 13, 2024	 
Danial Ariff bin Ramli	danial.ariff@hasta.utm.my	Staff	Dec 28, 2023	 
Nurul Ain binti Mohd Yusof	nurul.ain@hasta.utm.my	Staff	Dec 15, 2023	 

For admin, they can view the staff list at the staff tab. In the staff list, admin can view staff details such as their name, email, position and joining date.

SQL Statement:

```
SELECT
    s.id AS staff_id, u.id AS user_id,
    u.name, u.email, s.position,
    s.created_at AS joined_date
FROM staff s
JOIN users u ON s.user_id = u.id
ORDER BY s.created_at DESC;
```

Explanation:

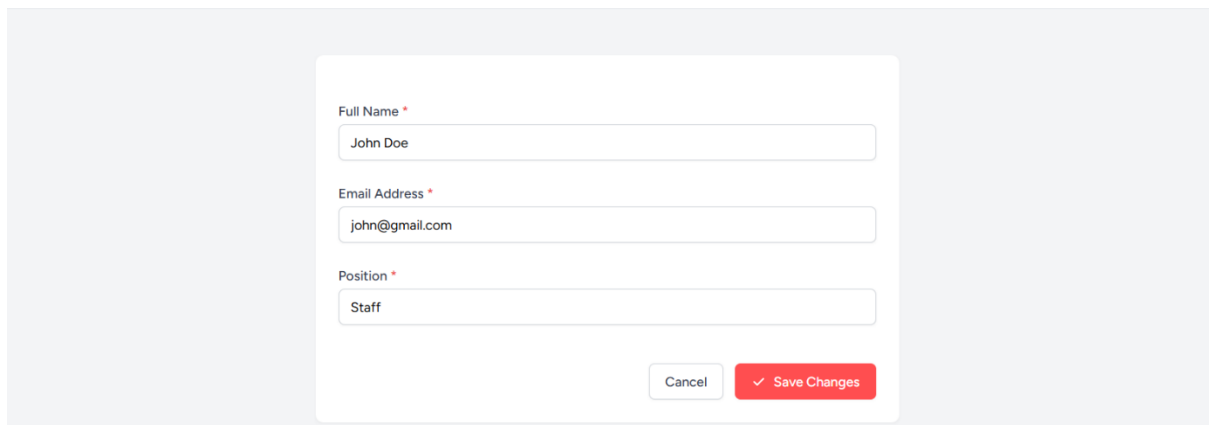
The statement retrieves all staff record, and the newly joined staff record is shown first.

SQL Statement:

```
UPDATE staff
SET
    status = 'Inactive', updated_at = NOW()
WHERE id = ?;
```

Explanation:

The statement updates the staff status to Inactive after pressing delete. (Soft Delete)



Admin can also edit staff details as shown in interface above.

SQL Statement:

```
SELECT
    s.id AS staff_id, u.id AS user_id,
    u.name, u.email, s.position, s.status
FROM staff s
JOIN users u ON s.user_id = u.id
WHERE s.id = ?;

UPDATE users
SET
    name = ?, email = ?, updated_at = NOW()
WHERE id = ?;

UPDATE staff
SET
    position = ?, updated_at = NOW()
WHERE id = ?;
```

Explanation:

The first statement fetches the staff data selected to be edited first for later update purpose. The second statement updates the staff details in user table, meanwhile third statement updates the staff details in staff table.

Admin can also create new staff as shown in interface above.

SQL Statement:

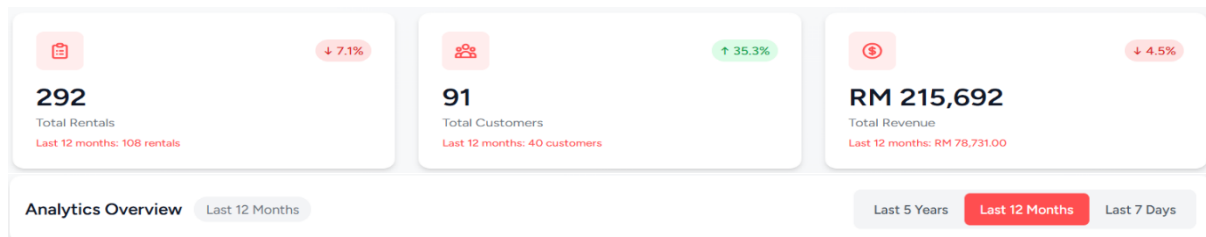
```
INSERT INTO users (  
    name, email, password, role,  
    created_at, updated_at  
)  
VALUES (?, ?, ?, 'staff', NOW(), NOW());
```

```
INSERT INTO staff (  
    user_id, position, status,  
    created_at, updated_at  
)  
VALUES (?, ?, 'Active', NOW(), NOW());
```

Explanation:

The newly created staff will create a user record first to store the name, email, password, and role, then only insert into staff record with its position and status. The sequence is important as if staff record is created first, it does not have the reference key to the user table. The statement first creates new user record then create new staff record using INSERT.

1.7.22 Admin Dashboard



At the top section, admin can see the total rentals, total customers, and total revenue of the company. Admin also able to see the performance (growth and drop rate) and instance data based on the filter [Last 7 Days: NOW() - INTERVAL 7 DAY, Last 12 Months: NOW() - INTERVAL 12 MONTH, Last 5 Years: NOW() - INTERVAL 5 YEAR]

SQL Statement:

```
SELECT
```

```
    (SELECT COUNT(*) FROM rentals) AS total_rentals,  
    (SELECT COUNT(*) FROM rentals WHERE created_at >= ?)
```

```
AS filtered_rentals;
```

```
SELECT
```

```
    (SELECT COUNT(*) FROM customers) AS total_customers,  
    (SELECT COUNT(*) FROM customers WHERE created_at >= ?)
```

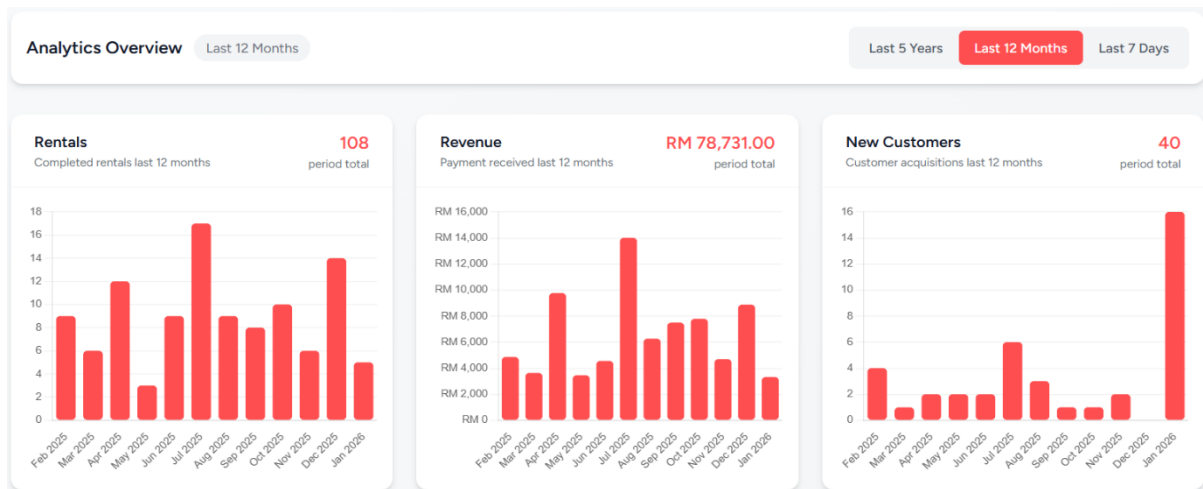
```
AS filtered_customers;
```

```
SELECT
```

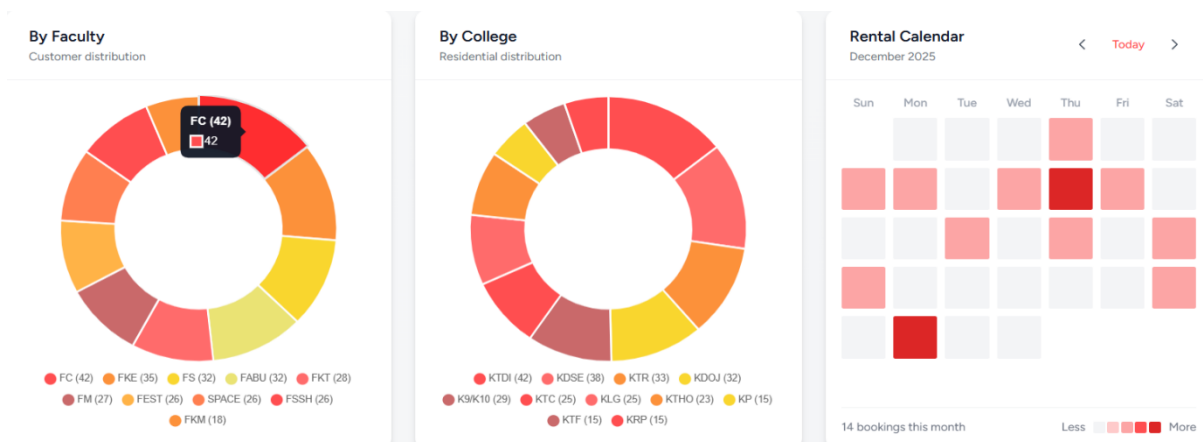
```
    (SELECT COALESCE(SUM(amount), 0) FROM payments)  
AS total_revenue,  
    (SELECT COALESCE(SUM(amount), 0) FROM payments  
WHERE created_at >= ?) AS filtered_revenue;
```

Explanation:

The first statement queries the total rental count and rental count in selected interval, the second statement queries the total customers and customers gain in selected interval, and the third statement queries the total revenue and revenue in selected interval.



Next, admin can view the analytics overview section, and be able to filter by last 5 years, last 12 months and last 7 days charts. In the rentals overview, admin can see how much booking was done in that month by hovering it and visualize which month has most or least booking as well. In the revenue overview, admin also can see which months contributed the most or least revenue and hovering it also can show the data of that month. For customer gain, admin can also view the number of customers registered for a Hasta account and hovering it also can see the instance data of that specific month. These visualizable graphs are very useful for market analysis and determine the successfulness of the marketing strategy.



Last but not least, the admin can view the marketing analysis by faculty or by college as well. The legend at the bottom will be sorted from the highest booking amount to lowest booking amount for that faculty and college. Meanwhile, by hovering it, admin also can see the clear amount of bookings done by that faculty or college customer. This is extremely useful so Hasta can rebate their customers on active faculty or college and support their marketing decisions. Furthermore, there is a booking heatmap at the right which acts like GitHub heatmap, the less in booking will be lighter in color, and more in booking will be darker in color. This is for admin to see which day has more booking, perhaps weekdays need to have more promotional events and weekends will generally have more bookings done.

I) Analytics Overview (Last 5 Years) - Displayed Yearly

SQL Statement:

```
-- Rental
SELECT YEAR(created_at) as year, COUNT(*) as count
FROM rentals
WHERE created_at >= DATE_SUB(NOW(), INTERVAL 5 YEAR)
GROUP BY year
ORDER BY year ASC;

-- Revenue
SELECT YEAR(created_at) as year, SUM(amount) as total
FROM payments
WHERE created_at >= DATE_SUB(NOW(), INTERVAL 5 YEAR)
GROUP BY year
ORDER BY year ASC;

-- Customer Gain
SELECT YEAR(created_at) as year, COUNT(*) as count
FROM customers
WHERE created_at >= DATE_SUB(NOW(), INTERVAL 5 YEAR)
GROUP BY year
ORDER BY year ASC;
```

Explanation:

The statement queries the total number of rentals, revenues and customer gains in each year in 5 years interval uses COUNT and GROUP BY year techniques.

II) Analytics Overview (Last 12 Months) - Displayed Monthly

SQL Statement:

```
-- Rental
SELECT DATE_FORMAT(created_at, '%Y-%m') as month,
COUNT(*) as count
FROM rentals
WHERE created_at >= DATE_SUB(NOW(), INTERVAL 12 MONTH)
GROUP BY month
ORDER BY month ASC;

-- Revenue
SELECT DATE_FORMAT(created_at, '%Y-%m') as month,
SUM(amount) as total
FROM payments
WHERE created_at >= DATE_SUB(NOW(), INTERVAL 12 MONTH)
GROUP BY month
ORDER BY month ASC;

-- Customer Gain
SELECT DATE_FORMAT(created_at, '%Y-%m') as month,
COUNT(*) as count
FROM customers
WHERE created_at >= DATE_SUB(NOW(), INTERVAL 12 MONTH)
GROUP BY month
ORDER BY month ASC;
```

Explanation:

The statement queries the total number of rentals, revenues and customer gains in each month in 12 months interval uses COUNT and GROUP BY month techniques.

III) Analytics Overview (Last 7 Days) - Displayed Daily

SQL Statement:

```
-- Rental
SELECT DATE(created_at) as date, COUNT(*) as count
FROM rentals
WHERE created_at >= DATE_SUB(NOW(), INTERVAL 7 DAY)
GROUP BY DATE(created_at)
ORDER BY date ASC;

-- Revenue
SELECT DATE(created_at) as date, SUM(amount) as total
FROM payments
WHERE created_at >= DATE_SUB(NOW(), INTERVAL 7 DAY)
GROUP BY DATE(created_at)
ORDER BY date ASC;

-- Customer Gain
SELECT DATE(created_at) as date, COUNT(*) as count
FROM customers
WHERE created_at >= DATE_SUB(NOW(), INTERVAL 7 DAY)
GROUP BY DATE(created_at)
ORDER BY date ASC;
```

Explanation:

The statement queries the total number of rentals, revenues and customer gains in each day in 7 days interval uses COUNT and GROUP BY DATE techniques.

IV) Demographics & Marketing Analysis

SQL Statement:

```
-- Faculty Pie Chart
SELECT
    faculty, COUNT(*) as count
FROM customers
WHERE faculty IS NOT NULL
GROUP BY faculty
ORDER BY count DESC;

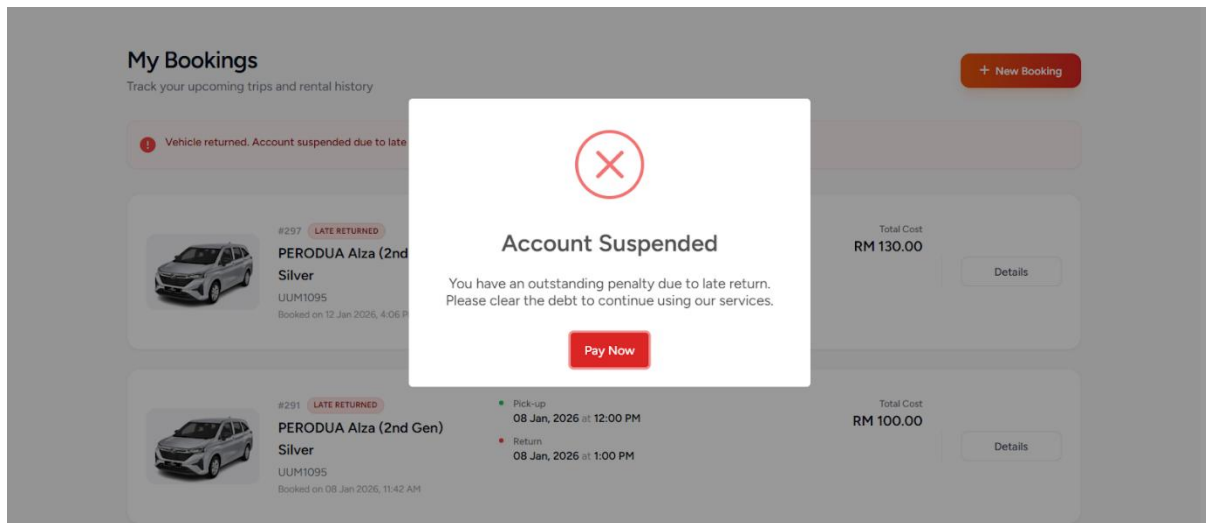
-- College Booking Pie Chart
SELECT
    college, COUNT(*) as count
FROM customers
WHERE college IS NOT NULL
GROUP BY college
ORDER BY count DESC;

-- Rental Calendar Heatmap
SELECT
    DATE(pickup_date) as date, COUNT(*) as count
FROM rentals
WHERE
    YEAR(pickup_date) = ? AND MONTH(pickup_date) = ?
GROUP BY DATE(pickup_date);
```

Explanation:

The first statement queries the count of each faculty by GROUP BY techniques, faculty with most rental is displayed first, the second statement queries the count of each college by GROUP BY techniques, college with the most rental is displayed first, the third statement queries the count of each rental in each day in selected time interval by using GROUP BY techniques.

1.7.23 Late Return Scenario



SQL Statement:

```
SELECT
    r.id AS rental_id, u.name AS customer_name,
    c.model AS car_model, c.car_plate,
    r.return_date, r.return_time,
    DATEDIFF(NOW(), CONCAT(r.return_date, ' ', r.return_time))
    AS days_overdue
FROM rentals r
JOIN customers cust ON r.customer_id = cust.id
JOIN users u ON cust.user_id = u.id
JOIN cars c ON r.car_id = c.id
WHERE
    r.status NOT IN ('Pending', 'Returned', 'Completed',
    'Cancelled', 'Rejected')
    AND CONCAT(r.return_date, ' ', r.return_time) < NOW()
ORDER BY days_overdue DESC;
```

Explanation:

This statement is used to detect the late return of customer.

SQL Statement:

```
UPDATE rentals
SET
    status = 'Returned', return_date = ?,
    return_time = ?, updated_at = NOW()
WHERE id = ?;

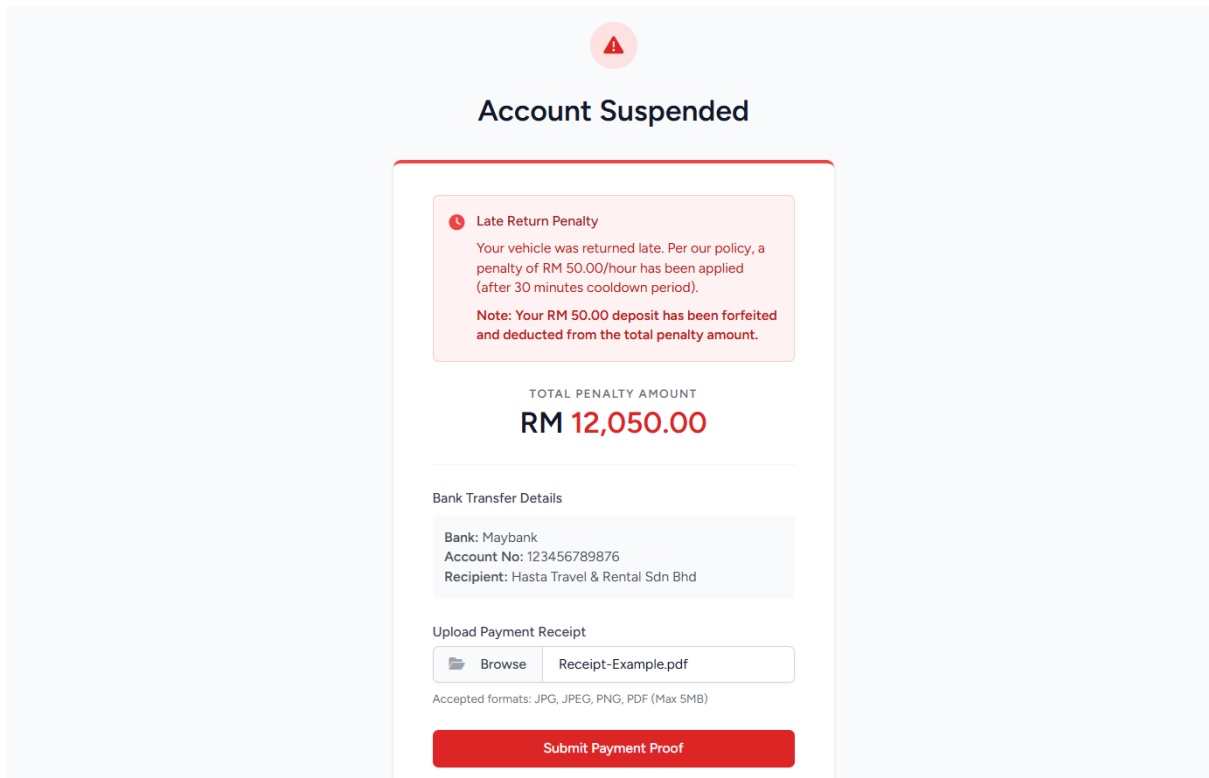
UPDATE cars
    SET status = 'Available', updated_at = NOW()
WHERE id = ?;

UPDATE customers
    SET status = 'Blacklisted', updated_at = NOW()
WHERE id = ?;

INSERT INTO penalties (
    customer_id, rental_id, amount,
    created_at, updated_at
)
VALUES (?, ?, ?, NOW(), NOW());
```

Explanation:

For late return case, the rentals need to be updated to Returned first after the customer issue return, then cars also need to be updated as Available just like normal return case, however, customer status need to be updated to become Blacklisted, and new penalty record is created in database.



The image shows a web interface for an account suspension notification. At the top, there is a red warning icon and the title "Account Suspended". Below this, a red-bordered box contains a "Late Return Penalty" section with a clock icon, explaining that a penalty of RM 50.00/hour has been applied after a 30-minute cooldown period, and that the RM 50.00 deposit has been forfeited. Below this box, the "TOTAL PENALTY AMOUNT" is displayed as "RM 12,050.00". Underneath, the "Bank Transfer Details" section lists the bank as Maybank, account number 123456789876, and recipient Hasta Travel & Rental Sdn Bhd. An "Upload Payment Receipt" section includes a "Browse" button and a text field containing "Receipt-Example.pdf". Below this is a note about accepted formats: JPG, JPEG, PNG, PDF (Max 5MB). At the bottom, there is a red "Submit Payment Proof" button.

The suspended customer need to pay the penalty first to be able to book again.

SQL Statement:

```
UPDATE penalties
SET
    receipt = ?,
    updated_at = NOW()
WHERE id = ?;
```

Explanation:

After the customer submitted the penalty, this statement updates the penalty record with uploaded payment receipt.

Issue Penalty

RENTAL ID	CUSTOMER	DATE	AMOUNT	RECEIPT	STATUS	ACTION
#297	Aliff Helmi	23 Jan 2026 17:28	RM12,050.00	 View	Pending	ISSUE
#291	Aliff Helmi	08 Jan 2026 16:47	RM150.00	 View	Verified	Issued

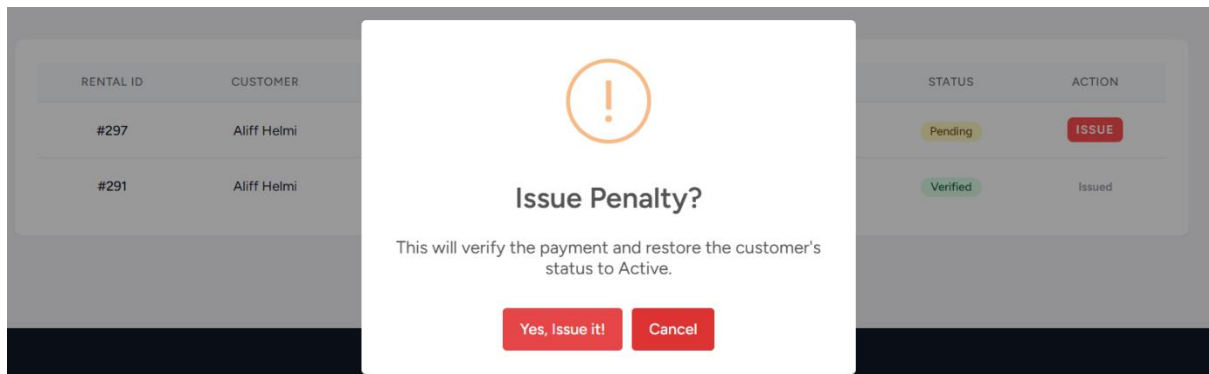
Staff can view and issue the penalty as shown in the interface above.

SQL Statement:

```
SELECT
    p.id AS penalty_id,
    r.id AS rental_id,
    u.name AS customer_name,
    p.created_at AS penalty_date,
    p.amount AS penalty_amount,
    p.receipt AS receipt_path,
    CASE
        WHEN p.approved_at IS NOT NULL THEN 'Verified'
        WHEN p.receipt IS NOT NULL THEN 'Pending'
    ELSE 'Unpaid'
    END AS status
FROM penalties p
JOIN customers cust ON p.customer_id = cust.id
JOIN users u ON cust.user_id = u.id
JOIN rentals r ON p.rental_id = r.id
ORDER BY p.created_at DESC;
```

Explanation:

The statement retrieves all penalty record, and the latest penalty record comes first.



Staff can choose to approve the penalty payment or not.

SQL Statement:

```
UPDATE penalties
```

```
SET
```

```
    approved_at = NOW(),
```

```
    issued_by = ?,
```

```
    updated_at = NOW()
```

```
WHERE id = ?;
```

```
UPDATE customers
```

```
SET status = 'Active', updated_at = NOW()
```

```
WHERE id = ?;
```

Explanation:

After the staff approve the penalty payment, the penalties need to be updated with the approval time and issued by which staff. The customer status also will be updated in database by this statement to become 'Active' again.

1.8 Summary

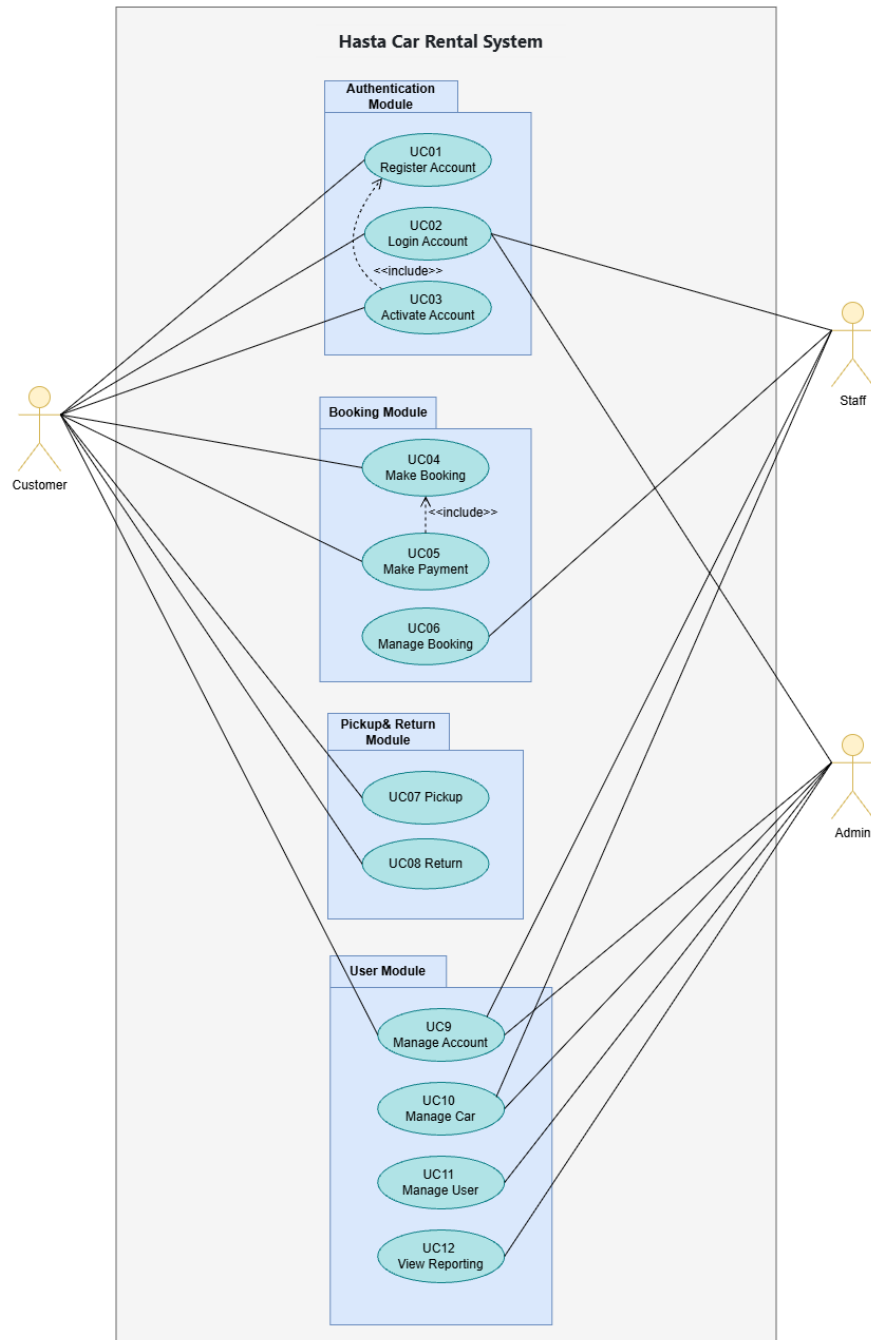
On the whole, it can be said that the logical database design of the Hasta Car Rental System has been completed in the form of well-structured relational tables that are viable enough to support the activities of the actual system. The final logical database design will ensure that important information regarding user roles, customer activation records and bookings, payment records, etc., can be stored and linked appropriately in the entire process of renting.

The normalization process helps ensure that the database is clean and accurate by reducing the probability of duplicate information and ensuring that there is no inconsistency in the information being inserted, updated, and deleted. The design can be easily implemented and maintained with an updated data dictionary that includes definitions of tables and attributes.

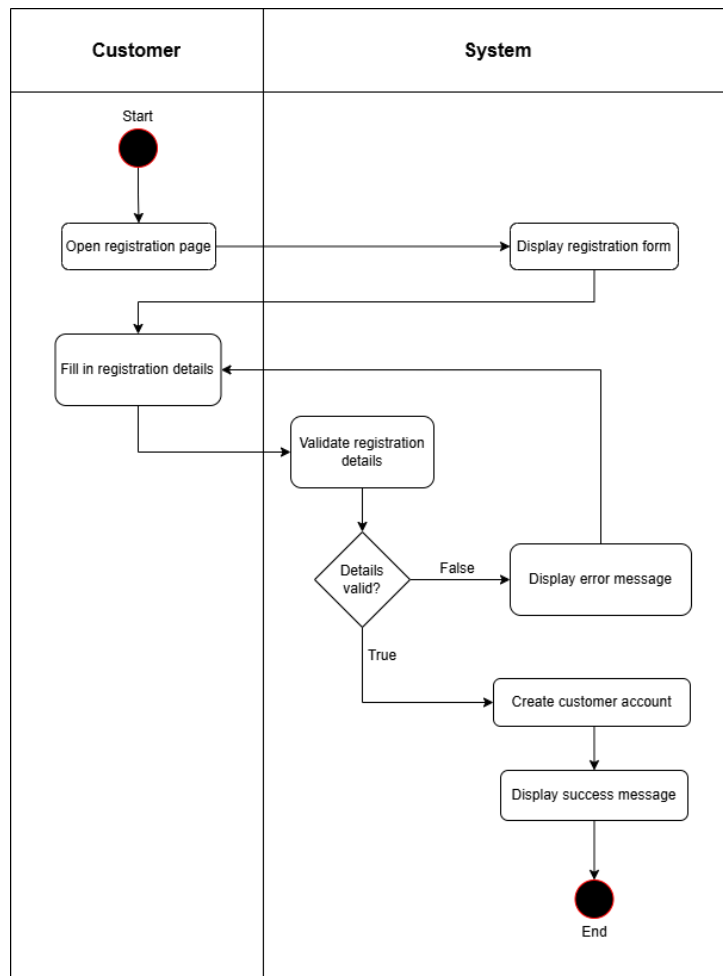
Lastly, it can be said that the proposed SQL statements prove that the logical database design is feasible and can be actually implemented, and the interface-to-transaction mapping proves that the logical model is viable enough to support the necessary activities of customers, staff, and admins. This is the end of the logical design, and the project can now be said to be ready to be implemented and tested in the database.

APPENDIX

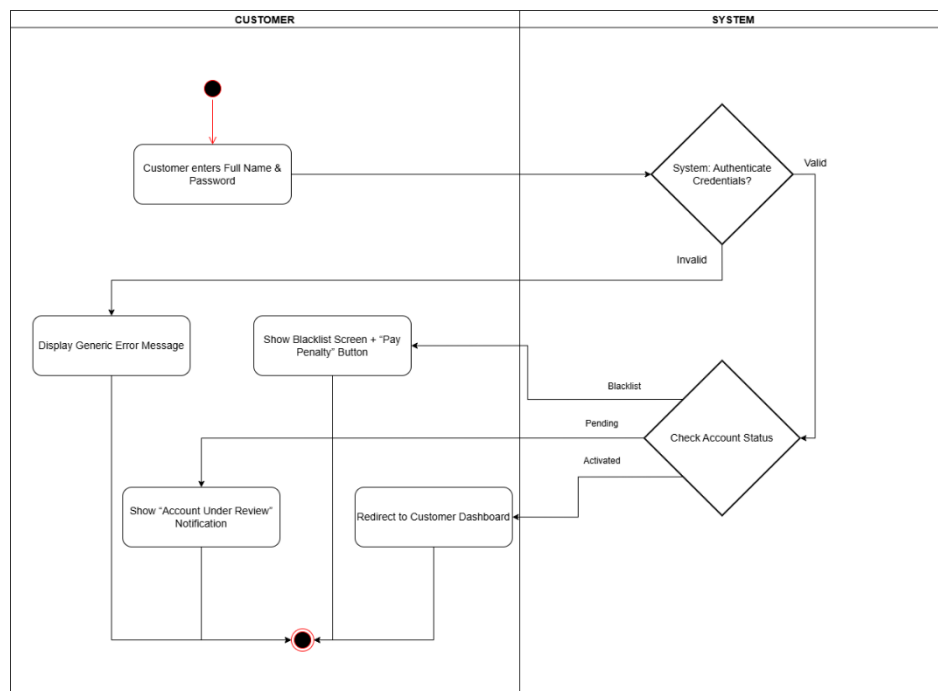
1) Use Case Diagram



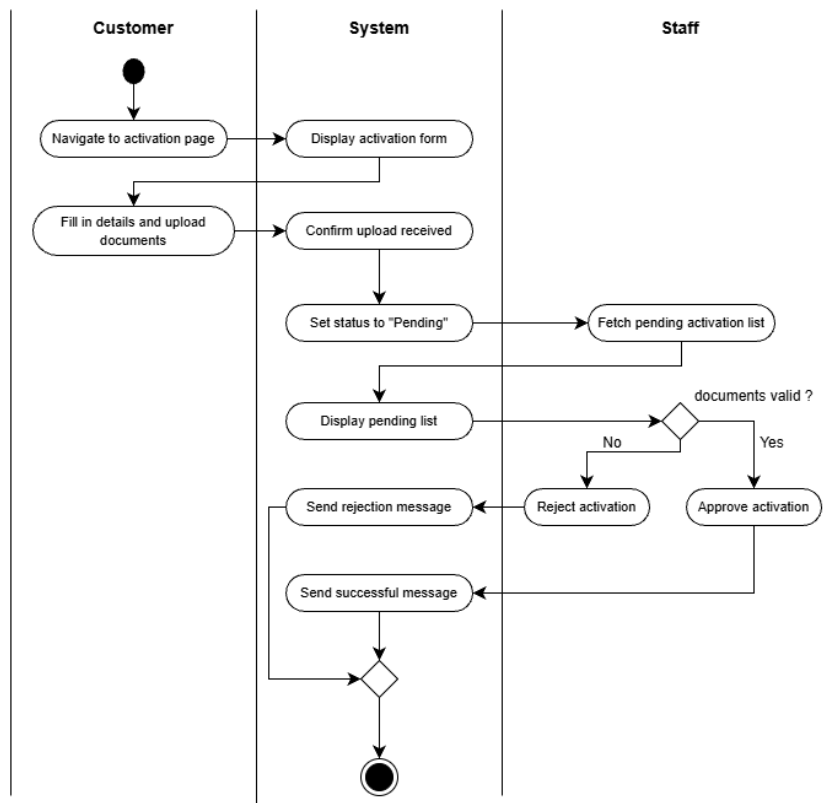
2) Activity Diagram of UC01 Register Account



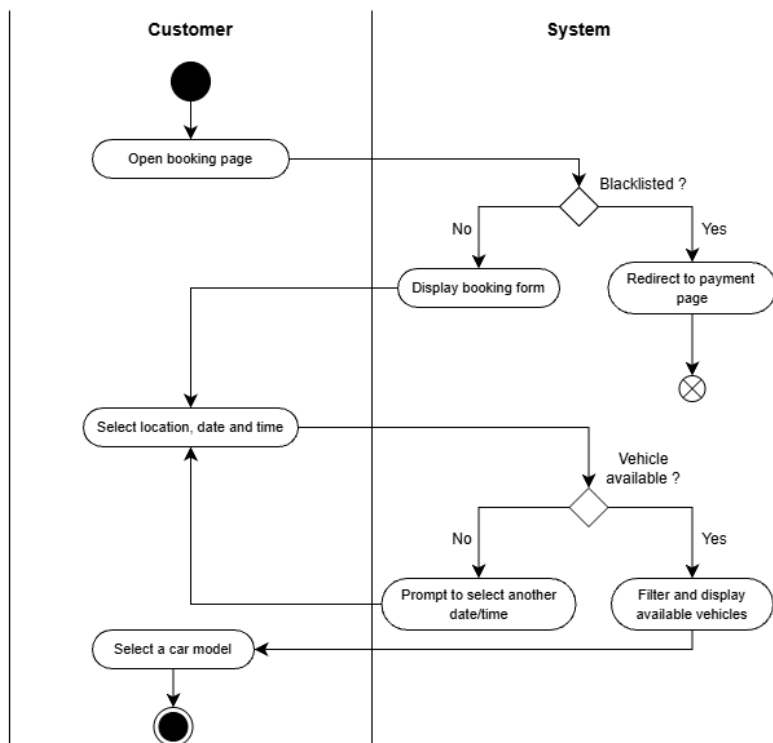
3) Activity Diagram of UC02 Login Account



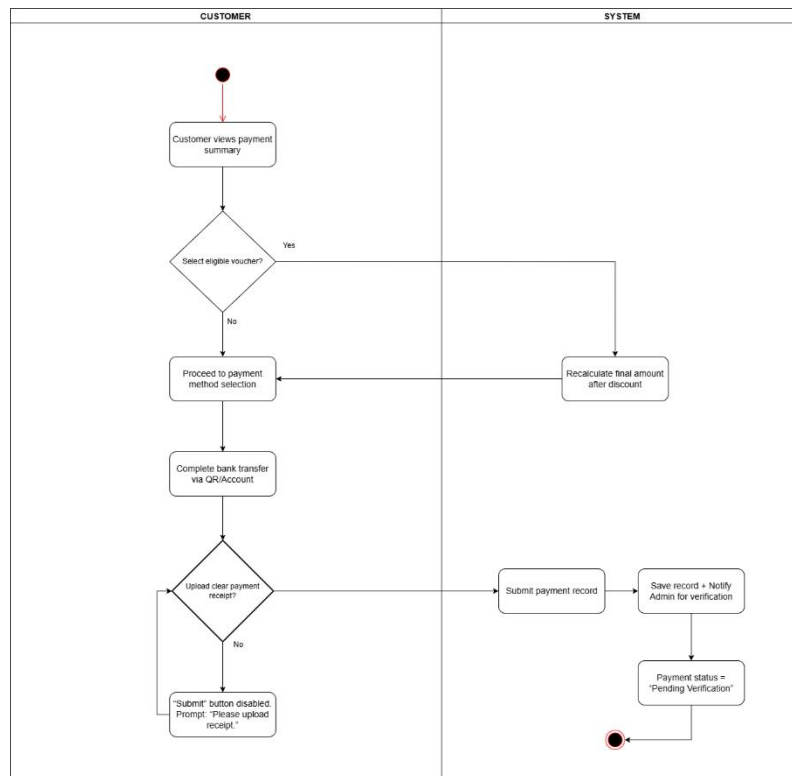
4) Activity Diagram of UC03 Activate Account



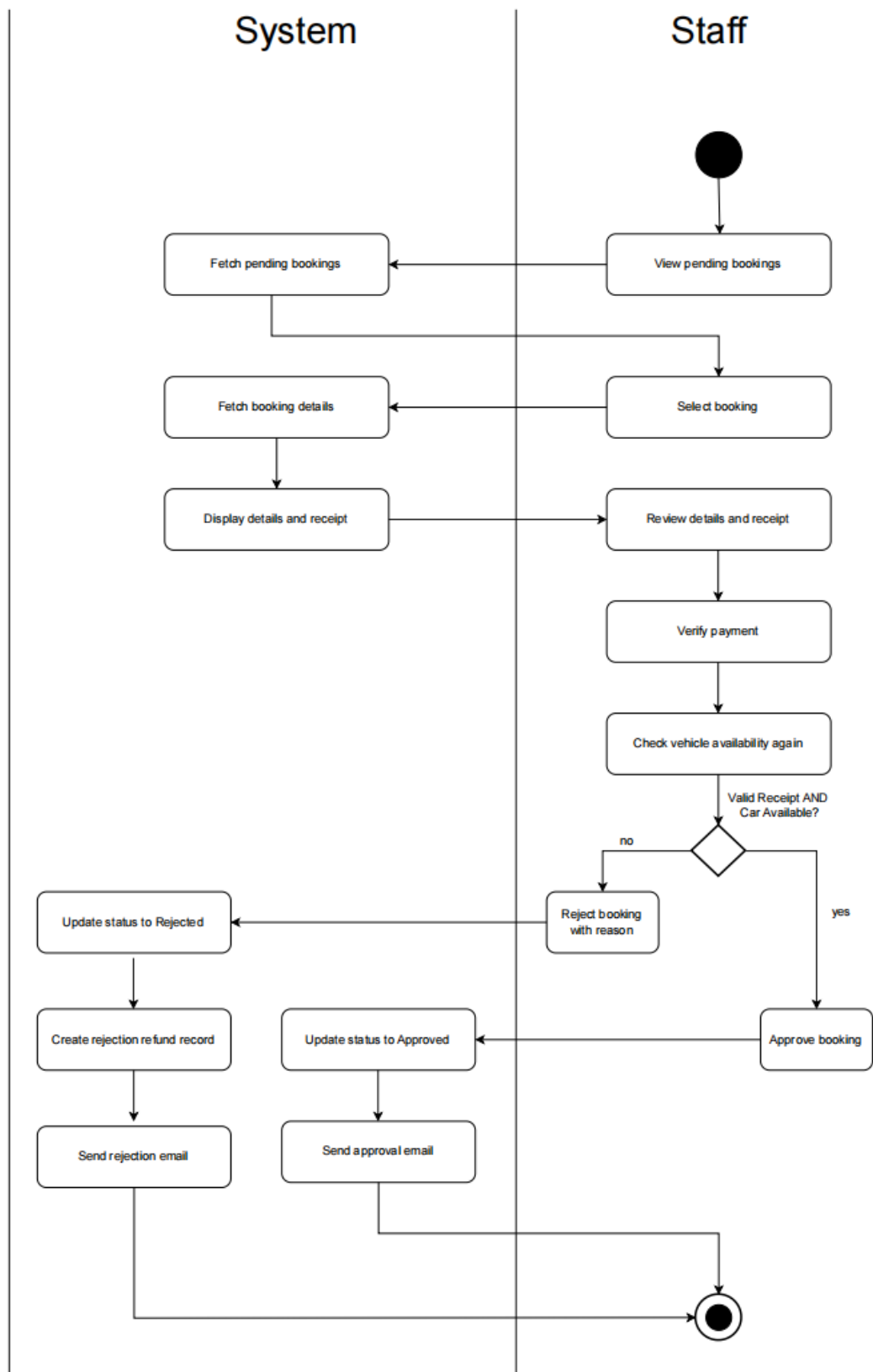
5) Activity Diagram of UC04 Make Booking



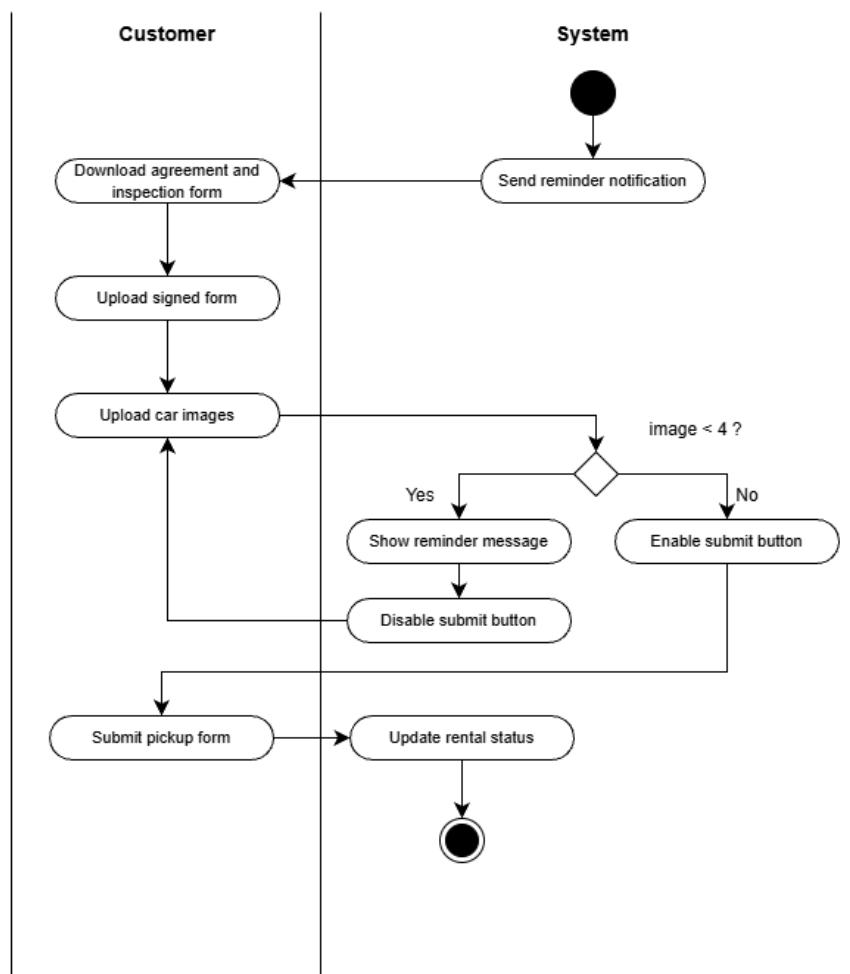
6) Activity Diagram of UC05 Make Payment



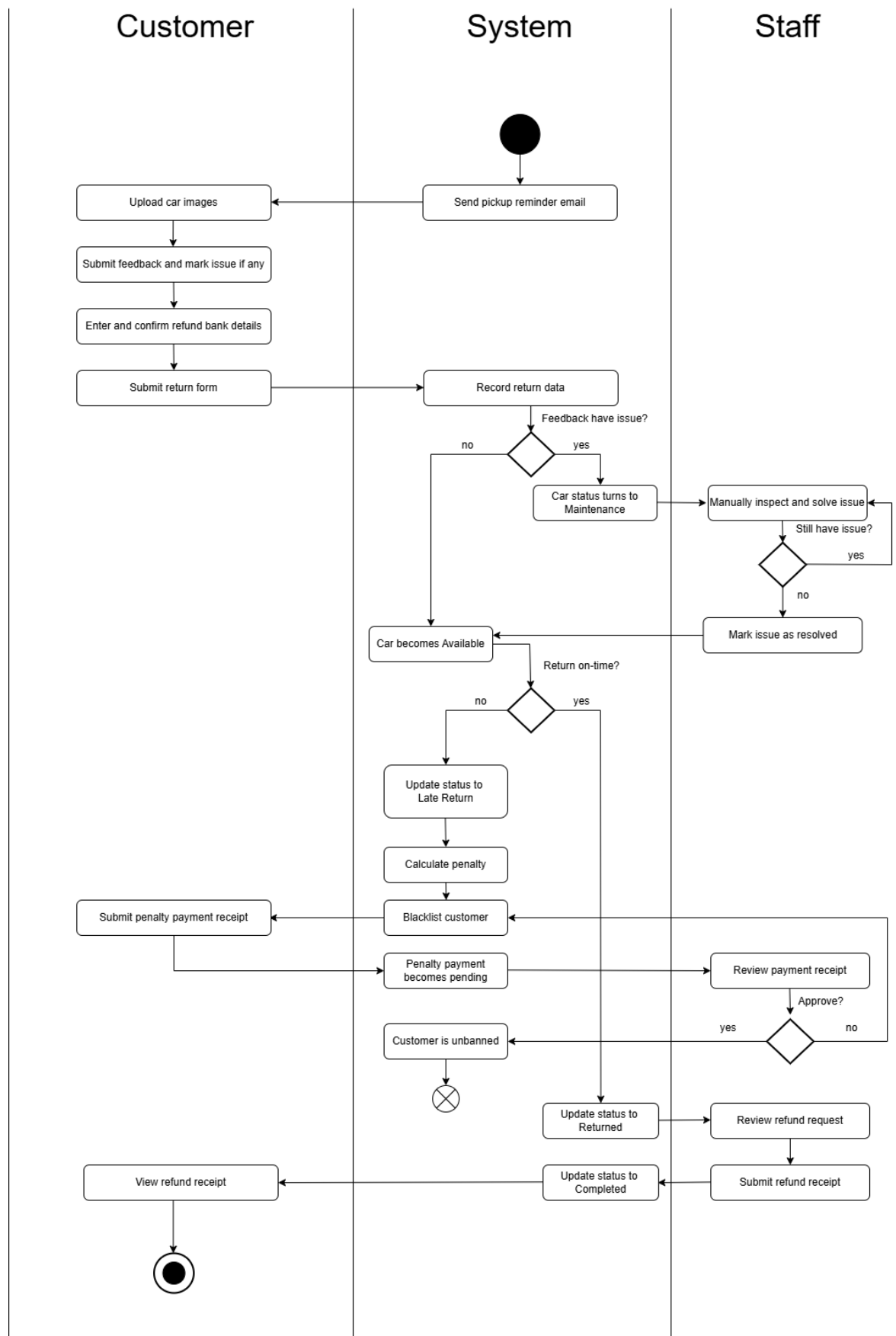
7) Activity Diagram of UC06 Manage Booking



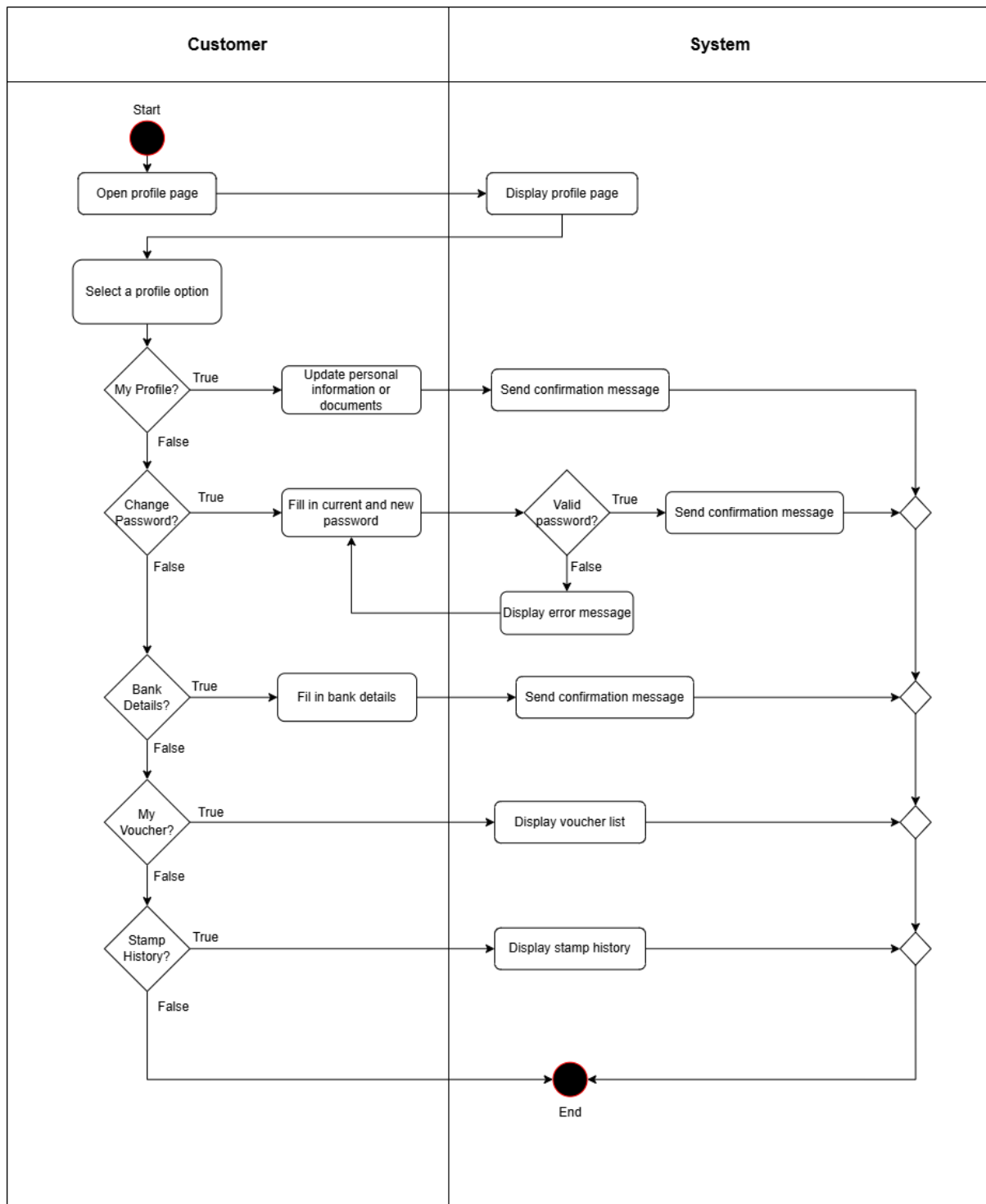
8) Activity Diagram of UC07 Pickup



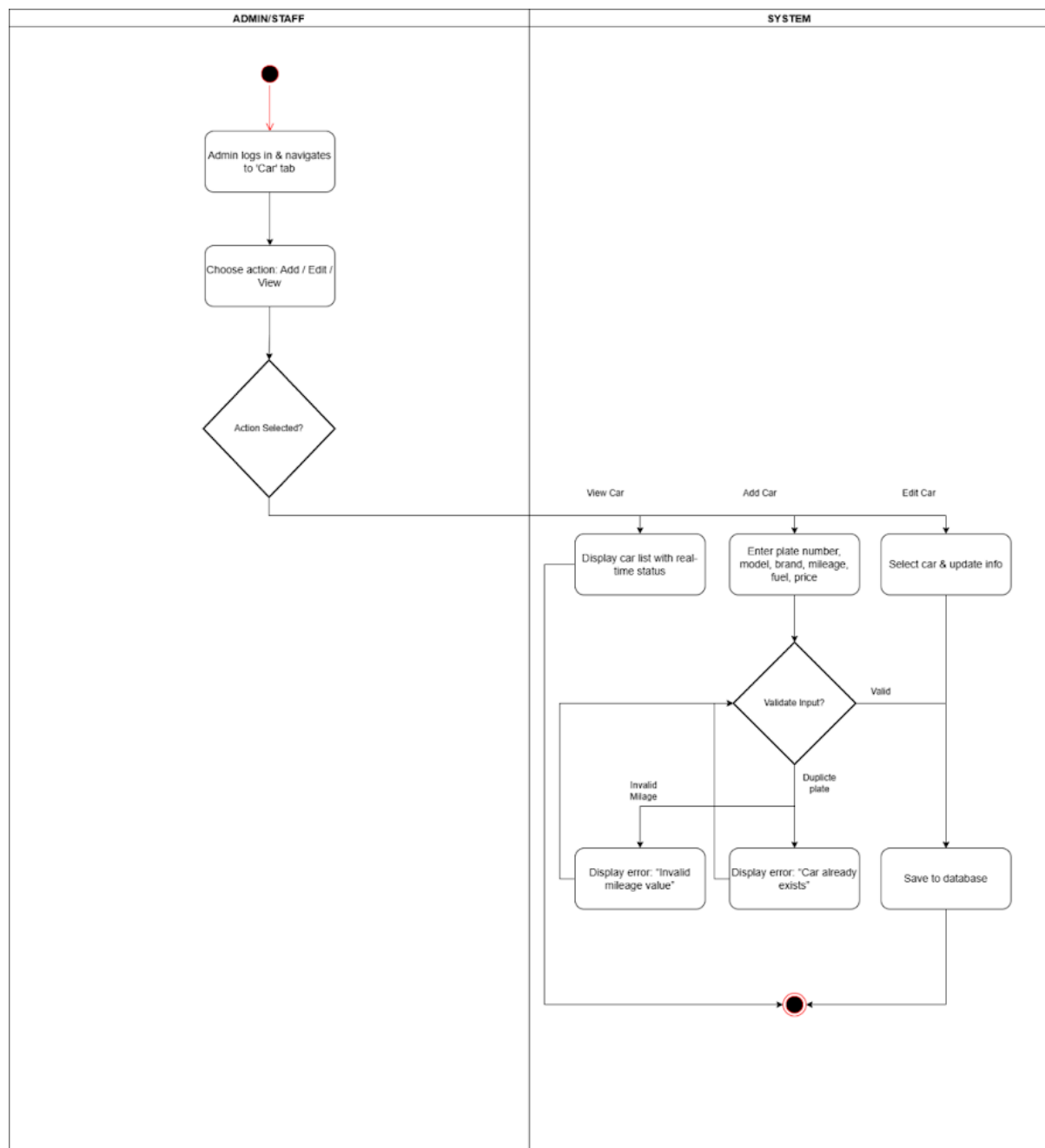
9) Activity Diagram of UC08 Return



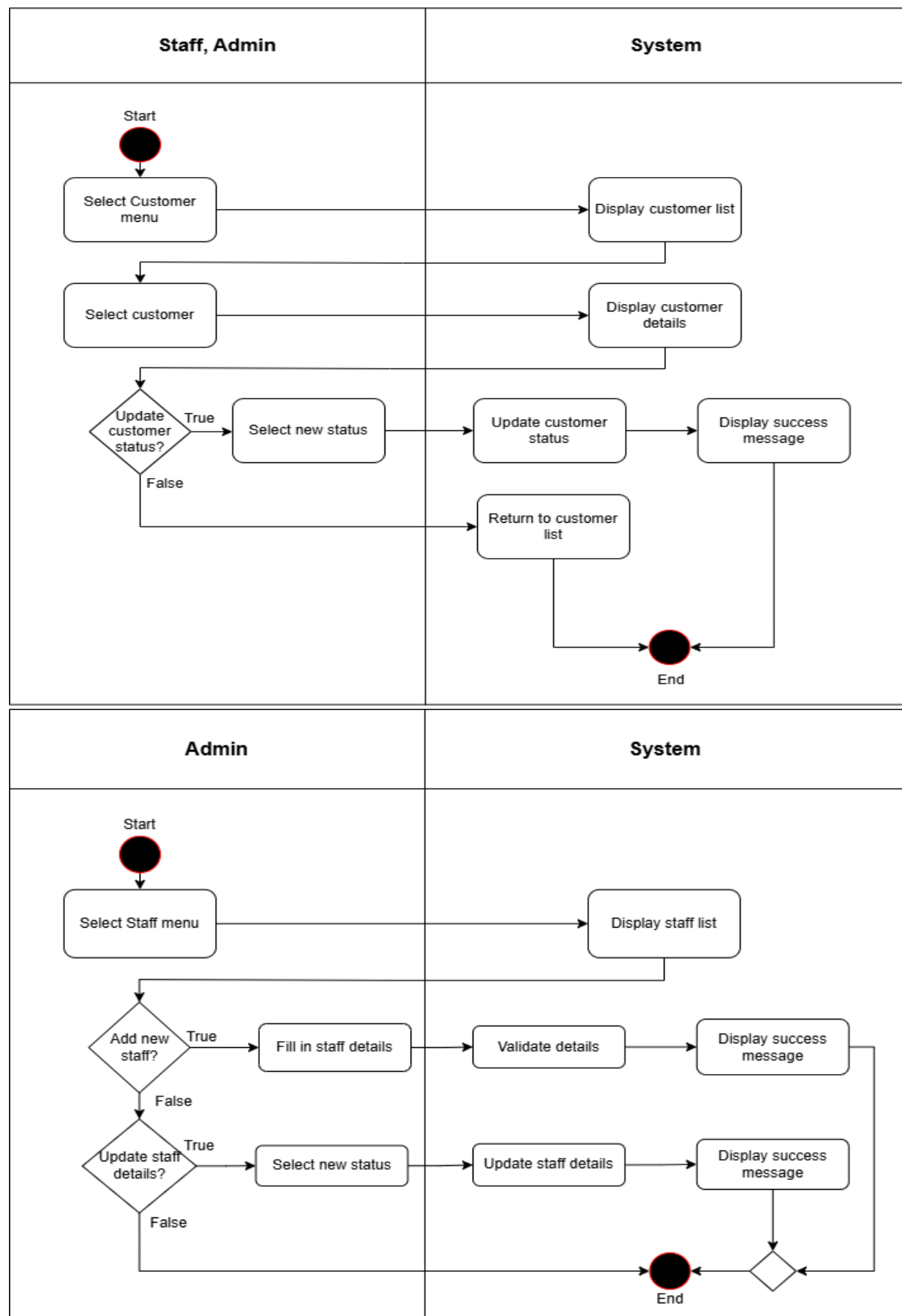
10) Activity Diagram of UC09 Manage Account



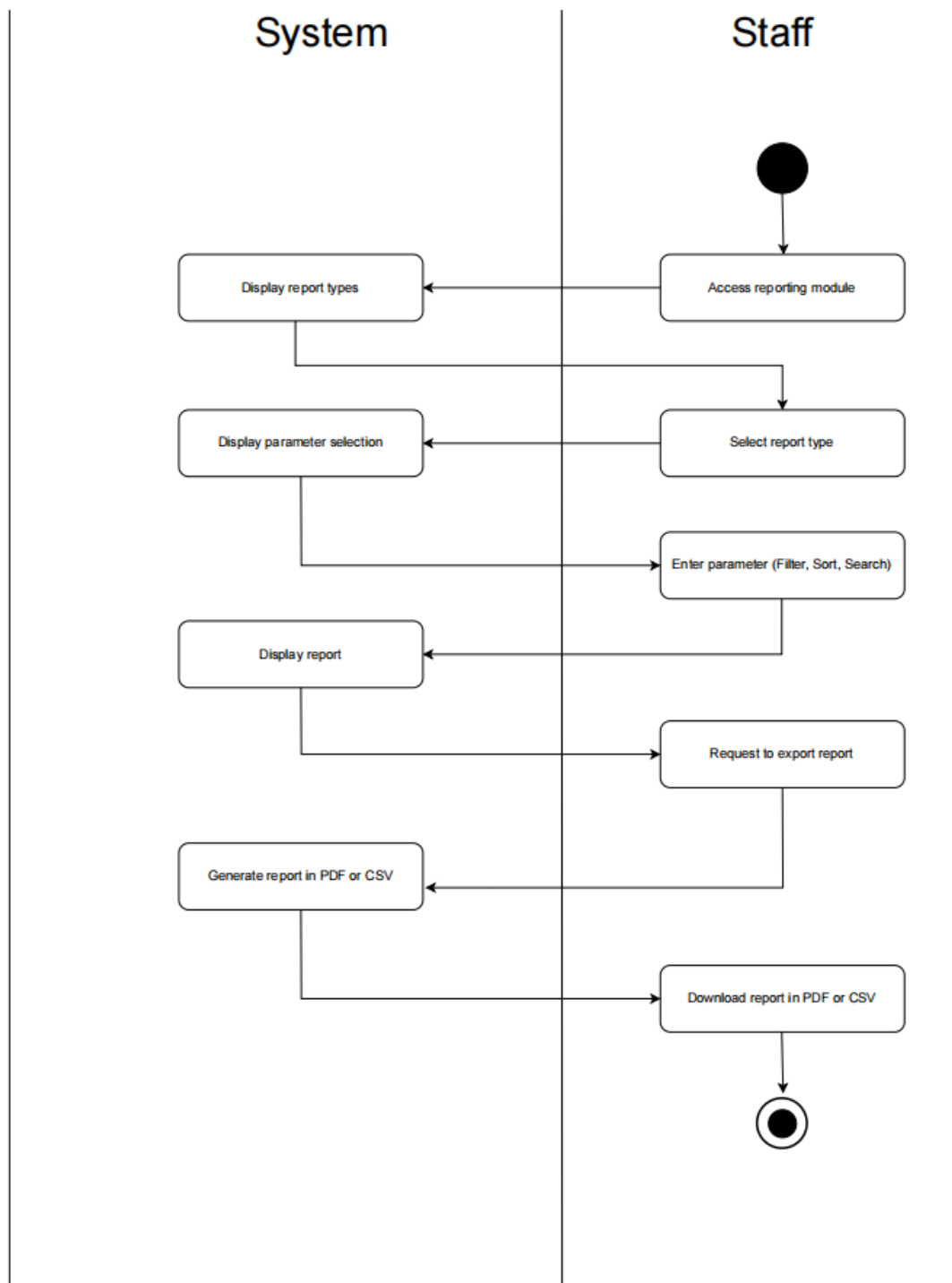
11) Activity Diagram of UC10 Manage Car



12) Activity Diagram of UC11 Manage User



13) Activity Diagram of UC12 View Reporting



Meeting Log #1

Date: 26 December 2025

Time: 9:00 p.m.

Member Involved:

1. Chun Yao Ting
2. Evelyn Ang
3. Mirza As-Siddiq bin Tohari
4. Muhammad Aliff bin Helmi

Objective:

To identify phase 3 database logical design deliverables and discuss the task allocation among members.

Discussion Summary:

We had discussed how the conceptual ERD and business rules that were developed in phase 2 would be refined to get an updated set of business rules, which then will be transformed to a logical ERD by deleting non-relational characteristics and by making sure that keys, attributes, and relationships are right. Next, we talked about updating the data dictionary that will match to the logical ERD with the PK/FK constraints and definitions of the field. Then we discussed how to perform the normalization. After that, we explained how the final relational database schemas will be developed and prepared the required SQL statements that will demonstrate how a database will be implemented.

	Task Distributed	Person-In-Charge
1.	Updated Business Rule	Mirza As-Siddiq bin Tohari
2.	Logical ERD	Evelyn Ang
3.	Updated Data Dictionary	Chun Yao Ting
4.	Normalization	- Chun Yao Ting - Evelyn Ang
5.	Relational DB Schemas	Muhammad Aliff bin Helmi
6.	SQL Statements (DDL & DML)	All members

